NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

TOMÁS FILIPE PINA GABRIEL

BSc in Computer Science

OVERLORD: CENTRALIZED CONTROL IN HIGHLY DECENTRALIZED SYSTEMS

MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

Draft: March 29, 2026

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

OVERLORD: CENTRALIZED CONTROL IN HIGHLY DECENTRALIZED SYSTEMS

TOMÁS FILIPE PINA GABRIEL

BSc in Computer Science

Adviser: João Carlos Antunes Leitão
Associate Professor, NOVA University Lisbon

Co-adviser: João Pedro Alves Bordalo
Phd Student, NOVA University Lisbon

MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

Draft: *March 29, 2026*

Abstract

The Web3 movement is currently pushing for a more decentralized (and democratic) architecture for the Internet. As such, decentralized systems are gaining popularity once more. Increasing the level of abstraction, Swarm Computing, where highly heterogeneous devices can cooperate directly and in a mostly autonomous fashion, demands for highly flexible and robust decentralized solutions, including distributed and decentralized membership, as well as communication protocols.

The peer-to-peer literature is full of different proposals for such protocols. However, the majority assumes a static configuration, meaning that the parameters that govern the operation of the system are defined at bootstrap time, based on expectations on the operational conditions of the system throughout its entire lifetime. These remain unchanged even if these conditions change significantly, making the system potentially inefficient or even unavailable. The autonomic computing paradigm has been proposed many years ago as a way to achieve a generic architecture to build systems that monitor themselves and their environment to make decisions on how to reconfigure themselves, and apply those reconfiguration actions. Solutions focused on autonomic computing are usually fully decentralized, or fully centralized, leaving space for the study of hybrid solutions.

In this work we design a centralized (or partially centralized) component that can make management decisions for a large-scale decentralized system, which we named Overlord. We explore the idea of autonomic computing to govern peer-to-peer protocol parameters. To this end, we create a design for the architecture, implement it using the Babel-Swarm framework, and evaluate its performance against static configuration alternatives in a message dissemination case study.

"Swarm computing Airdropped" what does this mean? Should I just remove the reference?

Keywords: Decentralized Systems, Autonomic Computing, Peer-To-Peer, Self-Adaptive Systems

Resumo

Sistemas descentralizados têm vindo a ganhar popularidade de novo, não apenas devido ao sucesso das *blockchains* no contexto das criptomoedas, mas também pelo movimento *Web3* que promove uma arquitetura mais descentralizada (e democrática) para a internet. Elevando o nível de abstração, *Swarm Computing*, na qual dispositivos altamente heterogéneos podem cooperar diretamente de forma maioritariamente autónoma, exige soluções descentralizadas flexíveis e robustas, nomeadamente protocolos distribuidos e descentralizados de gestão de membros e comunicação.

A literatura de *peer-to-peer* propõe diversas abordagens para tais protocolos. No entanto, a grande maioria assume uma configuração estática, o que significa que os parâmetros que gerem a operação do sistema são definidos no momento do seu arranque e baseiam-se em expectativas sobre as condições do sistema ao longo do seu tempo de vida. Isto pode resultar num funcionamento ineficiente ou até na indisponibilidade do sistema. O paradigma da computação autónoma foi proposto há bastante tempo como um padrão de arquitetura para a construção de sistemas que se monitorizam a si próprios e ao seu ambiente para tomarem decisões sobre como se reconfigurarem e as aplicam autonomamente. Soluções baseadas em computação autónoma tendem a ser altamente descentralizadas ou altamente centralizadas, o que abre espaço para o estudo de uma solução híbrida.

O principal foco é a conceção de um componente centralizado (ou parcialmente centralizado) capaz de tomar decisões num sistema descentralizado em larga escala, o qual nomeámos *Overlord*. O principal foco é a exploração do conceito de computação autónoma para a gestão de protocolos *peer-to-peer*. Para este fim, criamos o desenho da arquitetura, a sua implementação através da *framework* Babel-Swarm, e a avaliação através de um caso de estudo de partilha de mensagens, comparando a nossa solução com alternativas de parâmetros estáticos.

Palavras-chave: Sistemas Descentralizados, Computação Autónoma, Peer-To-Peer, Sistemas Auto-Adaptativos

Contents

List of Figures v

Acronyms vii

1 Introduction 1

2 Related Work 5

2.1 Decentralized Systems . . . 5

2.1.1 Peer-To-Peer Systems . . . 5

2.1.2 Membership . . . 7

2.1.3 Gossip Protocols . . . 9

2.1.4 Case Studies of Decentralized Applications . . . 11

2.1.5 Summary . . . 18

2.2 Autonomic Computing . . . 18

2.2.1 MAPE-K . . . 19

2.2.2 Monitoring . . . 20

2.2.3 Planning & Analyzing . . . 23

2.2.4 Execution . . . 26

2.2.5 Summary . . . 27

2.3 Frameworks . . . 28

2.3.1 Cactus Framework . . . 28

2.3.2 Appia Framework . . . 29

2.3.3 R-Appia Framework . . . 29

2.3.4 Rainbow Framework . . . 31

2.3.5 Babel Framework . . . 32

3 Overlord 35

3.1 Architecture . . . 35

3.2 Monitoring . . . 37

3.2.1 Integration with Babel Swarm Metrics Framework . . . 37

3.2.2 Aggregation 38
3.2.3 MON-Collect 40
3.3 Rule Engine and Rules 47
3.3.1 Rules 48
3.3.2 Rule Engine 49
3.4 Overlord Manager 52
3.4.1 Responsibilities of Overlord 55
3.5 Summary 56

4 Evaluation **58**
4.1 Message Sharing Application 58
4.1.1 Monitoring Reliability 59
4.1.2 Fanout Adaptation Process 61
4.1.3 Evaluation Scenarios 63
4.2 Summary 82

5 Conclusion **83**
5.1 Conclusions and Contributions 83
5.2 Future Work 84

Bibliography **85**

List of Figures

2.1 General Gossip-Based Communication Between Nodes 10
2.2 IBM's MAPE-K (Adapted from [27]) 20
2.3 Babel Framework Architecture 32

3.1 Architecture of Overlord and how each component interacts. 37
3.2 How Overlord Collects and Aggregates Metrics. 39
3.3 Execution of the Session Phase. 42
3.4 Example of a possible resulting tree from an arbitrary membership. 43
3.5 Example of a run where node c fails to respond to its parent within the expected time 43
3.6 Example of the Monitor-Collect phase. 45
3.7 An Example of a node's behavior when its parent fails. 46
3.8 A visual representation of why a node cannot simply retransmit its data to Overlord after its parent has failed. 46
3.9 Example of a Full MON-Collect Execution. 47
3.10 Example of a rule. 49
3.11 Example of reconfiguration dissemination and execution. 50
3.12 Example of a Full Rule Engine Execution. 51
3.13 Interactions between the Overlord manager and other components. 52
3.14 Interactions between the Node Manager and other components. 52
3.15 Interactions between the Node Manager and other components. 54
3.16 Execution steps of an Overlord sequence and their relation to the MAPE-K loop phases 55

4.1 A Visual representation of the benefits of record aggregation in this case study 61
4.2 Node count evolution over time 64
4.3 Average reliability of Overlord evolution over time. 65
4.4 Comparison of the average reliability evolution over time. 66
4.5 Node count evolution over time. 67
4.6 Average reliability evolution over time. 68

4.7 Average RMR evolution over time. 69
4.8 A visual example of the expected improvements of Overlord on the free riders experiment 70
4.9 Average reliability evolution over time 71
4.10 Comparison of Average Reliability Evolution over Time 72
4.11 Comparison of Average Latency Evolution over Time 73
4.12 Comparison of the Average RMR over Time. 74
4.13 Evolution of the number of nodes over time. 75
4.14 Evolution of average reliability over time. 76
4.15 Evolution of average reliability over time. 76
4.16 A visual example of the expected improvements of Overlord on the packet loss experiment. 78
4.17 Average reliability evolution over Time 79
4.18 Comparison of Average Reliability Evolution over Time 80
4.19 Comparison of Average Latency Evolution over Time 81
4.20 Comparison of Average Reliability Evolution over Time 81

Acronyms

CAN Content Addressable Network *(p. 14)*

DAG Directed Acyclic Graph *(p. 12)*
DHT Distributed Hash Table *(p. 7)*

ECA Event Condition Action *(pp. 21–24, 26, 34, 45, 59)*

FIFO First In First Out *(p. 47)*

INA In-Network Aggregation *(pp. 19, 21, 25, 37, 80)*
IPFS Interplanetary File System *(p. 1)*

P2P Peer-To-Peer *(pp. 5–9, 11, 13–16, 19, 21, 22, 25, 33, 48)*

QoS Quality of Service *(p. 27)*

1

Introduction

This chapter discusses the motivation and goals of this work. First, we elaborate on the motivations and the problem our work aims to solve. Then we focus on the main objectives and the contributions made. Finally, we provide a brief explanation of the remainder of the document's structure.

Motivation

In today's age, most services and applications on the internet are supported by cloud infrastructures such as Google, Amazon and Microsoft[29], which host a variety of services for developers to host their applications, and also serve users directly. This paradigm is highly centralized and sits on the premise that one trusts and relies on these services and companies, while they dictate the terms of service with little competition. **Web3** emerged to address these issues, focussing on a user-centric Internet with a more democratic and cooperative view of the Internet, moving power away from big companies by design. Web3, ultimately aims to promote decentralization, security, and privacy. Web3 is still in its early stages and has not yet become competitive, but has revived the appeal for research in decentralized systems [12]. Examples of this resurgence are the success of the blockchain in the world of cryptocurrencies[48], as well as of the Interplanetary File System (IPFS) for decentralized file sharing[28].

Was this it?

Tor works by sending traffic through 3 different servers. Don't think it fits here right?

Decentralized systems introduce an additional complexity layer, as cooperation usually relies on more complex solutions and protocols in order to achieve a true decentralized system[53]. As a consequence, many protocols[39, 36, 61, 45] have been studied and developed. The amount of protocols developed is also related to the different benefits they can offer, as due to complexity of such systems, developers may need opt for different approaches which might provide different guarantees. Moreover, protocols like these usually have local parameters which can be adjusted depending on certain conditions of the system. However, most protocols assume a static configuration, i.e. parameters of these protocols are entered at the start of the system, based on expectations of how the system will perform throughout its lifespan.

I did not know if you wanted me to cite examples of protocols with static configs or simply a paper that says that most have static configs.

Closely related to the peer-to-peer idea, swarm computing is a paradigm inspired by swarm intelligence, in which members of a system cooperate and coordinate to achieve self-organization, leading to a global behavior denominated as swarm behavior [31]. Swarm computing refers to computer systems that follow this paradigm. At its core, swarm systems rely on cooperation between system participants, therefore, communication and membership protocols are essential to achieve this cooperation.

This better? Or should I just remove this statement? I feel like swarm computing is only a motivation for TaRDIS, not for my solution. However, maybe I should just move this to the research context part

In response to growing demands and growing complexity in these kinds of systems, IBM's autonomic manifesto was proposed to grant systems the ability of self-configuration with less intervention from a user[26]. This is a very important concept that should play a part in the resurgence of decentralized systems, since it allows systems to manage themselves according to environment conditions, reducing or even eliminating the need for constant human intervention.

More Citations in this part of autonomic?

Moreover, current solutions of autonomic computing are either fully centralized[25, 23], in the sense that they do not consider distributed systems at all, or are highly decentralized[55], meaning that components of a system either manage themselves independently or have to coordinate for executing changes. Coordination of nodes usually imply resort to complex solutions, that can limit the type or architectures developed.

Ask more or less what I should say here

Problem Statement

Centralized approaches to autonomic systems, in this context, are usually not scalable solutions[25, 23, 58], as they are not conceived with decentralized systems in consideration. Decentralized approaches, involve each component managing itself or communicating with the remaining components for deciding and executing changes[55, 73]. These solutions are usually very complex and also very network intensive, since they resort to complex protocols and solutions, which share these issues. For example, reconfigurations such as changing the stack of protocols usually requires the components involved having to reach a *quiescent* state[59]. However, less complex reconfiguration actions, such as changing local protocol parameters, might be possible with different and less complex solutions. Supporting more complex reconfigurations limits the design of less complex ones, since for a system to support both, the same general architecture is used. If we consider only adaptation of protocol parameters, the need for complex solutions is no longer a concern, leading to new ways of achieving system reconfiguration. This presents us with a promising research direction, in large-scale decentralized systems, which have their protocol parameters managed by a centralized component

Didn't understand if the issue is the word centralized.

Research Context

The Babel-Swarm Java framework [22] is a framework that assists developers with creation of distributed protocols, allowing them to focus entirely on the logic of their solutions,

by abstracting error-prone and repetitive low-level tasks associated with creation of these systems.

Babel currently features a vast library with several implementations of said protocols, for example [36, 39]. Moreover, Babel also has support for reconfiguration of local protocol parameters. However, it contains no mechanism for automation of this process, making it entirely the responsibility of the developer to manually create and request reconfiguration of these protocol parameters, and also specifying how they should be reconfigured.

Moreover, this work was also developed in the context of the **TaRDIS** (Trustworthy and Resilient Decentralized Intelligence for Edge Systems) European project.

"The project TaRDIS focuses on supporting the correct and efficient development of applications for swarms and decentralized distributed systems, by combining a novel programming paradigm with a toolbox for supporting the development and executing of applications" [67].

Objectives

With this work, we explore the potential of a system in which a central component manages a large-scale decentralized system. Our solution is composed of a central component that collects information about the system's overall performance, makes a decision on whether any of the protocols require reconfiguration, and if so which reconfigurations, and disseminates this decision to all participants of the system. This entire process must be carried out without sacrificing system performance, and will also have to consider scalability of the system.

Should I just not talk about scalability?

Moreover, our solution was implemented using the Babel-Swarm Java framework[22]. As Babel has no self-adaptation mechanisms, we extended Babel with an adaptive framework implemented using this architecture, enabling developers to create solutions capable of self-reconfiguration without constant tampering by a human user.

Additionally, we evaluate the solution using an application case study. This consists of developing a distributed application, and having our centralized component manage the distributed protocol used in this application, aiming to improve its performance. We then evaluate the developed system under different environment conditions, and report how our solution adapts. Moreover, we also test the same application without the developed component, and compare their performance.

Contributions

This dissertation contributes with a new approach of development of self-adaptive decentralized systems, proposing a new architecture, where a centralized component oversees system monitoring and adaptation (with cooperation from each node on monitoring). It also contributes with an implementation of this architecture on top of the Babel-Swarm Framework, as well as complete evaluation of the solution with a case study, testing the

adaptive solution against static protocol parameters. Moreover, we also contribute with a paper that is to be submitted to the next 45th International Symposium on Reliable Distributed Systems (SRDS 2026).

Since we haven't yet finished the paper, can I mention it?

Document Structure

This document is structured as follows:

Chapter 1 In this chapter we explain the main motivation for this research, as well as the problem it aims to solve. Moreover, we describe the main objectives and contributions of this dissertation, as well as its research context.

Chapter 2 This chapter presents three main areas that are core to this thesis. First we discuss decentralized systems, which touch on subjects such as peer-to-peer systems, communication protocols and relevant distributed protocols developed, and how they could benefit from a solution such as ours. Secondly, we explore the concept of autonomic computing, while discussing the different components such as system monitoring, planning, and execution. Finally, we introduce relevant frameworks that are either a useful foundation to build upon, or have themselves made contributions to the world of autonomic computing.

Chapter 3 This chapter details the designed solution, mainly its components and how they interact to achieve the desired behavior. First we give a general overview of its architecture. Then we discuss monitoring of the system. Moreover, we detail how Overlord adapts parameters of protocols based on the monitored information of the system. Finally, we present how Overlord orchestrates such components to present us with a solution that works harmoniously to achieve its main objectives.

Chapter 4 In this chapter, we evaluate the proposed solution and its ability to improve performance of the overall system, through means of a case study. We describe the evaluation test-bed and evaluate its performance under specific environment conditions against alternatives without our solution.

Chapter 5 Finally, this chapter presents the final conclusions on the developed solution, delving into its contributions, and also clarifying its limitations, while also proposing possible future work.

Review this.

2

Related Work

In this chapter, we examine the work conducted in the fields of decentralized systems and autonomic systems, and highlight the most relevant contributions in the context of our work. In Section 2.1, we introduce the key concepts of distributed and decentralized systems. Section 2.2 analyzes the concept of autonomic computing, including the many components required for a system to manage itself. Finally, in Section 2.3, we review relevant frameworks by discussing their contributions and identifying areas of improvement in light of this thesis's goals.

2.1 Decentralized Systems

In a decentralized system, workloads and responsibilities are distributed among participants of the system. Instead of relying on central points that provide a specific service, all users have to cooperate to serve other users and be served by other users. This approach provides a balanced workload and fault tolerance. To understand the concept of decentralized systems, it is essential to understand the principles of peer-to-peer systems and its associated concepts.

2.1.1 Peer-To-Peer Systems

A Peer-To-Peer (P2P) system (also referred to as a P2P network) consists of nodes that can directly communicate and interact with each other, making them capable of sharing information and services without relying on centralized servers. A very important feature that defines these systems is the fact that a node serves as both a resource or service provider, and a requester [62]. This models human interaction–whenever we need something, we ask our friends or relatives, and if they cannot help, they might connect us with their peers, which in turn might be able to help us [71]. P2P systems have several characteristics that distinguish them from more traditional approaches to distributed systems, such as:

1. **Symmetric role:** As mentioned before, a node has the role of both server and client;

2. **Scalability:** P2P systems can scale to have thousands of nodes, harnessing the power of many computers instead of just one computer;

3. **Heterogeneity:** P2P systems do not impose any hardware capacity restrictions, i.e. a node can be very hardware limited and another can be very hardware capable.

4. **Distributed Control** In its most pure form, P2P requires load and responsibility distribution among nodes, resulting in a fully decentralized solution, where any random node could be removed, and the system would still be able to perform [62].

5. **Dynamism:** In a P2P system, nodes may leave and join the system at any time, meaning that its topology will constantly be dynamic.

P2P Taxonomy Even though a clear definition of P2P systems was given in the previous paragraph, the actual classification of a P2P system can vary. At the surface level, a P2P system can be decentralized or hybrid, where hybrid P2P systems can be closer to a traditional centralized architecture.

2.1.1.1 Pure Decentralized P2P Systems

A pure P2P system has absolutely no centralized point, i.e. all nodes must have equivalent functionality. This means that there are no centralized nodes or super-nodes (nodes that act as a centralized server to a subset of clients [75]) [53]. Without a central point of failure, upon failure of a node, another one can simply take over. However, this decentralized nature can introduce challenges that would simply not exist in a centralized or hybrid solution. It can also introduce such as unpredictability of the system.

2.1.1.2 Hybrid P2P Systems

Hybrid P2P Systems combine strengths of both centralized and pure P2P architectures. These systems feature a central server, but do not remove the feature of peer cooperation. Hybrid systems can be categorized in two main types [53]:

Centralized The system contains a central server that provides some service to others, or has more responsibility than other nodes, and peers maintain a connection to this central server. This approach sacrifices some benefits of pure P2P systems, such as being more vulnerable to attacks and having a single point of failure, but can also provide better coordination among nodes, and optimized routing.

Decentralized Instead of centralization in one node, some nodes of the system have a more important role. These are called "super-nodes" and act as a centralized server to peers. Like the centralized approach, they can offer benefits such as optimized routing and coordination, but removing the single point of failure. If a super-node fails, peers can

contact another super-node, or, in case they all fail, ordinary peers can assume the role of super-peers.

Overall, the kind of system we propose, in its normal functionality is ideally a pure P2P system, as peers should cooperate to achieve whatever the system's goal is. However, its autonomic component, i.e. the component that makes decisions, is fully centralized. This approach is further explored in Chapter 3.

2.1.1.3 Main Concerns of Protocols of P2P Systems

Although P2P systems have their advantages, providing scalability and fault-tolerance[53], this type of system also brings new concerns to designing such systems, that would otherwise not exist in different approaches. For instance, imagine a system for sharing files. Where in a centralized approach, files could be located at this central component, in a P2P approach, we have to deal with problems such as locating the file, how to transmit the file to the user that requested it, among other preoccupations.

As such, protocols designed for P2P systems have some concerns. Such concerns are how each participant is connected to other participants, and what limitations this imposes. Moreover, protocols must also deal with eventual failures in communication, and ensuring that every message reaches its intended participant. As these kinds of systems should be scalable, they also have to consider membership churn, as the more the system grows, the probability of nodes constantly entering and leaving the system increases[57, 16], and this can result in the instability of the network. Another important aspect, is concerning routing communication, as in network communication it is impossible not to deal with latency issues, finding the shortest path is a key aspect[1, 46]. Moreover, another challenge associated with these kinds of systems, is agreement between participants. In a P2P systems, systems agreeing on a current state of the system is one of the most challenging aspects, as it implies that these participants have to be strongly coordinated, which can be challenging in a large-scale system[74, 21]. Centralized approaches do not have such issues, as the system is the one responsible for maintaining the concept of state. Finally, in order to truly provide a scalable and fault-tolerant approach, protocols should also consider load-balancing, ensuring that responsibilities do not fall upon a certain participant or group of participants. As such, we now mention the main aspects that are core to P2P systems, and enable them to harness their full potential.

I don't know if this is what you wanted in terms of concerns, but I can't still change this to something else.

2.1.2 Membership

Before exploring the concept of overlay networks, an important concept is membership. This is a core aspect of a P2P network. Most systems assume global membership (i.e. every node knows all nodes in the system), but in decentralized systems, especially large-scale systems, this is not the case. In large systems, the topology is always being changed, with nodes constantly joining and leaving the system, and keeping this information updated for all nodes is not a viable option. Consequentially, protocols rely on *partial views*, which

are sets of (contact information for) subsets of nodes (typically logarithmic [40]) in the system [37]. Typically, a node only needs to know a logarithmic amount of nodes for message routing to be done with $O(\log(n))$ complexity, where n is the number of peers in the system [60]

Overlay Networks

An overlay network is a network built on top of an existing network (referred to as the underlay network). The overlay relies on it for basic network communication such as routing and forwarding. Nowadays, most overlay networks are built on top of the TCP/IP protocol [68]. The nodes of an overlay network have logical connections that can span many physical connections. This concept is the foundation for P2P systems (and for decentralized systems as a consequence). A P2P system is an overlay network that establishes a (logical) connection between its peers. Overlay networks are usually divided in two categories:

Structured Overlay Networks

Structured Overlay Networks are usually based on the concept of consistent hashing (a technique used to deterministically distribute keys across nodes based on their unique identifier), which is fundamental to Distributed Hash Tables (DHTs). DHTs are useful because they allow nodes to find data efficiently, as opposed to having to search through all nodes. For instance, if a node is looking for a specific resource, they can find its location by calculating the hash of this resource. Some notable examples of DHT protocols are Chord [65] and Kademlia [45] "a peer-to-peer distributed hash table with provable consistency and performance in a fault-prone environment". Additionally, DHTs can offer this while maintaining little information on the nodes of the system. However, when a node detects a failure by one of its peers, it cannot simply replace it for another random peer, meaning it has to resort to a structured way of finding a suitable replacement [40]. It is also important to note that, a resource can also be a specific peer, instead of data.

Unstructured Overlay Networks

Unstructured overlay networks are also known as random overlay networks, and this is due to their topology. Each node has a selection of random neighbors. It is important to note that, even though not all peers are directly connected, communication between them is still possible through gossip-based protocols (discussed further in Section 2.1.3). One aspect to consider is the number of neighbors each peer has. Every peer will have a subset of nodes as their neighbor set, which is usually called a *partial view* (as mentioned in Section 2.1.2). Defining the number of neighbors of each node is a topic that matters most to the protocol being used, and will be discussed in Section 2.1.3. Overall, unstructured overlay networks are very fitting for systems where nodes are joining and leaving at a high rate. However, this comes at a cost since, due to not having any structure, routing can be very challenging [17].

2.1.3 Gossip Protocols

After establishing a P2P network, we must then establish a way for peers to communicate.

Gossip protocols are a highly scalable and resilient way for nodes of a distributed system to communicate[14, 6, 33], taking advantage of the cooperation that P2P provides. The general idea of gossip consists of, when a node disseminates a message (to another node or a group of nodes), instead of directly transmitting the message individually to every node, send the message to its direct neighbors, which send it to their own direct neighbors. This process repeats until all the intended recipients receive the message. A practical way to reason about gossiping protocols, is to associate message dissemination as it were a contagious disease in epidemic situations [4]. Interestingly, we can also infer the number of neighbors (to which a node has to send a message) necessary for the message to spread from the theory of epidemics [18]. This value is ideally logarithmic in relation to the number of nodes in the system [39, 33]. This parameter is referred to as *fanout*. High values of *fanout* leads to more redundancy, but overall more reliability, as the probability of every node receiving the message gets closer to 1. Lower values of *fanout* reduce redundancy, however, they also reduce reliability, as not every node may receive the message. One variant of this protocol is *flood*, in which a peer sends the message to **all** of its neighbors. Another approach is using a *hop-count*, where a message carries a variable, and each time its transmitted, its *hop-count* increases. When the *hop-count* reaches a certain value, peers stop transmitting it along the network. This reduces redundancy, however, it also reduces reliability, as not every node may receive the message.

One common use for gossip based protocols is to broadcast messages, effectively sending it to every peer in the network. The most naive approach, is for that node to send the message directly to each node. However, this strategy is unbalanced, as one single node is responsible for dissemination of the message to everyone, while the other nodes simply receive it. A more reasonable and P2P approach, would be for other nodes to cooperate and help deliver the message to their neighbors as it is passed through the network (as a gossip approach suggests).

Figure 2.1 provides a general illustration of how a gossip protocol works. Each node, when it receives a message, chooses two (an example value) random nodes to send the message to. Now we discuss the three main approaches to gossip [40]:

Eager Push Gossip Consists of the most basic model. When a peer receives a message, it transmits it to a random selection of its neighbors (can be all of them). This approach is initiated by the sender. This is the variant represented in Figure 2.1, and is also the variant we used in the case study 4.1 in order to adapt, specifically adapting its fanout parameter.

Pull Gossip This approach consists of nodes informing their peers of the data they have periodically (e.g. sending the hash of the data). When a node receives a hash they do not yet have, they can request it directly from peers. This is a suitable choice in case payloads

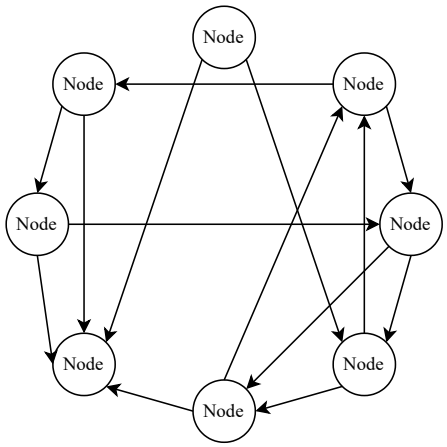


Figure 2.1: General Gossip-Based Communication Between Nodes

are large, since these are only sent to peers if they request. However, in order to receive a message, there are two interactions (*request* + *message*), which means that, if messages are small, it produces additional network overhead without any justification for it.

Lazy Pull Gossip Upon receiving a message, a peer forwards its identifier to a random selection of peers (e.g. its hash). If peers do not have the message, they can request it. This approach removes the necessity for periodically checking each peer for new messages, however, each message needs three interactions (*message ID* + *message request* + *message*), meaning that, as with the pull gossip approach, if messages are small, this is not a viable option.

Overall, the choice of which approach to use comes down to a trade-off between redundancy and latency. The eager push approach produces more redundant traffic, while the two other approaches produce more network traffic, however also producing less redundancy.

Performance Metrics of Gossip-Based Broadcast

I added this whole section. Review.

The case study we have implemented relies on adapting a gossip-based broadcast protocol. As such, in order for our component to make changes to this parameter, it must have an understanding of how the system performs. Consequentially, we reference important metrics[36, 39], which indicate how they can help monitor the performance of this type of broadcast protocol.

Reliability The reliability of a system represents the percentage of correct nodes that delivers each disseminated message. A reliability of 100% means that the broadcast

protocol functioned correctly, and every node successfully delivered all sent messages. If the reliability is bellow 100%, it means that not all nodes received every message. Ideally this value should be as high as possible, meaning that, if not all nodes are receiving every message, the broadcast strategy should be changed.

Relative Message Redundancy (RMR) This metric captures message overhead. It is defined as:

$$\left(\frac{m}{n-1}\right) - 1$$

where m represents the total number of message payloads exchanged during the broadcast procedure, while n represents the total number of nodes that received and delivered this broadcasted message. This metric can only be applied when at least two nodes receive said message. The optimal value for RMR should be zero, as it means each node received the message exactly once, and consequentially, the number of duplicate messages was zero. However, achieving a lower RMR can come at the cost of a high reliability. In contrast, high values of RMR usually indicate a poor broadcast strategy. As such, a robust broadcast strategy should aim to have a high reliability value, while presenting a lower RMR. In terms of pure gossip-based broadcast approaches, this value tends to be *fanout* -1.

Last Delivery Hop (LDH) This is a metric that measures the number of hops that the last delivered payload of a message took to reach its destination. As with the RMR metric, this value should be reduced. This metric is also associated with the latency of messages in a gossip-based broadcast protocol, as more hops indicate more nodes that must forward the message before it reaches its recipient.

2.1.4 Case Studies of Decentralized Applications

This section analyzes P2P case studies that gave significant contributions to the medium of P2P systems, as well as being important to the work developed during this dissertation. Moreover, we explore P2P case studies that could benefit of a system such as Overlord, by suggesting relevant parameters that could be a good fit to be managed.

2.1.4.1 HyParView

Added this about HyParView. Review. Also, should it be in a different location?

In order to function, gossip-based broadcast approaches require a membership, as in order for them to communicate, the nodes must be able to contact each other. However, containing the entire membership of the system is not a scalable solution. As such, membership protocols usually rely on *partial views*[33, 37], which instead of the total members, contains only a subset of these. Gossip approaches select a random amount of peers from this *partial view*, to send a message to.

HyParView[40] is a protocol that rests on this idea of a *partial view*, however, it proposes two different *partial views*, which are managed with different objectives in mind. HyParView proposes an *active view*, equal to fanout +1, and a *passive view*. The active view, is the main set of members, used by protocols to contact other peers of the system. The passive view is used as a backup. Upon a failure of a node in the active view, a node from the passive view is selected and contacted. If the contact succeeds, then this new node is moved to the active view. If the connection cannot be established, then this node is removed from the passive view, the process repeats with another node from the passive view until a new member can be established. The passive view is maintained by, periodically, performing a shuffle, which updates this view.

HyParView, successfully provides gossip-based approaches, a strategy of managing membership that is cost-effective, and highly resilient to failures. Its approach of maintaining an active view, while reducing costs of maintaining a larger view of the system members, allows for fault-tolerance, and also allows for efficiency, as maintenance of the partial view is done periodically.

2.1.4.2 MON

Added this about MON. Review. Also, should it be in a different location?

MON[41] resulted from a lack of an existing mechanism for researchers executing tests on large scale distributed systems to execute management commands, such as querying nodes for any occurring error messages, or simply effecting changes in each participant. Existing solutions at the time consisted of a fully centralized approach, where this central component would contact each node with a command. As a consequence, the management overlay network (MON) was created.

MON creates an on-demand overlay network in order to propagate management commands to every node, as well as collecting and aggregating data back. The on-demand approach, allows for avoiding issues that might arise from having to repair the overlay along the runtime of the system[36]. When a management command is required, the overlay is created, allowing the central component to disseminate its command, and receive any results back. MON, when creating the overlay, allows for creation of either a tree, or a Directed Acyclic Graph (DAG). With efficiency on the creation of the overlay logic, it provides a high probability node coverage.

Due to the centralized component in our work, MON provides a suitable algorithm for tasks that involve communication between this component and the remaining system participants. MON can provide an overlay network comprising the entire system upon demand from a central component, while also allowing for data collection back to this component, with the added benefit of enabling in-network aggregation (a concept further elaborated on Section 2.2.2.1). Its approach also ensures low costs in this task, as there is no maintenance required to repair the created overlay, and its creation is always requested exactly when needed.

2.1.4.3 Plumtree

Added this about Plumtree. Review.

Gossip-based broadcast approaches present a highly resilient and scalable solution to this task, however, they result in excessive message overhead to ensure high reliability due to their probabilistic nature[14, 6, 33]. Moreover, tree-based broadcast approaches consist of more simple solutions, with less message overhead, however, are more fragile and prone to failures[39].

Plumtree[36] provides a broadcast tree solution harnessing the benefits of both approaches. Relying on a membership protocol, it uses two sets of members it can contact. The first set *eagerPushPeers* is the one used following an eager push gossip approach, and the second one *lazyPushPeers* uses a lazy push gossip approach. Creation of the tree is done by moving peers from the eager set to the lazy one, until the first set becomes a tree with no cycles. This process is done with a reduced amount of messages. The broadcast of messages is done by using push gossip to the branches of the tree. To ensure reliability, periodically, IDs of broadcasted messages are pushed to the *lazyPushPeers* set, meaning that if the tree broadcast approach failed, the message should still be delivered with a high probability. The lazy push approach is also used to heal the tree upon failure. The Plumtree approach, also allows for optimal latency, as a result of the mechanism responsible for creation of the tree. It is also capable of optimizing and discovering new paths by adding new nodes to the tree.

Overall, Plumtree provides a suitable approach, being cost-effective, as it uses a reduced amount of messages and logic for tree creation, and since it uses a tree in order to push broadcasted messages, effectively reducing the number of duplicates. Moreover, it also provides resilience, as when a message is not received, a lazy push gossip approach is used to heal the tree.

Comparing Plumtree and MON In comparison to MON (Section 2.1.4.2), Plumtree offers a similar approach, however, it additionally provides the tree healing component. MON creates a tree for short-duration interactions, not requiring such mechanisms to patch the tree upon failures. The tree is only used upon its creation, assuming that no significant issues will occur during this period. In contrast, Plumtree aims to provide a tree for continuous use, needing the healing component. Due to this need, it also relies on more costly mechanisms, relatively to MON. This comparison relates to our work, as both approaches were considered and evaluated during elaboration of a mechanism to transport monitored data from nodes to the central component.

As mentioned by [36], Plumtree could benefit of a self-adaptive strategy to manage the membership service it depends on. Together with a membership protocol, our solution could provide this behavior and improve Plumtree's performance.

2.1.4.4 Overlay Networks For Monitoring

Added this part about using overlays for monitoring. Also, should it be in a different location?

Since our solution relies on monitoring a large-scale system, it is important to investigate how this matter was approached in different studies. [38] argues that centralized monitoring of a large-scale system is a challenging task. As such, they propose an unstructured overlay network, where neighbors are tasked with monitoring each others.

This solution argues that in order to monitor a large-scale system, each component must be monitored by at least one other component, although it is desirable that a certain threshold amount of components monitors each component. Moreover, monitoring should also be load-balanced, as to not overload a single component or group of components. Finally, there must be a fault-tolerant way to disseminate the monitored data. As such, they use HyParView (Section 2.1.4.1) to create an unstructured overlay network that organizes components for them to manage themselves.

It successfully presents a scalable solution to monitoring a system, by providing a decentralized way of monitoring it. As stated in their publication, this solution lacks a mechanism for self-configuration of components based on the monitored information. As such, our central component, provided that it could receive such information, could also make decisions, simply swapping the metric collection system used.

I tried to tie this to our work, but I don't know if it is necessary, as the link is kind of self-explanatory. Review.

2.1.4.5 Ganglia

Added this part about ganglia for monitoring. Also, should it be in a different location?

Ganglia[44] is a distributed monitoring system that was designed with to address the main challenges in monitoring large-scale distributed systems. Such challenges constitute their scalability, fault-tolerance, and low overhead of resources used.

Ganglia provides distributed scalable monitoring of distributed systems, such as large-scale clusters, by relying on a multicast protocol to monitor state within clusters, and uses a tree to then aggregate the cluster's state information. Given that Ganglia has been mainly tested and developed for clusters and high performance, it targets stable environments, which are usually not the case of P2P systems, as the system grows to be highly dynamic, with participants constantly joining and leaving.

2.1.4.6 Astrolabe

Added this part about Astrolabe for monitoring. Also, should it be in a different location?

Astrolabe[69] is another approach at monitoring large-scale distributed systems. It monitors the state of distributed resources while reporting this information to its users. It relies on an agent elected for executing a gossip-based protocol to provide and receive information about resources. This implies that this information should be limited, as gossip protocols, although they are resilient, they also generate considerable redundant

information. Information can also be aggregated as to be searched by attribute, instead of name. Astrolabe also relies on a failure detection mechanism in the event of failures.

Astrolabe is a suitable approach to monitoring a distributed system, as it provides scalability. However, the use of a gossip-based approach to share information, limits the kind of information that can be transmitted. In order for large amounts of data to be shared, its approach have to depend on using gossip to transmit identifiers of information, and participants would check if they already have it or not, in an approach similar to lazy pull gossip broadcast. Moreover, it is designed in a way that information is received upon request, providing only the information that is requested. This is a positive aspect if users wish to search for specific information, however, in systems where all of it is needed upon request, this feature becomes less useful.

2.1.4.7 Gnutella

Gnutella [7] is a file-sharing protocol used for data storage and searching in P2P distributed systems. In Gnutella, all gnodes (nodes of Gnutella) can share and receive files. Its topology does not enforce any type of hierarchy, meaning Gnutella is a real example of a pure P2P system, due to all nodes having the same functionality and responsibility. Gnutella also uses a gossip-based protocol for the communication between nodes, which means that a node that enters the network, only needs one address (known as a contact) in order to be connected to it.

Overall, Gnutella used a flooding approach, which led to a lot of network traffic and redundancy, which eventually killed its usage, however, it was studied and analyzed in order to improve its efficiency and functionality.

Moreover, some relevant parameters that could be worth exploring in terms of adaptation could be the *simultaneous connections* which represents the number of simultaneous connections each peer tries to maintain. This parameter could be adapted in the case queries are taking longer than expected in order for files to be found with less time consumed. Moreover, the *time-to-live* of a message (which works as the *hop-count* mentioned in Section 2.1.3), could also be increased if nodes are not being able to receive messages.

Check this, as I don't know if it makes sense with Gnutella's way of working. Need to read and check.

2.1.4.8 Freenet

Freenet [10] is an adaptive P2P system that allows for publication, replication and retrieval of data, while also providing anonymity. Files are mentioned without referring their location, and a file can be replicated in many locations.

Each node knows their upstream and downstream neighbors to provide privacy. A request, when its sent, contains a *hop-count* (as mentioned in Section 2.1.3) to help prevent infinite chains, and also contains an identifier for nodes to know if they are receiving a duplicate, or a never before seen file.

In order to obtain a file, a node must first calculate or obtain the key of the file. It then sends a request with that same key, an identifier and a *hops-to-live* value to itself. When a

node receives a request, it will check its own data to look for the file and if its found, it will send the file back down the stream until it reaches the requestor. If it does not find the file, it will search for the nearest key in its table and forward the request to that node. When this node eventually receives the file requested, it will pass it down the stream until it reaches the nodes that requested it in the first place. This protocol provides the fully decentralized approach, being another example of a pure P2P system due to all nodes having the same responsibility. This makes it a candidate approach for a future case study, as it is a system fitting the decentralized approach we aim for. However, the anonymity component would have to be studied, as Overlord could need some information about nodes in order to make decisions. Similarly to Gnutella, a possible target of our adaptive solution could be the *hops-to-live* parameter, which could be dynamically adapted depending on the system's topology.

This could probably break the anonymity part since you would have a central component know of your location. Check if we should leave it or not.

2.1.4.9 Content Addressable Network - CAN

Content Addressable Network (CAN) [56] is a distributed infrastructure that provides the functionality of hash-tables (where content is mapped by a hash of its key) through a decentralized system.

CAN was designed to be a more scalable alternative to Napster and Gnutella. Napster, has the index of its files in a single central server, meaning there is a single point of failure. Gnutella, on the other hand, does not centralize the index of its files, however, it floods the entire network, making it a non-scalable alternative. CAN proposes a solution to these challenges, providing a scalable alternative P2P system.

Since I didn't speak about Napster, maybe remove it

The process of file transferring between peers is inherently scalable, so the authors affirm that the issue is locating the file. CAN proposes that nodes are logically organized in a dimensional space. Nodes dynamically organize themselves into a structured overlay network that represents a virtual coordinate space. Each node is responsible for a specific region of this space.

To communicate effectively, each node maintains a record of the IP addresses of its neighboring nodes that oversee adjacent coordinate zones. This group of direct neighbors acts as a routing table, enabling the system to relay messages between any two locations in the network through the virtual coordinate system. In general, CAN manages to suggest an approach that constitutes a pure P2P solution. This is a positive aspect to be considered since, one important aspect we focus on is scalability of the system. Other approaches might introduce restrictions during evaluation. CAN has several parameters that could be a target of adaptation, some of which are *dimensions: d* which is the number of dimensions in the coordinate space, which could be adjusted by Overlord with different number of participants. Another parameter, could be *realities: r* which refers to the replication factor, which could be adapted if lookup times are too high. Finally, parameter p which is the number of peers per zone, could also be adapted upon certain conditions. Different values of these parameters can provide different effects on the system, mainly affecting path

length, latency and routing fault tolerance, making it a suitable candidate to test Overlord with.

See if these parameter adaptations make sense, or it just does not. Read the parameters and see if they already do this alone.

2.1.4.10 Pastry

Pastry [60] is an alternative that addresses the resource location problem. It provides routing at the application level and resource location in a P2P system. When a node receives a message, and a key, it re-routes the message to the node that has an identifier closest to the key. Like the other examples, Pastry is a completely decentralized and scalable solution. Assuming we have N nodes, pastry can route to numerically the closest node to a key in $\log N$ steps. Pastry has been used by many as a building block, such as PAST [15], which is discussed in the next section, and Scribe[61]. It provides efficient routing in a decentralized manner, being an effective solution to the resource location problem, and a suitable fit for Overlord to adapt.

Is the relationship with our work ok here?

Pastry has some parameters that may be relevant candidates for adaptation: L which is the set of numerically closest nodes, and M which is the set of the physically closest nodes.

Not convinced that these parameters are actually adaptable.

2.1.4.11 PAST

As mentioned previously, PAST [15] was built on top of Pastry as a storage system providing scalability, high availability, persistence and security. PAST nodes form an overlay network capable of reorganizing itself. PAST also provides high availability by replicating files at multiple nodes. To ensure balance of storage, PAST has a quota system, in which users are assigned fixed quotas, or are allowed to use as much storage as they contribute. To provide privacy, users may use encryption.

The aim of PAST is to combine the scalability and self-organization of systems such as Freenet (Section 2.1.4.8) with the reliability and persistence expected from storage systems. A possible way of research would be to implement an Overlord style architecture, that considering these different solutions, could choose between the best for certain conditions and possible problems that might arise during this system's execution.

This had nothing to do with our work, so I invented this. Maybe just remove PAST completely.

2.1.4.12 Scribe

Added this about Scribe. Review.

Scribe[61, 8] is a multicast application built on top of Pastry (Section 2.1.4.10), using its resource location mechanism for this task. Scribe allows multicast groups of nodes with several senders in each group. Each group is associated with a topic, and interested nodes may subscribe to this topic, effectively joining this group. Each group of nodes has a root, which is selected in relation to the topic, and named the *rendez-vous* point. Scribe uses a mechanism that consists of parents of nodes notifying their children in the tree that they are still transmitting. When child nodes stop receiving these notifications, they must rejoin the tree.

Scribe is a high reliability and fault-tolerant solution. However, due to its approach, it cannot assure that it will always deliver messages, as it can only offer best-effort guarantees.

This is all mixed. Protocols, applications, ...

2.1.5 Summary

In summary, decentralized systems are an effective solution that provides not only decentralization, but also scalability, availability, and fault-tolerance. These can also introduce challenges, such as coordination, longer execution times, and inconsistency issues. This is one of the reasons for exploring a centralized solution in terms of adaptation. In general, to effectively evaluate our solution, the system it is tested on should be the closest to a pure P2P system in terms of how it achieves its overall goal. The autonomic component of the system is fully centralized. However, issues concerning the complexity that centralized approaches introduce in distributed systems had to be tackled.

2.2 Autonomic Computing

After presenting the concept of decentralized systems, it is also important to detail what advancements have been made in the field of adaptation, focusing specifically on the adaptation of decentralized systems. This paradigm surged in the 2000s and became popular due to rapid advancements being made on the field of computing, which led to applications not only becoming increasingly complex, but also heterogeneous and dynamic. Distributed systems did not help on this matter, since even the most simple distributed system can succumb to the problems of network communication. In 2001, IBM suggested the concept of autonomic computing [26], as a different paradigm for systems and application design, which seeks to strengthen computer systems while also reducing human involvement. With the emergence of this new paradigm, IBM defined four core properties of an autonomic system, usually referred to as the **self-* properties** [27]:

I added more of a Segway introduction. Is it good?

Self-Configuration Consists of the capability for a system to reconfigure itself according to high-level goals, i.e. through specifying an objective and not necessarily how to achieve it.

Self-Optimization A system that is capable of optimizing the use of its resources under changes within itself or its environment.

Self-Healing A system that autonomously detects and determines issues within itself. These issues can go from hardware related problems to software errors [52]. It is necessary that, through the self-healing process, the system is not further harmed by, for example, new bugs introduced. Fault-tolerance is a crucial aspect.

Self-Protecting The ability of a system to protect itself from attacks. This means that the system should be able to provide security, privacy and integrity of data.

As mentioned before, a self-adaptive system should be able to provide these four properties. However, the main properties this dissertation focuses on are self-configuration and self-optimization.

The autonomic computing paradigm is inspired by the human nervous system, the most sophisticated example of autonomic behavior existing in nature today. It possesses the ability of detecting issues within itself, and, as an unconscious reflex, respond with corrective actions. For instance, when a human is exposed to low temperatures, blood vessels constrict to minimize heat loss, or when the eye is exposed to light, its pupil contracts. These are responses or reactions to changes to the human's environment.

The goal of the autonomic paradigm is to enable a system to analyze itself and, based on high level goals, respond to changes, ensuring its continuous and correct operation [50]. In essence, the objective is to create a system that works similarly to a human nervous system, quickly reacting to anomalies within itself or adverse external conditions, and carrying out measures to counter them, bringing back the system to its optimal state.

Adaptation Throughout this document, the term adaptation will be used quite frequently. As such, it is important to understand its meaning in this context. Adaptation refers to the action of changing a certain behavior of a system based on when a condition is met [59]. For example, suppose we have a system that needs to broadcast information from one node to all others. If communication is made through a gossip based approach, one might simply adapt to have a bigger or smaller *fanout* depending on the number of nodes in the system.

2.2.1 MAPE-K

MAPE-K, originally proposed by IBM, is a control loop (a loop that collects information on the system, acts upon it and feeds it back into the loop [63]) and a well-recognized engineering approach to achieve self-adaptive systems [3]. It consists of four computational steps: *Monitor*, *Analyze*, *Plan*, *Execute* over a *Knowledge* base, as illustrated in Figure 2.2.

An autonomic element can be any type of computing resource such as an application server, storage device, database, among others [32]. The autonomic element consists of a *managed element*, which represents a software or hardware resource that achieves autonomic behavior through integration with an *autonomic manager*. *Sensors* (also called *probes*) are responsible for collecting information about the performance of the *managed element* and its environment, that is considered relevant for making a decision. *Effectors* implement modifications on the *managed element*. The *autonomic manager*, with the data collected by the *sensors*, can *monitor* the *managed element* and *execute* reconfiguration actions through the *effectors* [27].

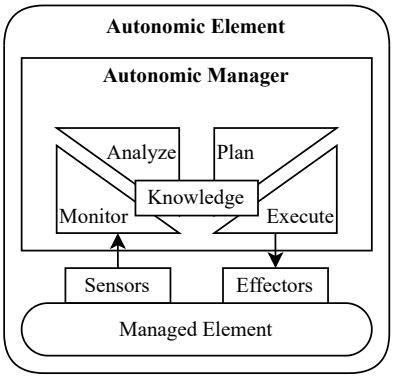


Figure 2.2: IBM's MAPE-K (Adapted from [27])

A component *K* (*Knowledge*) keeps data about system (including its environment) and data concerning its adaptation goals. This data can come from human experts and/or gathered data from the *sensors*.

The component *M* (*Monitor*) gathers particular data from the execution environment, the system itself, and other needed information, storing it in the *K* component. The component *A* (*Analyze*) is responsible for interpreting the collected data and detecting relevant changes from the expected behavior of the system. If a change is detected, it is also responsible for triggering the *P* (*Plan*) component, which uses the provided information to orchestrate a workflow of reconfiguration actions (i.e. adaptation) necessary to achieve the system's goals. Finally, the *E* (*Execute*) component carries out these actions in order to bring the system back to a normal state of behavior.

This separation into multiple components helps the designer focus on one activity at a time, and implement and modify each one of them as a separate module of a bigger system.

As stated in [50], 'Sensing'; 'Analyzing'; 'Planning'; 'Knowledge' and 'Execution' are keywords to identify an autonomic system, and although the MAPE-K model does not define the entire field of autonomic computing, the objectives of each component correspond to the main areas of concern when designing an autonomic system. Consequentially, the following sections describe each component relatively to the advancements made in its field, however, independently of the MAPE-K model.

2.2.2 Monitoring

As stated in Section 2.2.1, monitoring consists of gathering information about a system and its environment. The collected data should be limited to useful information that allows identification of key behaviors of the system through interpretation, which leads to decisions that positively impact those behaviors.

For instance, a P2P system composed of nodes broadcasting messages to other nodes through a gossip-based broadcast protocol. Relevant information can include latency, reliability, duplication rate, among others. This essentially constitutes information that can be used to assess the current status and behavior of the system.

According to [27], there are two types of monitoring in self-adaptive systems:

Passive Monitoring Consists of observation and collection of data without actually interfering with the operation of the system. For instance, operating systems provide tools for observing the CPU and memory usage, runtime information of the system, devices mounted, per-process information, among others.

Active Monitoring Involves engineering the software, adding code to its implementation specifically tailored to collect the specific required data. In our case, since our adaptive solution adapts parameters of protocols, it is important to assess the performance of said protocols. As such, an active monitoring approach is required, where protocols must be ready to monitor their own behavior.

Maybe when mentioning monitoring, specify that in our case, the babel metrics framework uses an active monitoring approach, as protocols should be ready to collect metrics of their own behavior.

2.2.2.1 In-Network Aggregation

Consider a P2P system, where nodes use a gossip-based broadcast protocol to broadcast generated messages to each others. Monitoring a system such as this one, might require monitoring information about each message disseminated. Moreover, not only there is a need to monitor information about each message, but also at each node (e.g. when a certain node delivered a message). Depending on the number of messages exchanged, network traffic can become a limiting factor. However, in this system specifically, the autonomic component does not need all reception times of a message for each node, only the node that took longer to receive the message. As a consequence, instead of simply forwarding all information, non-useful information can be discarded before it is sent to the central component. This is the basis for network aggregation. It surges due to a need for new techniques that reduce the monitoring overhead [49]. Other applications with similar problems prompted research in solutions for In-Network Aggregation (INA). It consists of aggregating data in advance along the network path, i.e. combining or summarizing data as it travels through the communicating nodes [20]. This is achieved through a distributed aggregation protocol.

One of the most common problems with aggregation functions is the continuous aggregation problem[11]. Usually, input values of the aggregation functions change over time, and this means discarding all previous values and restarting the aggregation process in favor of the new ones. In our case, metric aggregation will be performed on demand and periodically, meaning that collection of the input values will be done at the same instant, for all protocols, and it will be the task of the adaptive component to keep track of over-time information. Our solution essentially consists of taking a "snapshot" of the

system at a certain instant. This means that aggregation protocols defined are not a subject to the continuous aggregation problem.

I added a lot here explaining continuous aggregation, is it good? I feel like I'm explaining a whole random problem, which I then just say it does not apply. So why am I even mentioning it here.

There are two approaches for in-network aggregation [19]:

In-Network Aggregation with size reduction: Consists of combining and compressing information from distinct sources before sending it to the next node. For example, in the broadcast system mentioned at the beginning of the section, instead of each node simply adding their information about messages on top of information from other nodes, for example the time at which it received every message, it could aggregate it, comparing its delivery times to the ones received from other nodes, and keeping only the higher timestamp, discarding the lower values which, the central component does not need. The discarded information depends on the system and on what it intends to monitor. In this specific example, to make the decisions it can, the central component only needs the last delivery time of a message.

In-Network Aggregation without size reduction: Consists of forwarding the data along the network without any kind of reduction. In our broadcast system example, it means simply joining information about messages with the one monitored from other nodes, and forwarding along the network, without removing any unnecessary information.

Choosing which approach to use depends on the needs of the application, and above all, on the need for detail of the information monitored. For the purposes of this dissertation, specifically the case study implemented, the main objective is monitoring the overall performance of the system, meaning that the first approach is the most adequate one, as some information monitored can be aggregated before reaching the central component.

Concerning aggregation functions, these are the function which dictate how the data is aggregated. As an input, they take (collections or several entries of) data, and output the aggregated data. There are two main aspects that concern aggregation functions [19]:

Lossy and Lossless Functions that compress data in either a lossy or lossless manner. With lossy functions, the original values are not recoverable and a certain degree of detail and precision is lost. With lossless functions, the original values can fully be reconstructed after the function has been applied[19].

Duplicate Sensitive and Insensitive A function is duplicate sensitive if its output changes when the same input value is repeated multiple times. In other words, the frequency of values influences the result. A function is duplicate insensitive if its output depends only on the set of distinct input values, not on how many times each value appears. Repeating the same value does not affect the result. For example, an average is duplicate sensitive, since repeating a value changes the computed average. In contrast, a maximum function is duplicate insensitive, as the result remains the same regardless of how many times the maximum value appears[19].

Another aspect to be considered, is that these functions should not be resource intensive, as if too much time and computing resources is spent on executing these functions, we lose the efficiency that aggregations are supposed to provide.

Overall, monitoring is very dependent on the application that is being subjected to it. Consequentially, it is important to have mechanisms tailored to each specific system, that can efficiently convey the current status of the system's performance. Moreover, INA solutions are important in terms of efficiency, and saving network resources. Since nodes will be transmitting their monitored information to other nodes, it is important not to introduce too much network traffic.

2.2.3 Planning & Analyzing

Once the system has been monitored, and the autonomic component has sufficient information about its state, it must then evaluate whether it is operating as intended, or if it requires adaptation. If reconfiguration actions are indeed required, the system should determine the appropriate corrective measures necessary for it to return to the normal expected behavior. 'Analyzing' was also joined with this section due to it usually being co-joined with the planning phase.

Although there are many models (e.g. graph-based, process-based and state-based), according to [59], policy-based solutions are considered to be the best approach. This is due to their declarative nature, which results in a better understanding by a developer or an operator of the system and when it decides to adapt. Policy models can range from more simple solutions (like if-else behavior) to more complex solutions (e.g. policies defined with a high-level goal).

2.2.3.1 Policy-Based Planning

Event Condition Action (ECA) rules These are located on the simpler side of policy-based models. ECA rules consist of a set of rules that specify what actions are taken by the system when certain events occur. When an **event** takes place, the **condition** is evaluated, and if it is met, an associated **action** is executed [**Mccarthy1989**]. The designer of the system can specify what events should trigger an adaptation, and what adaptations are triggered by each event.

For example, consider a P2P application, where nodes broadcast messages to each other using a gossip-based broadcast strategy. If the average reliability of the system (i.e. percentage of nodes that deliver all messages) decreases, it might be beneficial to increase the *fanout* parameter. For this purpose, we might have an ECA rule specifying that, when the reliability reaches a number which is less than a defined threshold x, then increase *fanout* to y.

Although the idea of ECA rules was initially conceived in the context of databases, their concept fits perfectly in the context of self-adaptive systems and has widely been used in research done in its field. This is due to them providing a direct way of mapping

specific events (e.g. when the average reliability reaches a defined critical level) to actions (e.g. increase *fanout*). Assuming a system is capable of collecting information about itself and adapting, implementing ECA rules would complete the system, enabling it to map actions to events. However, a challenge associated to ECA rules, can be the extensive knowledge about the system that administrators must have. Since the whole concept is for events to be directly connected with actions, this means that, to conceive a policy, one must have a deep understanding of the system including potentially aspects related with their low-level nature. Another possible issue, might be the scalability of these rules. If a system grows to have more and more rules, it may become impossible to foresee what could occur in the event of multiple policies in place. These conflicts need to be resolved, and more research on this matter has emerged. Primarily, [43] proposes precedence relationships between policies, enabling the coexistence of inconsistent policies. Also, [30] researched conflicting authorization policies.

Goal Policies Goal policies come as a more high-level approach to ECA rules, in the sense that, instead of defining a specific action, one would define a desired state, and let the system find a way to achieve that state [27]. For instance, assume a P2P system composed of communicating nodes. We could specify that the communication latency between nodes in the system should remain under 50 ms. The autonomic manager (remember Section 2.2.1) uses internal rules (i.e., knowledge) to adjust resources or configurations as needed to reach the desired state. An approach to goal-oriented policies was presented in [5], where the general objective is to refine high-level goals into low-level operations that can subsequently be mapped onto ECA-based policies. In approaches like [66], goals are expressed in a temporal logic (a formalism used to describe how propositions or conditions evolve over time), and plans are generated to achieve a particular goal in a given environment.

Goal policies require planning from the autonomic manager, making them more resource-heavy than ECA rules defined in the previous section. These policies also suffer from the fact that each state has to be described as desirable or not desirable, and if a desirable state cannot be reached, the system will not know which will be less desirable than the other.

Utility Functions Utility functions are a way to overcome the main problem of goal policies. Instead of states being classified as desirable or undesirable, states will have a scalar value of desirability. A utility function is a function that takes an expected effect of a configuration as input, and a value as output, which represents its level of desirability. Utility functions enable on-the-fly determination of an (expected) optimal feasible state, whereas goal policies guide the system toward any state that meets both feasible and acceptable criteria [72]. Usually, utility functions are used to compare alternative configurations, where one applies it to the different expected effects of a given configuration, and based on the output, choose the best one.

The greater problem of utility functions is its definition. Every aspect that influences the decision must be quantified, although research concerning this question is being done. A contribution in the field of utility functions was presented in [72] where utility functions are employed to dynamically allocate resources within a realistic autonomic computing system.

Policy Languages Policy languages are a form of specifying policies in a system, i.e. specify policies in systems employing policy-based planning. Policy configurations are often described in XML format to ensure accuracy and compatibility across different platforms. Current methodologies provide only definition and configuration of policies prior to initialization of the system, meaning the policies defined remain static throughout the system's runtime. This is not ideal, as policies should be capable of self-adaptation to better align with the goals of the system[2].

For instance, consider a policy that adjusts the monitoring rate based on the level of dynamism in the monitored object's behavior. If the rate is too high, the associated processing and communication costs become excessive. However, if the rate is too low, the system might fail to respond to a sudden change [2].

The number of contributions to this field is extensive, however, some notable ones include Ponder [13] and PDL [42], which provide an ECA rule-based implementation, where Ponder was devised both for distributed system's and security management. Another contribution, specifically on the field of policy conflict detection and resolution (specifically ECA-based policies) is [9]. Finally, a language specification that supports dynamic self-adaptation of policies [2].

2.2.3.2 Architectural Model Based Planning

This approach consists of creating a model of the system (the *architectural model*). This model reflects the system's behavior, outlines its requirements, and its objective. The model may also represent aspects of the system's environment. This model is then used to help reason about the impact of certain adaptations. The great benefit of this approach is that, assuming the model correctly represents the system's behavior, it is possible to observe how it will react to adaptations, and if it continues to operate smoothly. This is achieved by applying adaptations to the model first, observing the resulting state from the adaptations, and if the results are positive, apply them to the system.

The use of this approach is compatible with the use of policy-based planning. The adaptations on the model can be specified with ECA rules. For instance, if the model starts operating outside the expected behavior, the correcting measures that return the model to a stable state might be specified by means of ECA rules.

A negative aspect of this approach is the fact that, there will always be a delay between an event happening in the system, and the model being updated to reflect that event. If the delay is significant, and the system changes its state frequently, a specific planning

might be carried out when the model assumes the system is under a state that in fact has already changed to another [27]. For example, a distributed system where nodes communicate with each other. Assume that for a moment, latency increases drastically, but then returns to its expected value. Even though latency has already returned to its normal value, the system might already be deciding to implement a counter-measure (e.g. increase the *timeout* value) to an outdated state of the system.

2.2.4 Execution

After completion of the planning phase, the system must carry out the decided adaptation plan. Depending on the type of adaptation decided, the system might be affected in different ways.

In the context of our solution, we are adapting protocol parameters. Essentially this removes any need for coordination during the adaptation of said parameters, meaning that nodes, when they receive reconfigurations from the autonomic component, might directly adapt and change these parameters. However, we also include information about the process of executing reconfigurations of a system (e.g. changing the protocol stack being used) as it constitutes related work, and might also be important for any future lines of development of our work.

When we have nodes communicating with each other, one cannot simply decide to adapt on its own. For example, a node that decides to adapt while it is in the middle of communication with other nodes. If nodes have to switch protocols, and their messages are incompatible, those messages might be completely lost, leading to undesired disturbances during the adaptation process.

An approach to ensure the system operates as expected, and no disturbances are introduced, is to make a node *quiescent* before adapting. A node being in a *quiescent* state, implies that it is not currently engaged in communication, either initiated by itself or any other node, i.e. before becoming *quiescent*, a node has to complete any communication it is currently engaged in [34]. Before a node becomes *quiescent*, it will usually become a *passive* node, i.e. it will not initiate any communication, but will still answer to communication from other nodes. Another aspect of this approach, is that other nodes around the target, will have to become *passive*, or they might initiate communication with this node. To achieve this kind of coordination, some form of a configuration manager is needed to identify these surrounding nodes and change them to a *passive* state.

The work described in [24] suggests two algorithms. The first algorithm suggests letting ongoing communications finish, and then block all nodes of the system before adapting. This approach is disruptive as it stops the whole system, even though some nodes might not even be affected by the adaptation taking place. The second algorithm suggests only blocking the target nodes, and any others that are in current communications with a target. All requests made to any of those nodes will have to be postponed until after adaptation has taken place, however other nodes can continue to operate unaffected.

Offering a less disruptive alternative, the work in [70] proposes a solution that only requires stopping the target node, and executing adaptation only when all other nodes assure they will not initiate any interaction with it.

2.2.4.1 Switching Protocols

As mentioned previously, distributed applications run several protocols in order to achieve their goals. Depending on the conditions of the system and its environment, one might choose different protocols for different situations, which may require switching between them. Some protocols also need to maintain a state.

Switching the protocols executing on a distributed system is a task that can become complex very rapidly, specially if the state of one protocol also has to be transferred to the new protocol. The work in [59] describes an example using two distinct **total order broadcast** algorithms. Total order broadcast, a frequently used as a case study, ensures that all nodes receive messages in the same order. Assuming two general protocols $TO1$ and $TO2$, switching between the two of these is not trivial, especially due to the system's distributed nature. If messages between the two are incompatible, and a node broadcasts a $TO2$ message, and another node is still executing $TO1$, that message might be lost. Moreover, if a node is executing $TO1$ and switches to $TO2$ without properly transferring its sate, messages can also be fully lost. To solve these issues, the general approach is to place nodes in a *quiescent* state, and use consensus protocols to make them agree on when to change, and when communication can be resumed. In this fashion, no messages are lost during the transition of the two protocols.

However, this comes at a cost. Consensus protocols are quite complex and usually consume a lot of network traffic. Consequentially, in order to avoid the restrictive behavior that strong coordination among nodes is associated with, our work focuses on adaptation of protocol parameters, instead of changing the protocols and protocol stacks being used altogether. However, the idea of adapting entire protocol stacks should not be ignored, and should be part of future work on the thematic of this dissertation.

2.2.5 Summary

Overall, the contents discussed in this section proved important during the elaboration phase of this dissertation. Since our work is situated in the context of P2P systems, INA provides an alternative approach that reduces network traffic, as well as providing more efficient communication, as non-useful information is progressively filtered out from the system during the monitoring phase. In terms of the adaptation itself, policy-based planning is our main focus, specially ECA rules, since they provide a clear relation between expected events and reactions from the system. Since we are mostly concerned with specific parameters of protocols, low-level knowledge of the protocol is a necessity. Consequentially, approaches of higher-level goal policies design, were not as focused on during this dissertation. Finally, as mentioned previously, since this focuses on parameter

adaptation of distributed protocols, complex solutions of executing adaptation were not necessary.

2.3 Frameworks

In this section, we examine relevant frameworks, for supporting distributed systems, from which we can take lessons. These frameworks being Cactus, Appia, R-Appia, Rainbow and Babel, the framework relevant to our research context. These are either a good base for implementing self-adaptation, or have themselves made contributions to research of self-adaptive systems. This chapter focuses on their primary objectives, their advantages and disadvantages, and, where applicable, how their autonomic capabilities compare to the work we developed. Additionally, we explore their limitations and explore possible improvements.

2.3.1 Cactus Framework

The main objective of Cactus [25] is to provide a system capable of surviving, i.e. tolerable to attacks and failures. The authors propose fine-grained customization which allows detailing many aspects of the system, enabling choice between trade-offs of different guarantees. They also propose dynamic adaptation allowing services to change their behavior at runtime as a reaction to intrusions or failures.

Cactus addresses functional properties and quality of service attributes (e.g. reliability, performance, security). In order to provide a high degree of customizability, different properties and functions constitute different modules which interact trough events. A service is implemented as a composite protocol and each variant is implemented as a micro-protocol. A micro-protocol is in essence, a collection of event handlers, executed when a given event that they are bound to, occurs. An example of an event can be a message arrival.

When it comes to adaptation, Cactus, due to its event-driven nature, allows for changing out old micro-protocols for new ones by simply binding and unbinding respective events. Cactus allows different approaches of adaptation, such as modifying parameter values of already in place protocols, and changing protocols in place for others. This second approach has two variants which are event-handler rebinding (discussed above), and dynamic code loading. Dynamic code loading implies that new micro-protocols are loaded into a running composite protocol.

However, this approach of dynamic code loading demands coordination among peers, that is achieved through consensus protocols, which in turn are known to be very complex and consume a lot of network traffic. Moreover, adaptation is triggered by means of events, and since there is no centralized component, if too many adaptations occur at the same time starting in different nodes, some inconsistencies might occur. Another issue is related to the fact that the adaptation logic is joined with the adaptation execution, which

may raise challenges in changing the adaptive behavior. Finally, Cactus does not offer concurrency management.

2.3.2 Appia Framework

Appia [47] is a framework that allows for building and composing network protocols, providing communication channels that use said protocol composition. Protocol compositions are referred to as protocol stacks, and can be reused to create different communication channels that comprise the same protocols. Appia is referred to as a protocol kernel, which is a framework that supports composition of micro-protocols. These micro-protocols are then organized in a stack. Appia also supports coordination between channels and a way to specify inter-channel constraints (e.g. channels that provide information about node failures).

An Appia module is called a *layer* (a micro-protocol) that provides a service. Composing these layers on top of one another gives us a protocol stack that offers a given Quality of Service (QoS). Once a stack is defined, multiple channels can be created implementing that same stack, which will have its associated QoS. A channel with a QoS has also a stack of *sessions*, which maintain state variables for each protocol. *Sessions* interact through events.

While Appia lacks built-in adaptive capabilities, it provides a solid foundation for their implementation. Its protocol composition enables fine-grained configuration, allowing a hypothetical adaptive component to dynamically switch between protocol stacks when system performance declines. This was achieved with R-Appia, which will be discussed in detail in the following section (Section 2.3.3).

2.3.3 R-Appia Framework

R-Appia [58] is built on top of Appia, presented previously, and provides dynamic reconfiguration of protocols.

Appia has some issues that prevent its support for adaptation, such as:

- Does not include a mechanism for monitoring the system.
- Only supports static protocol composition, meaning that protocols cannot be modified at runtime without stopping the system.
- Does not have a mechanism that enables coordination among multiple nodes for reconfiguration.
- Does not have a mechanism that forces channels to become *quiescent*
- There is no mechanism that allows carrying over a state from one protocol stack to another.

R-Appia, comprises add-ons (additions to the Appia kernel without modifying it) that mitigated these issues, but also comprises changes to the Appia kernel.

First, the authors address **context sensors** (that enable monitoring of system performance). To enable system monitoring, The authors created new event types, as well as a *GenericSensor* that can be added to any protocol stack. R-Appia also provides a *Channel Reconfigurator*, a technique that forces channels to become *quiescent* by circumventing an optimization of event flow that restricted this behavior. R-Appia also introduces a **buffering layer**, which hides the reconfiguration from the application. Since the application does not know that a reconfiguration is taking place, it will keep sending messages, and the **buffering layer** is responsible for storing those messages until communication is restored. Also, the protocols developed for Appia need some adjustments too.

R-Appia also presents a reconfiguration strategy to account for different types of reconfiguration. Some adaptations can be carried out without any coordination of nodes and some require coordination between them. The elements of each combination include **orchestration**, which determines whether nodes require coordination, and **local reconfiguration mode**, which specifies whether the node must first become *quiescent* or if the reconfiguration can be applied directly.

Zero Quiescence Reconfigurations that involve changing configuration parameters of protocols. This kind of reconfiguration does not require any coordination between nodes, i.e. orchestration is uncoordinated. Locally, the adaptation can be applied directly without need for the channel to become *quiescent*.

Local Quiescence When the reconfiguration action does not involve changing the structure of messages that pass through the protocol stack. Nodes do not require coordination, but the protocol composition requires *quiescence*. It also means that part of the state from the previous stack needs to be transferred to the new one.

Global Quiescence Nodes will need strong coordination between them. Protocol compositions will also need to be put in a *quiescent* state just as in local *quiescence*. This is highly intrusive, although necessary due to certain adaptations needing this intrusiveness.

The mentioned features constitute add-ons to Appia, meaning that they do not involve modifications to the Appia kernel. These next modifications enact changes to the Appia kernel:

1. **Pool of Headers:** A message in Appia contains a stack of headers, which every protocol pushes when the message passes through the respective protocol. This means that, when another node receives the message, each protocol will have to pop its corresponding header, which implies strong coordination among nodes. R-Appia, instead of a stack, proposes a pool of headers, where each protocol can check if its header exists and remove it from the message. This does not remove stacking of protocols, just headers in messages.

2. **Dynamic Update of Event Routes:** Appia only provides support for static composition of protocols due to an optimization regarding the pre-computing of routes for all events in a communication channel. The solution discussed in the last paragraph suggested putting the channel in a *quiescent* state, however, since they can now change the kernel itself, they propose a new solution, which consists of storing the pre-calculated routes, and updating them relatively to the new protocol stack.

R-Appia is a relevant approach to tackle decentralized adaptation approach, providing dynamic reconfiguration to the Appia kernel, either with add-ons or by modifying the kernel. Its main limitations, come from Appia, mainly the stacking of protocols, that is not ideal as it restricts communication, and also the fact that its execution is single-threaded. Moreover, it is not a scalable solution, as all nodes contact directly with the autonomic manager. This means that, if we have a high number of nodes in the system, all of them will be communicating their status with the monitor in the autonomic component, leading to scalability issues, possibly dragging the whole system to a halt.

2.3.4 Rainbow Framework

The Rainbow Framework [23], is a proposal on the architectural model based planning (referred in Section 2.2.3.2). Rainbow makes use of architectural models to dynamically monitor and adapt a system, while also focussing on reutilization of adaptation strategies and infrastructure across different systems.

To achieve the goal of reusability, Rainbow splits the framework in two aspects. The *adaptation infrastructure*, which concerns the reusable aspect, and the *system specific adaption knowledge* which, as its name indicates, concerns more system specific aspects. Rainbow is also divided into three infrastructures:

System Layer Infrastructure An infrastructure that represents communication with the actual system, containing probes and effectors to enact changes in the running system.

Architecture Layer Infrastructure Responsible for the adaptation decision and execution. This layer contains *gauges* that aggregate information received by the *probes*, a *model manager* that provides access to the architectural model, a *constraint evaluator* that is responsible for detecting issues and triggering adaptations if needed. Finally, the *adaptation engine* that determines how to correct the system, and then carries out the corrections.

Translation infrastructures Essentially a translator between the system layer and the architecture layer, that mediates mapping information across the abstraction gap between these two.

Rainbow monitors the system (that runs in the system layer) through the *model-manager*, while the *constraint evaluator* evaluates the model for constraint violations, triggering the

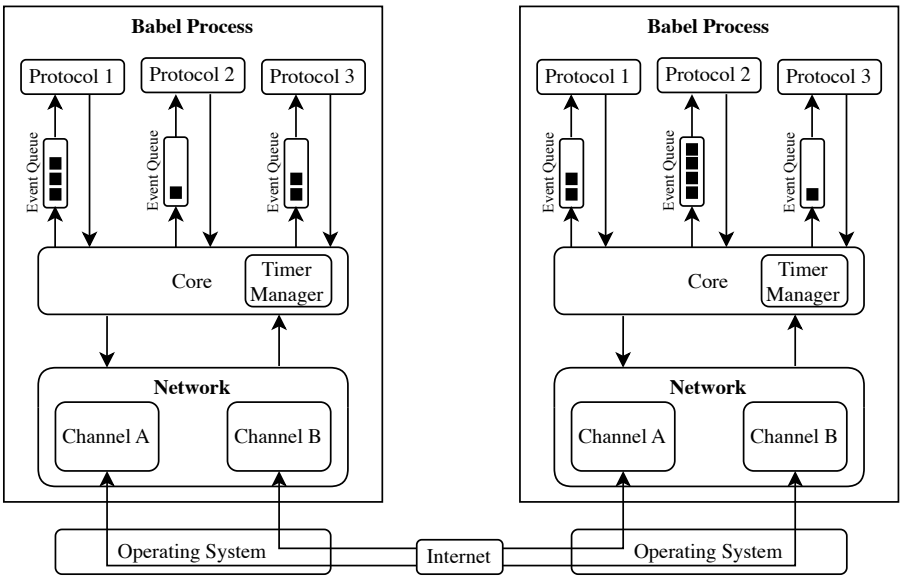


Figure 2.3: Babel Framework Architecture

adaptation engine if needed. The *adaptation engine*, is then responsible for effecting changes on the running system.

The adaptation knowledge is application specific. It consists of behavioral constraints, strategies and properties of components. Adaptation knowledge can be reused through the notion of *architectural styles*, which can be used by systems that share semantic and structural properties between themselves.

Rainbow separates the autonomic component fully from the system, thus being external to it. Since it is based on architectural model planning, it might share their benefits, but also their negatives such as a delay between the model and the system, causing planning based on an outdated state of the system. It should also be noted that the main focus of Rainbow is not on distributed systems.

2.3.5 Babel Framework

Babel [22] is a framework that proposes abstracting the error-prone and repetitive tasks of dealing with low-level aspects of designing distributed systems and protocols such as communication channels, timers and concurrency. It abstracts the low-level network and timer aspects, leaving the programmer to design protocols without needing to spend significant amounts of time with these concerns. Moreover, Babel provides a programming environment for protocols and distributed systems in an event driven paradigm.

Figure 2.3 illustrates the architecture of Babel. The framework has three main components, which are:

Protocols Protocols are programmed by developers using Babel. A protocol is modeled as a state machine that evolves through receiving events which are queued in the *event queue*. Events include triggered, notifications, or events created specifically for the protocol being developed. Execution is also multithreaded as protocols have specific threads for handling events and a thread for themselves (i.e. protocols can execute concurrently, although without having any shared state).

Core The Babel core is the principal component responsible for mediating interaction between protocols (inter and intra process). When a process wishes to communicate with another one, it is the core that delivers events to connect them. When a process wants to communicate with another process, the core delivers the message to the appropriate network channel, which in turn sends the message to the appropriate network address. The core also handles receiving the message at the other process' end. Apart from this, the core also keeps track of timers and delivering events when the corresponding timer triggers.

Network The network is abstracted by Babel using channels. Instead of dealing with the challenges of handling low level aspects of network communication and error handling, Babel provides simple primitives for protocols to use for interacting with one another: `openConnection`, `sendMessage` and `closeConnection`. Channels generate specific events related with channels such as when an *outgoing connection* is established or disconnected. Babel also provides a way for developers to create their own channels to help deal with network specific enforcement.

Babel provides an event driven paradigm that abstracts low level aspects of designing protocols, which means developers can model their protocols as state machines that evolve through means of events, such as timers, channel notifications, receiving messages from other participants, and requests. Developers only need to register these events to their respective handlers

The Babel framework contains an already implemented metric collection framework of protocols[51]. It allows developers to monitor hardware usage, generic metrics for protocols, and user defined metrics (through the use of records). This framework also provides monitors and metric exporters, as well as a mechanism for defining aggregations.

Maybe move the entire metric explanation that is in Chapter 3 to this part here?

Due to our research context of the European project TaRDIS[67], we used a specific variant of the Babel framework, named Babel-Swarm. This is a variant that supports the development of distributed protocols for swarm systems, containing mechanisms for self-discovery, self-configuration, and security features. The self-configuration aspect is under-developed, as reconfigurations must be created and enforced manually by the developer in the respective protocols and nodes.

Consequentially, in similarity with Appia, Babel developers would benefit from a self-adaptive solution, with which they could enable their protocol parameters to be managed by the system itself, with reduced input from a human. Although our solution

was developed for the Babel framework, the architecture is independent of it, as its logic could be adapted to fit other solutions that might require adaptation

I have expanded quite a bit here. See if it all makes sense. Especially this last part.

2.3.5.1 Summary

After analyzing all frameworks presented in this section, Babel was chosen as the base framework, on which Overlord will be built on top of. Apart from being developed "in-house", it provides some relevant benefits that other frameworks lack. For instance, Appia executes in a single thread across all protocols, and requires stacking them, leading to restrictions in interactions between them, whereas Babel provides a thread for each protocol and does not require stacking. Cactus does not offer concurrency management, while Babel shields the developer from issues related with concurrency by avoiding (mutable) shared state among protocols. Since the Babel framework is also being used in our research context, the developed solution is built upon its foundation, using the base functions to share messages between nodes, share information with resort to notifications and requests between different executing components, while also using its newly developed metrics framework[51] in order to allow its protocols to be monitored by each of the nodes it is run by.

3

Overlord

In this chapter we introduce our designed solution. We present its components and how they interact to achieve the desired behavior. In Section 3.1 we give an overview of the designed solution's architecture in order to provide a more generic understanding of how all components function together. Section 3.2 explains how Overlord monitors the entire system, as well as how to transport the collected information to the central component. Section 3.3 details how a developer can create and define their own rules, and how Overlord uses the monitored data to evaluate these rules in order to decide and enforce adaptations to system protocols. In Section 3.4 we present the Overlord Manager, which contains the responsible for orchestrating all components of the designed solution. In this section, we also clarify the different responsibilities of both the developer and Overlord.

3.1 Architecture

Before explaining in depth the components of Overlord, it is important to understand what these components are and how they fit in the overall solution.

Our solution consists of an architecture that enables a decentralized P2P system to have a centralized component responsible for monitoring and improving its performance. This centralized component currently consists of one single node of the system, which is denominated the **Overlord** node, not to be confused with the Overlord sequence, which represents and entire sequence of Overlord, since it requests monitoring of the data, until it disseminates the decisions it made. An Overlord run, is a concrete execution of this sequence. The crucial steps that make up the Overlord sequence are: (1) metric collection of the whole system, (2) metric data transportation to central node, (3) rule definition and evaluation, and (4) adaptation enforcement in all nodes. We will provide a brief description of each of these steps for a better understanding of the general flow of execution, as well as explaining how Overlord orchestrates said components for them to work harmoniously.

Metric Collection This step consists of collecting the monitored metrics of each protocol running on a single node, while also performing any transformations necessary to the data collected.

Data transportation to Overlord We are dealing with decentralized systems, that need the ability to scale. For this reason, simply forwarding the monitored data to the Overlord node is not a viable solution, since it could overload it. It would also be expensive to the network, as redundant or unnecessary data would not be filtered out and would simply be sent directly to the Overlord node and be discarded there. As to develop a truly scalable solution, our architecture encompasses mechanisms that deal with such challenges. Our approach consists of creating a disposable single-use tree, that is used by each node to transport their monitored information to the central component. Along this path, monitored information not useful to the central component is also discarded through the use of aggregation functions.

I removed all mentions to MON-Collect. Left mention of the tree logic, however, maybe it should also be removed and swapped with "We came up with a solution for this", and not say anything else

Rule Definition and Evaluation With the monitored data, the system then needs to decide if any reconfigurations are necessary, and if so, which. For this purpose, we offer a mechanism that uses rules for adaptation, similar to ECA rules (Section 2.2.3.1). With this mechanism, the developer, who knows the system and its protocols, can define and register rules that will be evaluated in each Overlord run. If any reconfigurations are decided, this component will send them to other nodes. When creating a rule, the developer has access to the monitored data, as well as to a history of previous monitored data, in order to make it possible to detect overtime changes in the system's behavior (e.g. a continued increase in node count).

Reconfiguration Enforcement and Dissemination After Overlord decides on a reconfiguration, it must be disseminated to all nodes in the system in order to reach their respective protocols. Every node must listen to incoming reconfigurations from the central component. Upon receiving such reconfigurations, they are forwarded to their respective protocol. Our approach in dissemination of the reconfigurations also ensures that system load is balanced, and that this responsibility does not fall under one node or a specific group of nodes.

Overlord Manager Since Overlord represents the entire solution with the added components, along this chapter, we refer to the Overlord Manager, which represents the logic necessary to link all components that allow Overlord to fulfill its purposes. Its implementation is specified as a Babel protocol, which represents an interface of the entire Overlord sequence, abstracting the developer from the other components, as well as connecting all of them to work harmoniously.

Figure 3.1 presents our proposed architecture. It illustrates both the components that the main node is composed of, and the components that every participating node requires.

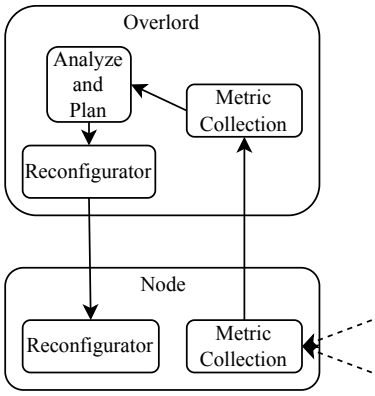


Figure 3.1: Architecture of Overlord and how each component interacts.

It also shows how these components interact, both inside of the same node, and between different nodes. Dashed arrows represent interactions with participants not represented in the figure.

3.2 Monitoring

The action of monitoring a system is one that is very specific to each protocol running in that system. Since Overlord relies on a central component, there are two important steps in the monitoring phase: (i) Collecting the metrics of protocols at each individual node (ii) Transporting that information to the Overlord node. Another aspect we address, is the aggregation of data along the network path to the Overlord node, which can improve efficiency of our solution. To address the monitoring of each individual node, as well as the aggregation of the data, we will use the already implemented Babel Metrics Framework[22, 51], as further explained in Section 3.2.1. For transportation of the data to the Overlord node, we developed an algorithm denominated as MON-Collect, explained in Section 3.2.3.

Was I supposed to cite MON, or the metrics framework, or something else?

3.2.1 Integration with Babel Swarm Metrics Framework

Overlord was implemented using the Babel framework[22], which contains an already implemented metric collection framework of protocols[51]. Metric collection is not the main subject of this thesis. As such, in addition to this mechanism is already being implemented in Babel, we integrated this framework into the Overlord sequence, allowing it to perform the monitoring of Babel protocols. This task is crucial to our solution, since it allows Overlord to evaluate the system's performance, and make the decisions about it.

The Babel metrics framework allows developers to monitor hardware usage, while also

providing data types such as counters, gauges and histograms, for measuring protocol-specific or general arbitrary metrics of implemented protocols. It also allows user defined metrics in form of records. This framework also provides a monitor that allows for collecting metrics in real-time, as well as an exporter which allows exporting the collected metrics. Both the provided monitor and exporter have available complete implementations. In terms of our implementation, the Manager is responsible for collecting monitored information from protocols, and exporting this information to other components, it serves as both the monitor and the exporter. When an Overlord execution is initiated, the Manager is used to collect the registered metrics of all relevant protocols, as well as performing the defined aggregations.

Records Instead of using the metrics already available in Babel metrics, the developer might need to create their own metrics to monitor a given protocol. For this purpose, the developer may create records, which consist of a map, allowing the developer to map keys to values. Records work with strings, meaning that they map strings to other stings. However, the developer can and should transform their values and objects into strings when creating records, and then, transform them back into their original format when accessing those records. This added complexity layer is a necessity of working with the generic metric monitoring framework.

Does this make sense? Here I'm already talking low-level, so I thought it might make sense

3.2.2 Aggregation

Another available tool is the aggregation of data. Default metrics such as counters and gauges have predefined aggregations, however, if a developer needs records, they can define aggregations that specify how those records should be aggregated. In the context of our work, this has two benefits:

- **In-Node Aggregation:** Since a protocol monitors crude metrics (e.g. number of messages, times at which a message was sent or received), it may be of interest to perform calculations with this data and transform it into more relevant and more easy to interpret metrics. Moreover, a protocol might monitor data that is relevant to other protocols, or some metrics might need data from more than one protocol in order to be calculated. For this purpose, before forwarding the monitored data, a node might aggregate all the data it monitored. This not only discards non-useful information at the node level (before it enters the network), but also can provide a much easier aggregation between nodes, since the individual monitored data is already curated.

I think this is much better explained now, however I feel like the word curated here is a bit odd.

- **In-Network Aggregation:** After the collection and calculation of meaningful metrics, instead of transmitting them to the Overlord node, the data can be aggregated in order to send only the most relevant information. For example, when calculating the latest delivery time of a message, instead of each node sending the exact time at

which it individually received the message, nodes can aggregate this information. Each node aggregates data with other nodes and keeps the most recent time. This way, the Overlord receives the final last delivery time without requiring every node's delivery time of that message.

One important aspect to mention is that, apart from the default metrics already defined, the aggregation of records consists of simply joining them on a list. For instance, considering an example of message records. Suppose that, in order to make decisions about the system, the central component needs to have the following information about a message: (i) Its ID, (ii) the timestamp corresponding to its creation, (iii) the timestamp indicating when it was last delivered, and (iv) how many times it was delivered. Simply joining two records of a same message on a list (which is the default behavior of the metrics framework), would imply keeping redundant information. First, for every record of the same message, both the ID and the creation timestamp would be repeated. Then, each node would also report the time at which it delivered the message. The central component would then be responsible to discard of redundant information, keeping only the most recent delivery time. As such, in order to harness the benefits of INA, the developer must specify how to aggregate their custom metrics. A practical example of aggregation's benefits can be found in evaluation Section 4.1.1.

I added this. Check if its good or if its just too much exposition.

Ideally, protocols should be implemented considering metric collection, and if records are used, aggregations should also be provided, essentially creating a library of Overlord ready protocols. In case protocols are not using the metrics framework, or simply do not monitor enough information about themselves, the task of enabling them with the metrics framework falls upon the developer. If the protocols use records for monitoring without providing aggregations, the developer should implement these aggregations as they see fit. They can also use the default aggregation, however, this has the cost of ignoring the INA component of Overlord.

Since I already speak of this in the responsibilities part, I don't know if I should maintain them both or just leave it there.

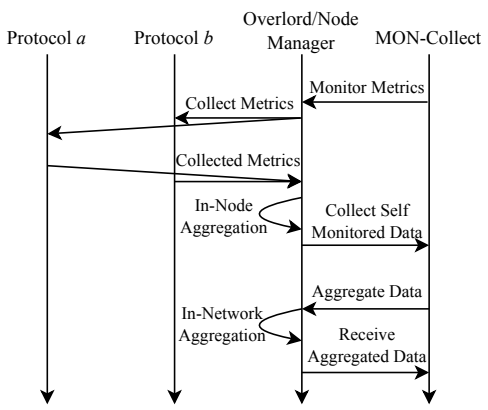


Figure 3.2: How Overlord Collects and Aggregates Metrics.

Figure 3.2 depicts how the different components interact during metric collection. The Manager acts as the monitor, as it requests protocols for their monitored information, while also acting as the exporter, since it provides this information to other components (in this case, MON-Collect). However, before providing the monitored metrics, it performs any in-node aggregations defined. Moreover, the Manager is also responsible for in-network aggregation upon request.

Finally, we have a mechanism to monitor each protocol and collect the necessary metrics on each node for the Overlord to decide on necessary reconfigurations. The only concern left is how this data will reach the Overlord node.

3.2.3 MON-Collect

This algorithm is heavily inspired on MON[41]. Given that we are working with a scalable distributed system composed of an undetermined amount of participants, a node cannot simply transmit its data directly to the Overlord node. If all nodes of the system were to continuously contact the Overlord with their telemetry data, it could become overloaded with excessive communication and data processing. This solution is not scalable.

An alternative approach would be for each node to broadcast its data, similar to a gossip-based broadcast protocol. However, this could pollute the network significantly, as protocol monitored data could be very large, and such broadcast solutions introduce a lot of duplicate information in order to reach a high probability of everyone receiving each broadcast[36]. Another concern to consider is that, since the data of each node is being aggregated, if the aggregation operations are not idempotent, the same data may be considered more than once. For non-idempotent operations (such as summation or multiplication), this behavior is undesirable. Two possible solutions are to use only duplicate insensitive aggregation functions (as mentioned in Section 2.2.2.1), or maintain a list of nodes that have already been accounted for. However, this is not desirable, as the first solution limits the kind of aggregations that can be made significantly, and the second solution undermines the original goal of reducing network traffic, as a list that could become as big as the total number of nodes would be transmitted through the network. Another alternative is to ensure that each node's data traverses each node exactly once. The proposed algorithm ensures this behavior, while also enabling the aggregation of data at each node, without any restrictions to the aggregation functions used. The algorithm developed for this purpose is referred to as MON-Collect.

MON-Collect constructs a disposable tree encompassing all participating nodes in the system, with the Overlord node as its root. This structure is then utilized to aggregate and transmit the monitored data up the tree, until it reaches the Overlord node. Maintaining this tree has costs, as nodes leaving or joining would mean repairing the tree[36, 41]. Instead of enduring such costs, we decided on creating a disposable tree for telemetry data collection, and subsequently discarding it once the process is complete. When a new execution is triggered, a new tree is then created. Since this tree is only needed at the

instant in which it is created, instead of repairing the tree, a new one is made exactly at this moment, essentially taking a "snapshot" of the system. This means that creation of the tree should be cheap and fast, as the longer the creation of the tree takes, the more changes can affect the system during this period.

I added this part comparing to repairable trees. Maybe I could even add a ref to that paragraph comparing MON and Plumtree. Review.

MON-Collect was also designed to be generic, allowing it to operate independently of Overlord, and to be used in different projects. This generality enables it to support arbitrary data types. In practice, the structure and semantics of the data, as well as the aggregation logic, are fully defined by the developer outside MON-Collect. This means MON-Collect needs assistance when working with specifics of the data being transported and aggregated. Overlord contains mechanisms for the developer to add or remove necessary intermediate steps. For example, the developer can introduce in-node aggregations (referenced in Section 3.2.2) before it is forwarded to be aggregated with data from other nodes.

MON-Collect execution consists of two phases: The **Session Phase**, which is responsible for creation of the tree. The second phase is the **Monitor-Collect Phase**, which uses this tree for monitoring telemetry data of each node and transporting it to the Overlord.

Session Phase This step is responsible for creating a tree composed by the nodes of the system with the Overlord at the root of that tree. A tree implies that there are no cycles. This aspect is positive in the sense that, since there are no cycles, there is no risk that the information of a node might be considered more than once. This algorithm assumes that there is a protocol for management of the membership, which means that, at all times, every node must have a set of members it can contact.

A MON-Collect run is the complete process since it is triggered at the Overlord node to the moment it receives the monitored data from the entire system. A run is triggered at the Overlord node, either by a periodic timer, or by command. By default, a run is triggered periodically, however, this is configurable. Each MON-Collect run contains a unique sequential ID. After triggering the execution at the Overlord node, the current ID is incremented, and a `SESSION` message is sent to each of the members of its membership. These nodes will subsequently send a `SESSION` message to each of their members, until it reaches all participant nodes of the system.

At this point, all nodes are trying to establish one single parent, and their children. When a node receives a `SESSION` message, it sets its parent to the sender of that message, after which it sends a `SESSION` to its members and waits for a response from all of them before sending a `SESSION_OK` to its parent. For each `SESSION` message a node receives after it has defined their parent, it responds with a `PRUNE` message, ensuring that each node receives a response. After a node has a response from every member (either a `SESSION_OK` or a `PRUNE`), it will send the `SESSION_OK` to the established parent. When the Overlord receives a `SESSION_OK`, then the tree is fully created, and the Monitor-Collect phase may begin.

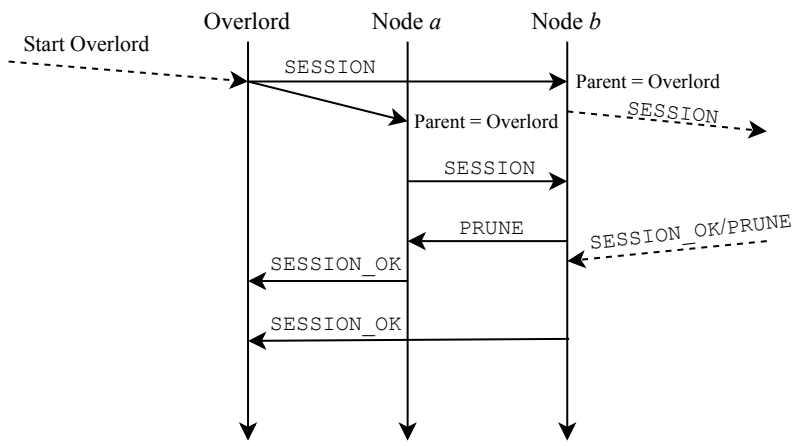


Figure 3.3: Execution of the Session Phase.

This process is illustrated in Figure 3.3. First, a run is triggered at the Overlord node. Considering that both nodes a and b compose overlord's membership, it will send a `SESSION` message to both of them. Both nodes set their parent to Overlord, establishing it as their parent. Node a has in its membership only node b, thus it sends a `SESSION` message to it. As node b's parent is already set to Overlord, it sends a `PRUNE` back to node a. Assuming node b has more members not represented in the figure, it will send a `SESSION` message to them. After their response, node b has its children, meaning it can answer Overlord with a `SESSION_OK` message. Since node a's membership is only composed of node b, as soon as node b answered (with a `PRUNE` message), node a answers with a `SESSION_OK` to Overlord.

Figure 3.4 presents a possible membership between the nodes of the system, and one possible resulting tree from the Session phase, represented by the blue edges.

MON-Collect also has the option for specifying a timeout value, which is the time that a node waits for answers. In case the node does not receive an answer from all its members, it will simply carry on with the responses it already has.

Mention the new way of nodes not timing out before their children do.

If a node happens to respond too late to its parent, its message will simply be ignored, as its parent will check that either the ID was already processed, or the ID is the same as the current one, only the Session phase has already passed. This late node, upon receiving a new `SESSION` message with a higher ID, will simply discard its current state, and join the more recent run.

This behavior is represented in Figure 3.5. Blue arrows represent messages from MON-Collect run x and red arrows messages from run $x + 1$. Dashed arrows represent messages from or to other participants not represented in the figure. Node a receives a `SESSION` message from its parent, which is forwarded to its neighbors b and c. While node b sends a `SESSION_OK`, node c takes time to send its response, and by the time it does so, node a has already responded to its parent. Consequentially, when node c answers, its

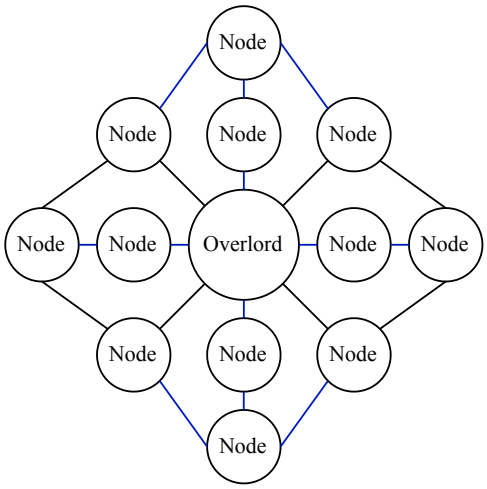


Figure 3.4: Example of a possible resulting tree from an arbitrary membership.

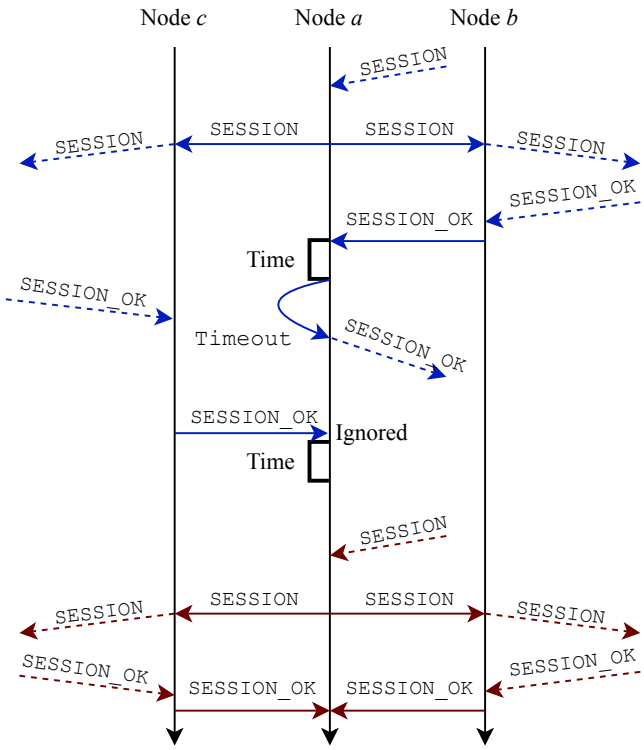


Figure 3.5: Example of a run where node *c* fails to respond to its parent within the expected time

message is ignored as the Session phase for run x has already passed. Eventually, run $x + 1$ begins, only this time both nodes answer in time, and both node b and c are a part of run $x + 1$.

I have a figure that maps MON-Collects behavior, so maps what happens when a node in state x does when it receives message y. Don't know if that would be nice to include somewhere, or how I would include it.

At the end of the Session phase, the resulting tree is considered disposable, as a new tree is created for each execution of the algorithm, as to avoid concerns of patching the same tree over time. Consequentially, creation of the tree should be as cheap and as fast as possible. Failures occurring during the execution of the algorithm is not ideal, however they can happen. In case a node fails after the creation of the tree, the failed node and its children will not be considered in that run. The specifics of what happens during a failure and how it can affect decisions made by Overlord can be found at Section 4.1.3.4.

Should I keep this part about failure?

Monitor-Collect Phase After the Session phase, the algorithm makes use of the constructed tree to transport the monitored data from each node to the Overlord.

Upon completion of the tree, the Overlord sends a `MONITOR` message to its assigned children. Nodes, upon receiving a `MONITOR` message, forward this message to their children. Simultaneously, each node collects the monitored data from all selected protocols. The Overlord node, in case it is participating in the system, should also execute this step. The node replies with a `COLLECT` message directed to itself (containing its own monitored data). Similarly, each child transmits `COLLECT` messages with their own monitored data.

In terms of our implementation in Babel, this implies communication between the MON-Collect protocol and the Manager protocol, which works as the monitor, being responsible for collecting the metrics monitored and supplying them to MON-Collect. After collection of such metrics, the Manager forwards them to MON-Collect, while also performing any in-between steps the developer decides.

Upon receiving a `COLLECT` message, the node appends the content to a collection. If a node has n children, then it expects $n + 1$ `COLLECT` messages (n from each child, and 1 from itself). Once the node receives all the expected `COLLECT` messages, it aggregates the data. In terms of implementation, aggregation is a task of the Manager, not of MON-Collect, so MON-Collect issues a request for the aggregation of data to the Manager protocol. It then interprets the data provided, and aggregates it according to the developer's specification. After aggregating all the data from the different `COLLECT` messages, the Manager subsequently triggers an event, indicating that the aggregation process is complete. MON-Collect, after receiving the aggregated data, forwards it to its parent. When the Overlord node receives all the `COLLECT` messages, the MON-Collect run has concluded.

This process is presented in Figure 3.6. The Overlord node, sends a `MONITOR` message to node a, which subsequently sends it to its (in this case) only child, node b. Node b forwards it to its children, not represented in the figure. Simultaneously, both node a and node b collect the metrics from the monitored protocols, and reply to themselves with

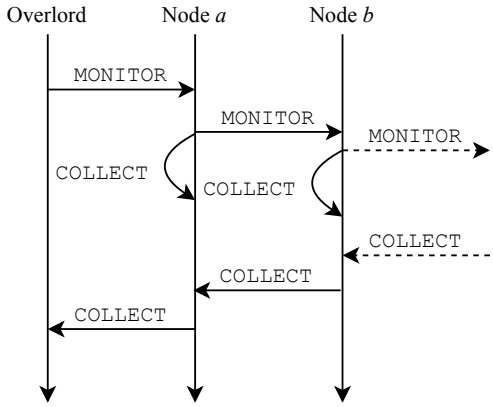


Figure 3.6: Example of the Monitor-Collect phase.

a COLLECT message. After receiving a COLLECT message from their children, both nodes aggregate the data and forward it to their respective parent. Overlord then receives a COLLECT message, meaning that the process is complete.

I added this figure and explanation. Is it good?

3.2.3.1 Handling Failures in the Monitor-Collect Phase

In case a node fails, there is also a configurable timeout value for the Monitor-Collect phase, however, since the tree is already created, the available information allows for a more sophisticated approach. If a node very close to the bottom of the tree has failed, then its parent node will wait a certain amount of time for its response. However, if all nodes wait the same amount, since messages between parent and child take time to reach their destination, nodes closer to the root will timeout before the parent of the node that failed can timeout and answer back to its parent. Consequentially, instead of simply losing information of one node (the one that failed), the information of that entire branch would be lost. Because of this, during the creation of the tree, each node reports to its parent (along with the SESSION_OK message), the height of its subtree (with the node itself being the root of this subtree), denominated h. For example, if a node has no children, then it will report $h = 0$. The parent node will then check all the h values reported by its children, and report to its parent the highest $h + 1$. There are also two more parameters (which are configurable), these being the base wait time b and the modifier value m. The final wait time t of each node is then calculated as $t = b + m \times h$. With this approach, if a node at the bottom of the tree fails, its parent will wait less time than a node at the top, and the parent node's timeout answer will reach its parent before they themselves timeout, effectively losing only the information of the failed node, and those below them.

This is very awkward here.

Since Overlord is built on top of Babel, which uses TCP connections, when a node fails or gets disconnected, the channel established between the nodes is closed, and an event is triggered. When this happens, MON-Collect, given that it is executing in the

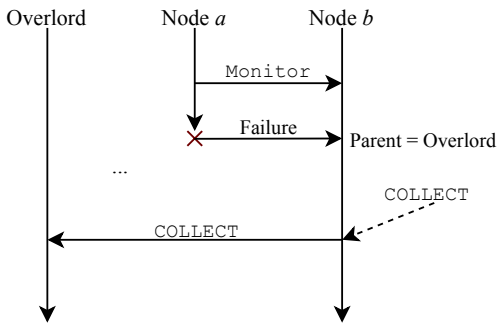


Figure 3.7: An Example of a node's behavior when its parent fails.

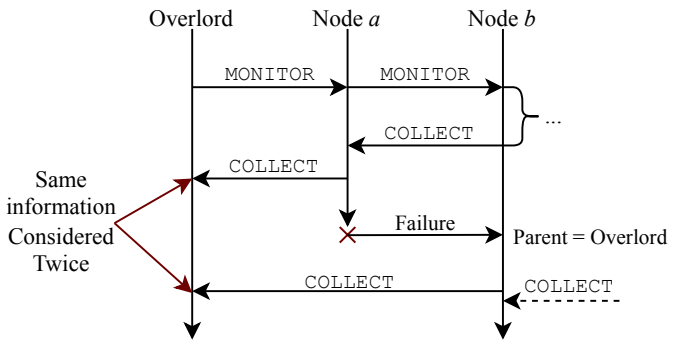


Figure 3.8: A visual representation of why a node cannot simply retransmit its data to Overlord after its parent has failed.

Monitor-Collect phase, will check if the failed node was a part of its process. If the failed node was its child, then the number of needed responses decreases in order to avoid having to wait for a failed node. If the failed node was its parent, then it overrides the parent and sets it directly to be Overlord. In case a response was not yet sent to the parent, the `COLLECT` will be sent directly to the Overlord node instead, as shown in Figure 3.7. In the event that the information was already sent to the parent, and then the parent failed, nothing can be done, since there is no way of knowing if the parent failed after it sent a `COLLECT`, or if the failure occurred before this event. Figure 3.8 illustrates why sending the data to Overlord regardless of whether it was already sent would not work. Node b sends its data to node a, which forwards it to Overlord just before it fails. Node b then receives a notification of node a's failure, prompting it to send its data to the Overlord instead. This leads to node b's data being considered twice. As such, if a parent fails after data was already sent, there is nothing to be done, and there is a risk that this node will not be considered in this run. With this approach, a parent of a failed node will not have to wait for it, and its children will have a chance that their monitored data is still considered on this collection.

Before explaining the rest of the Overlord sequence, it is important to have a clear vision of how a full MON-Collect run could be, as well as how it interacts with other protocols. Figure 3.9, illustrates a small portion of a system executing a full MON-Collect run. Dashed arrows represent communication with elements not represented in the figure (either protocols or other nodes), while blue arrows represent communication between different nodes and red arrows represent communication in the same node by different protocols.

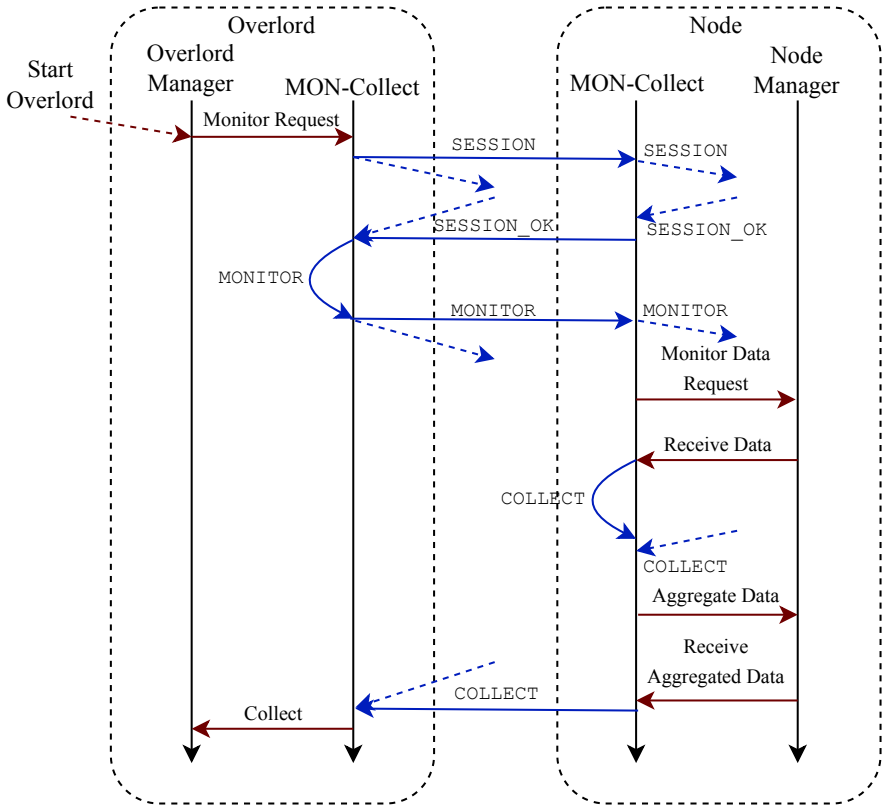


Figure 3.9: Example of a Full MON-Collect Execution.

3.3 Rule Engine and Rules

After MON-Collect finishes its execution, the monitoring phase is completed and the Overlord node has access to the monitored data from every protocol of every node in the system. After this, Overlord must analyze this data, check if the system is functioning as expected, and if not, take action to make changes to improve the system. This is the task of the Rule Engine.

This thesis' approach is inspired on ECA rules. ECA rules allow the developer to specify a condition (such as defining that a certain value should be lower than a threshold),

and an action that is triggered when this condition is met. This concept was further explained in Section 2.2.3.1. This approach allows the developer to create conditions which are specific to parameters of protocols/applications they developed, and create actions linked to these conditions.

3.3.1 Rules

Before explaining how the Rule Engine functions, it is important to first understand rules, and how they can be created. Rule definition and creation was designed with Babel in mind. Developers can create protocols based on a Generic Protocol, which enforces them to contain certain key methods and parameters. The same is true for messages, as they can be created based on Proto-Messages. In our case, developers can create a rule by extending a Proto-Rule, which enforces every rule to have key parameters and methods, such as: (1) An ID, identifying each rule, (2) a name, (3) and a description; Apart from these parameters, every rule must have: (4) The *evaluate* method, which allows the Rule Engine to prompt a registered rule to be evaluated.

Parameters must be specified by the developer. If two rules with the same ID are registered by the Rule Engine, it will throw an error and abort execution. Both the name and the description are only used for debugging purposes.

The *evaluate* method is mandatory, and its implementation falls upon the developer. Ideally, designers of Babel protocols could implement rules and package them with the protocols they create, however, rules do not simply depend on the protocols they are meant for. They also depend on the type of system where they execute, and its environment, which could affect usage of the protocol. This means that, even with packaged rules, a developer should always adjust these to the type of system it designs.

The *evaluate* method has two input parameters: (1) The monitored data (d), and (2) the history (h). The monitored data is, as its name implies, the metrics monitored by the Overlord in the current run, and the history is a list of previously monitored data. The size of the history can be configured. The history allows the developer to have a sense of the evolution of system performance (e.g. detecting a continuous increase of nodes over time). A rule also outputs a collection of reconfigurations r.

The equation you mentioned was it this it?

Evaluation of a rule is then defined as: $evaluate(d, h) = r$. Figure 3.10 illustrates a generic rule. As is depicted, it has two inputs and one output, and evaluation of this rule, providing both inputs, results in a list of reconfigurations as output.

I really don't know if this explains much. Please give input.

Reconfigurations Reconfigurations consist of key-value pairs, where the key represents the parameter being adapted, and the value represents its new value. These pairs must be associated with an identifier of the protocol to which they are meant for, in order for the component responsible for reconfigurations to correctly disseminate them to their respective protocol. In terms of their Babel implementation, these are represented by "Reconfigure" objects. These consist of a map from names of parameters to their new

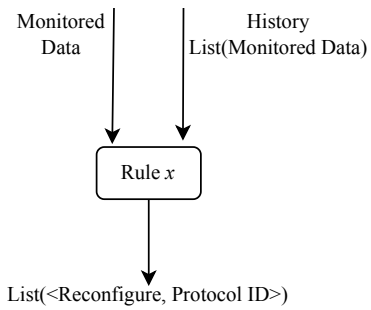


Figure 3.10: Example of a rule.

reconfigured values. If a developer needs to reconfigure, for example, a parameter p of their protocol, they must add the `@adaptive` tag to that parameter. They must then create a new Reconfigure object, and add an entry for p (e.g. `<"p", 4>`).

Each rule will output a list of pairs, composed of reconfigurations, and identifiers for the protocols they are meant for (i.e. `(<parameter, value>, protocol)`). Control of this list is fully handled by the developer inside each rule.

Rules are evaluated by the order of their registration, following a First In First Out (FIFO) order.

3.3.2 Rule Engine

After the rules designed by the developer are implemented, there must be a component that evaluates them all and disseminates their outcome to the appropriate protocols. This is the task of the Rule Engine. In order for rules to be evaluated, the developer must first register them. At the start of execution, the Rule Engine will then, one by one, register them. It is also possible to register and unregister rules upon request, meaning that, a developer can control which rules are actively being evaluated during execution of the system. Upon completion of the metrics collection process, the Overlord Manager forwards the collected information to the Rule Engine, which then evaluates each registered rule.

Reconfiguration Receiver Since only the central component runs the Rule Engine (as it is the only one that makes decisions), remaining nodes also require a component responsible for receiving reconfigurations. This is the task of the Reconfiguration Receiver. As each reconfiguration also mentions the protocol it is meant for, this component simply waits for reconfigurations, and upon receiving them, forwards them to their respective protocol.

In terms of its implementation in our framework, reconfigurations are disseminated through means of gossip-based broadcast. The Reconfiguration Receiver inspects all messages from the broadcast protocol, and checks if any contain reconfigurations. If

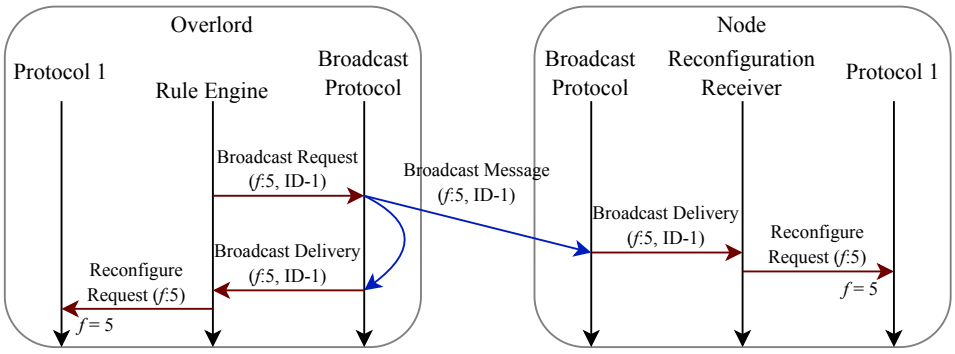


Figure 3.11: Example of reconfiguration dissemination and execution.

reconfigurations are found, the Reconfiguration Receiver will iterate through them, forwarding them to the appropriate protocol. In the case of the Overlord node itself, it does not need to run this protocol, as the Rule Engine implements this exact behavior.

Figure 3.11 depicts a generic example of this process. Blue arrows represent messages through the network, while red arrows represent interaction between protocols of a same node. The Rule Engine decides that protocol 1 should set its f parameter to 5. It then sends a request with the reconfiguration as payload to the broadcast protocol, which disseminates it to all nodes (in this case represented as a single node). The Reconfiguration Receiver, upon checking that a broadcast delivery contains reconfigurations, disseminates this reconfiguration to protocol 1. The Overlord node, receives its own message, prompting the Rule Engine to forward the reconfiguration to the appropriate protocol.

During the development of this dissertation, the idea to use the tree created by MON-Collect in order to disseminate the decided adaptations was proposed. However, this idea was quickly discarded, as a single failed node could lead to an entire branch of the tree not receiving the reconfigurations and keep operating on different parameters than the ones decided. A gossip-based broadcast approach was not used for collecting the monitored data due to it generating a lot of duplicate messages, which, since they contain the monitored data of each node, could become large messages. In this case, reconfigurations do not take as much space as monitored data, with the largest message being the number of adaptive parameters times the number of protocols. As such, it is a cost-effective solution to use a less exposed to failure approach, ensuring every node receives the reconfigurations, and guarantees every node's protocol is operating on the same parameters.

"Careful". Should I put back reliable broadcast or just leave it as it is?

Maybe take the "guarantees" off here.

Execution of Reconfigurations After all evaluations, the Rule Engine must then disseminate the reconfigurations decided to all participants of the system (including itself). In Section 2.2.4, it was referred that adaptation on a P2P system, can introduce some challenges. If two nodes decide two distinct adaptations, they could start operating on

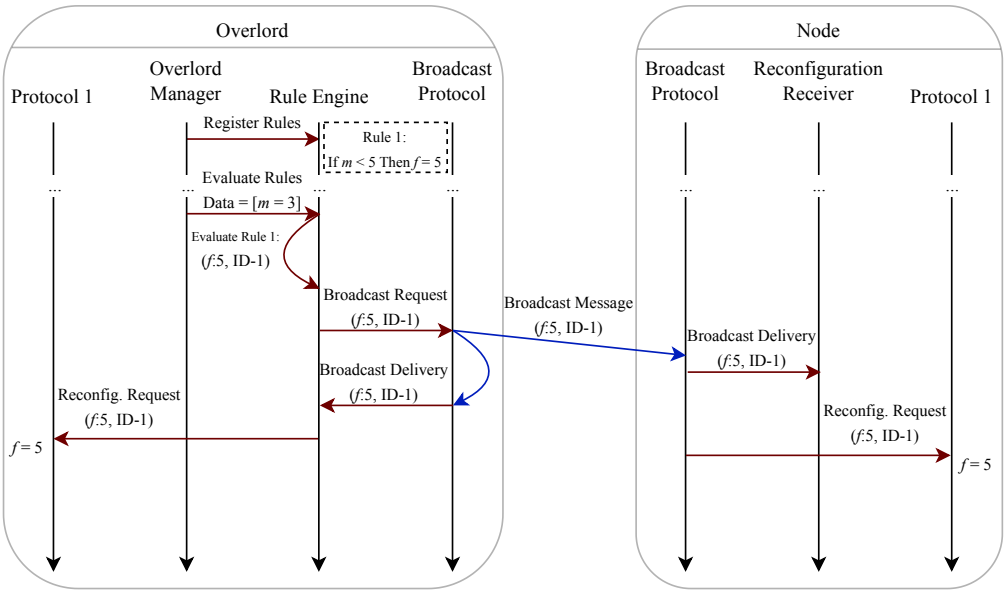


Figure 3.12: Example of a Full Rule Engine Execution.

different terms and break the entire system. Since in our solution, only the central component makes decisions, such complications do not arise. Moreover, our solution's adaptive mechanism ensures idempotent reconfigurations, as they map parameters to their new values, not modifiers (i.e. a reconfiguration that sets a parameter to the current value + 1 does not exist).

The architecture of a centralized component also allows reconfigurations that depend on previous values, by tracking and storing decisions made and which values were decided. When a modifier (e.g. an increment to the current value) is needed, the Overlord can simply retrieve the last value it decided, and apply the modifier. It then disseminates the new value, instead of the modifier.

I still have the image for clarification if this is too confusing.

Execution, as referenced in Section 2.2.4, can be a challenging aspect depending on the kind reconfiguration decided. Since our solution only deals with adaptation of protocol parameters, reconfiguration can be done without the need for any kind of cooperation between nodes, meaning they can directly exchange the old values for the new ones.

With this, we have a way for the developer to write rules that are evaluated periodically and adapt parameters of certain protocols, and also provided a way for these reconfigurations to be disseminated across the entire system. Figure 3.12 depicts an example execution of the Rule Engine and the Reconfiguration Receiver components, and how they interact with the other components.

Should I explain this figure?

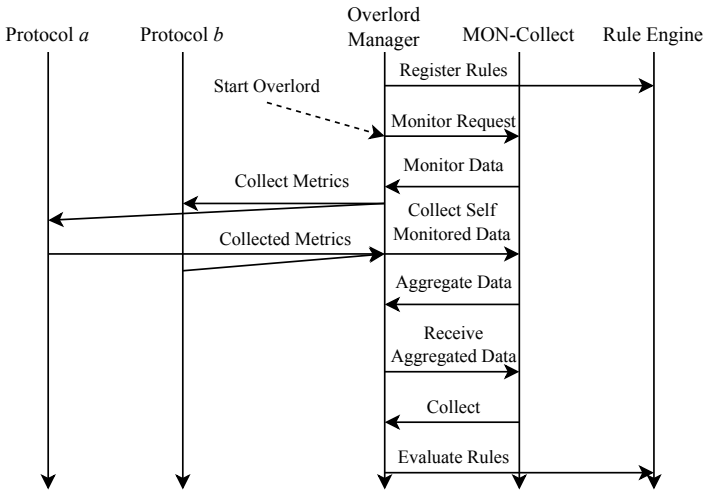


Figure 3.13: Interactions between the Overlord manager and other components.

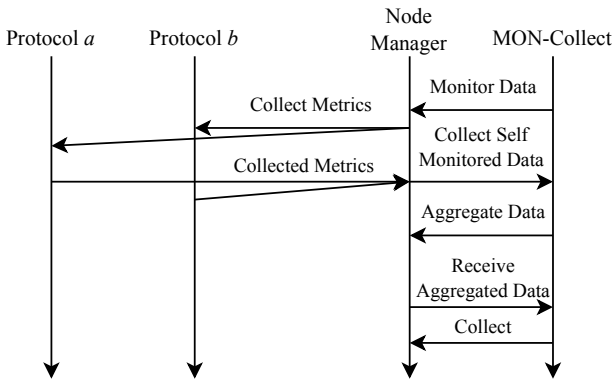


Figure 3.14: Interactions between the Node Manager and other components.

3.4 Overlord Manager

This represents our main component, which orchestrates the already explained components for them to work as one, achieving a single common goal.

Apart from interacting with each component, it also serves as an abstraction of these components to the developer. Instead of having to configure and interact with MON-Collect parameters, or with the Rule Engine separately, the developer needs only to interact with the Overlord Manager, becoming fully abstracted from the components that compose Overlord.

In terms of our implementation of this architecture, it is represented as a Babel protocol. While the central node runs the Overlord Manager, the remaining nodes run the Node Manager.

Figure 3.13 depicts how the Overlord Manager interacts with each component, while

Figure 3.14 depicts the same, only for a generic system node. Protocols a and b represent generic protocols that are chosen to be monitored by the developer. Since only the Overlord node has a Rule Engine, most interactions of the Node Manager are specifically with MON-Collect.

Some responsibilities of the Manager are:

- **Providing MON-Collect the Monitored Data:** The Manager is responsible for collecting the metrics from the selected protocols, and providing them to MON-Collect. In the case of Figure 3.13 and Figure 3.14, these are protocols a and b. The Manager, for each of the monitored protocols, retrieves the monitored metrics since the last collection, supplying them to MON-Collect.

- **Aggregating Data from Other Nodes:** As MON-Collect works with generic data, one of the tasks of Overlord is to handle specifics of data types. The Manager is responsible for aggregating the data upon request, by deserializing the raw bytes into metrics, performing the predefined aggregations by the developer, and then encoding the result into raw bytes and forwarding them to MON-Collect, so they can be transmitted to the next node.

  Figure 3.15 presents an example of this interaction. Protocol a is the one being monitored, specifically, its parameter n, which is always 1 for every single node. The defined aggregation is to sum the different n values. Note that this means the Overlord node, upon receiving the final n value, should observe the total number of nodes of the system, since n is 1 for each node. Dashed arrows represent interactions with elements not depicted in the figure. First, MON-Collect requests the monitored data from the Manager, which collects them from the monitored protocol a. Protocol a answers with its own monitored metrics, which in this case is the n value set to 1. The Manager then encodes this data into raw bytes for it to be sent to MON-Collect. After receiving all `COLLECT` messages from all its children nodes (in this case, only the data from itself, and another node), it will issue a request for aggregation of the data. MON-Collect sends a list containing all bytes to the Manager. These are then deserialized into metrics, as the first is its own data $n = 1$, and the one received from its child $n = 5$ (a result of aggregation with its own children's metrics). The Node Manager then aggregates this data, summing them ($n = 1 + 5 = 6$). It will then encode this data into raw bytes, and forward them to MON-Collect, so they can be sent to its parent.

- **Registering and Evaluation of Rules** After completion of MON-Collect, the Overlord Manager receives the monitored data of the system, and is responsible for triggering the Rule Engine to start evaluation of the rules. In terms of its implementation, the Manager is also where the developer should register rules, as to abstract the developer of having to deal with separate components.

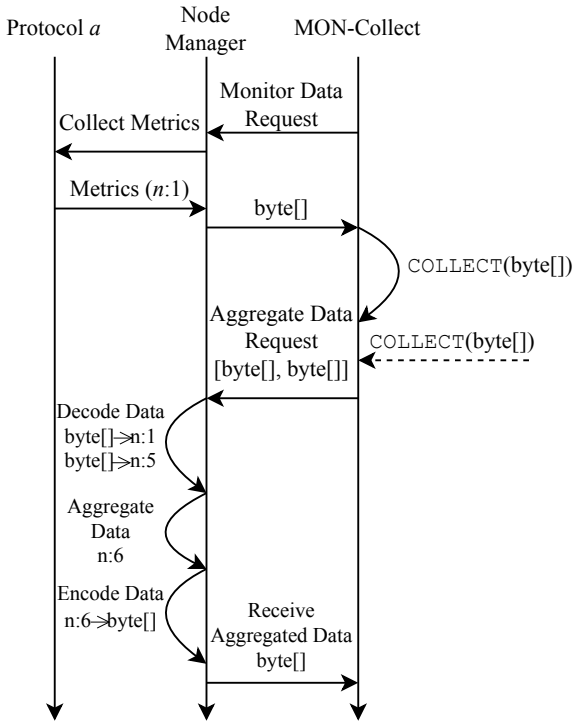


Figure 3.15: Interactions between the Node Manager and other components.

Apart from the mentioned main duties, there are other tasks enabled by the Manager, such as customizability of the Overlord sequence. In case the developer needs to add elements between each step, several tools exist to facilitate it. One example part of our case study, was to use aggregations during the monitor data phase, in which the monitored data is collected, however, before forwarding this data to MON-Collect for it to be aggregated with data from other nodes, an in-node aggregation is performed in order to transform bare metrics into refined metrics, which are to be aggregated with refined data from other nodes and analyzed by Overlord. The specifics of this thematic are further explored in Section 4.1.1.

Essentially its overriding methods, but I don't know if I should just put it here as I'd talk too much Java.

More on Knowledge Another task related to the Manager, is storing or sharing the monitored data. The Manager can share the monitored data of the system to help other external tools or protocols that might need it (for example external analysis tools such as Prometheus[54]). Consequentially, the Overlord Manager provides a configuration that stores monitored data in a `JSON` file, while also disseminating a notification containing the monitored data. Any interested protocol may subscribe to it and obtain the monitored data.

Too much Babel?

Finally, it is also responsible for numerous configurations of the Overlord sequence.

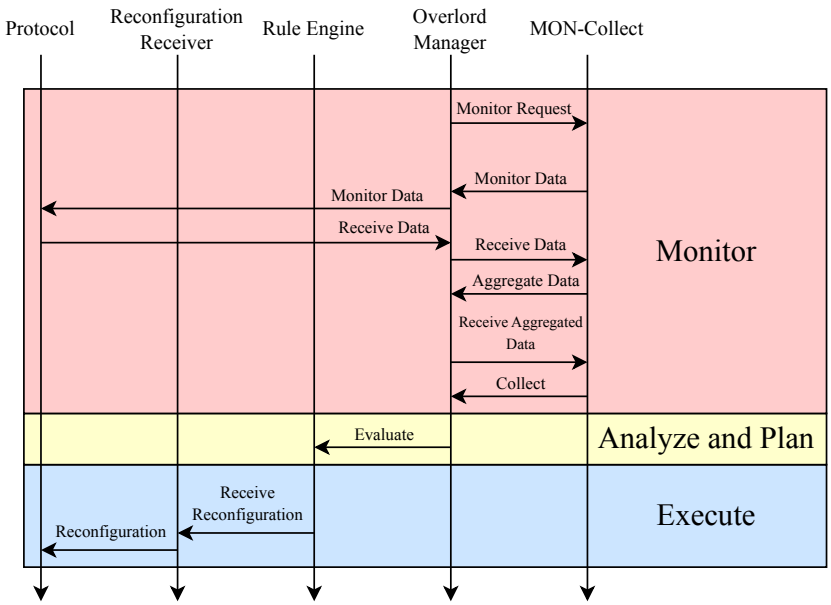


Figure 3.16: Execution steps of an Overlord sequence and their relation to the MAPE-K loop phases

For instance, it is possible to configure the time between each run and to enable or disable this periodic timer. It also allows specification of whether Overlord is a participating node of the system, or only an observer.

Before moving to the next thematic, it is important to establish a now more informed view on how Overlord works. In Figure 3.16, we align our solution with the MAPE-K loop, as referenced in Section 2.2.1, since it is a foundation for autonomic systems even today. It illustrates the flow of Overlord execution. The knowledge component is not represented as it does not correspond to a phase, but a source of information that the loop accesses when needed. However, the knowledge component is mainly contained in the Rule Engine.

Is this good?

Check if this makes sense.

3.4.1 Responsibilities of Overlord

Our solution allows a generic system to monitor itself and react to behavior changes that arise during execution, by means of enforcing rules. Because of this, some components cannot be generalized for all systems. These components' specification and implementation constitute a task of the developer.

Consequentially, it is crucial to clarify what are the responsibilities of the developer, and what is handled by Overlord. It is also important to clarify that, in terms of its implementation, Overlord has default behaviors for running without any tampering needed from the developer, although this provides very coarse grained monitoring and control of the system's behavior, and in this case, the developer must still be the one to

implement and register rules.

Monitoring Monitoring a system is very dependent on system semantics. Depending on which protocols are running, how many nodes are running each protocol, and what parameters might need to be adjusted, a developer could decide to monitor different aspects of the system. Consequentially, monitoring protocol metrics is a responsibility of the developer. Overlord has mechanisms for defining what metrics are to be collected of which protocols, and mechanisms for registering aggregations, but their definition and implementation must be carried out by the developer. Ideally, the protocols should already include metric collection and aggregations defined for those metrics, essentially creating a library of Overlord ready protocols. Overlord's default behavior is to collect all metrics, and to perform all default aggregations, thus, assuming protocols have aggregations already defined, the developer needs only to register the aggregations for metrics that use records. Consequentially, if protocols do not have these mechanisms, the responsibility of integrating them into the protocols falls upon the developer.

This could be shorter. Should I shorten it?

Data Collection After defining what metrics are to be collected and how they are aggregated, this data is collected individually on each node and requires MON-Collect in order to reach the Overlord node. This process is entirely handled by Overlord. There are a few parameters configurable by the developer, such as the time between collection of metrics, but most of this process is carried out by Overlord.

Rule Definition, Evaluation and Enforcement Creation and implementation of rules, constitutes a responsibility of the developer, as they are the ones that best know the protocols they implement, and should know what parameters could be adjusted to improve overall performance of the system. However, the process of evaluating these rules, is entirely handled by Overlord. The developer can, on request, register and unregister rules, but the rest of the process is handled automatically by Overlord.

3.5 Summary

In this chapter, we presented the solution developed during this dissertation. First, we gave an overview of the full architecture of Overlord, helping understand how all the components work together. Then, in detail, we explained MON-Collect, which provides our main component metrics from the entire system on-demand and efficiently, saving in network traffic. We then detailed the Rule Engine, which allows the developer to define rules, which it periodically evaluates, improving the performance of their protocols based on the collected information. We also present how Overlord orchestrates such components in order to fulfill its objective, as well as how it abstracts the developer of the complexities of said components. Finally, we clarify the responsibilities of both the

developer and Overlord, in order to provide a better understanding of what a developer using this architecture, will have to concern themselves with. In the following chapter, we present the evaluation for our solution.

4

Evaluation

In this chapter, we evaluate the developed solution, while displaying how it performs with a case study. Firstly, we begin by explaining our case study, how it was implemented, providing the "point of view" of a developer using the developed Overlord framework. In Section 4.1.1, we detail the role that average reliability plays in providing a basis for assessing the system's overall performance. Section 4.1.2 describes the process of adapting the fanout of the broadcast protocol according to the monitored reliability through the use of developer designed rules. Finally, In Section 4.1.3, we evaluate the developed solution under different scenarios that might occur during the execution of a system like this, testing the adaptive application against static fanout alternatives.

4.1 Message Sharing Application

The case study used for evaluation of our solution is a message sharing application. Each participant of the system generates messages periodically, and the main objective of the application is for these messages to reach all other participants. For this purpose, the application uses a gossip-based broadcast protocol in order to disseminate messages through the system. As discussed in Section 2.1.3, in a gossip-based broadcast protocol, each node, when it receives a message, forwards it to x of its neighbors, and depending on the value of x, the message will be received by every participant of the system with a high probability. This value is denominated as *fanout*, and as also discussed in Section 2.1.2, when the fanout is equal to $\log n$, where n is the number of participants in the system, the message has a high probability of reaching every node (assuming there are no kinds of disturbances occurring)[39, 33].

Since we are dealing with a pure decentralized system, it should be dynamic, meaning that the number of participants should be able to change. To ensure that all participants in the system receive all the messages, the fanout parameter has to be adjusted in relation to the number of nodes. Moreover, if some messages are lost, or some of the nodes do not forward a message, increasing the fanout should also increase the probability of all nodes receiving every message. Finally, if the number of participants reduces drastically

and the fanout is too high for this new number of participants, the number of duplicate messages will also increase. Managing the fanout parameter will be the main task of Overlord in order to ensure that, even with disturbances in the system, a high percentage of the participants will receive all the messages.

The Role of Reliability As introduced in 2.1.3, the reliability of a system is an important aspect concerning the performance of our system. Consequentially, it directly determines the fanout value decided by Overlord. This means that monitoring the reliability of the system is a crucial component, as it determines if the main objective is being fulfilled. In the context of our experiments, it was decided that a reliability above 98% is acceptable, although this value can change and is a configurable parameter, that will be referred to as l in the remainder of this section, in order to keep the specifications of this use case generic and independent of any concrete values.

Check if this is still true for all experiments.

4.1.1 Monitoring Reliability

As discussed before, Overlord tasks each node with monitoring their protocols locally, and periodically, and makes use of MON-Collect to transport and aggregate this monitored data to the Overlord central node. In the context of this case study, this means that each node must monitor the broadcast protocol. This protocol implements an adaptive version of an existing eager push gossip broadcast protocol[39]. This implementation is adaptive, which means that, the fanout parameter can be changed upon request, which is sent by Overlord when decisions are made by the Rule Engine. Moreover, this implementation also had to be expanded in order to include metric monitoring. For this purpose, we used the metrics framework available in Babel[51].

This version of the broadcast protocol enables each node to monitor its individual performance, which can be aggregated and processed with others' monitored information at the Overlord node. The broadcast protocol monitors every message, creating a record for each of these messages. For each recorded message, the broadcast protocol stores different information:

- **Sent Messages:** A sent message record contains the ID of the message (its unique identifier), the time in milliseconds at which it was created, and its creator's identifier.
- **Received and Delivered Messages:** These records contain the message ID, the identifier of its creator, the timestamp of its creation, the time at which it was delivered, and its hop count (as explained in Section 2.1.3).

In-Node Aggregation When MON-Collect triggers a collection, each node will first request the monitored metric information from all the appropriate protocols. In the context of this case study, the only protocol is the broadcast protocol, which will then provide all the records it created about the messages sent and received since the last metric

collection. Using the Babel metrics framework, it is possible to perform what we refer to as an in-node aggregation. Essentially it serves as a way for processing the raw monitored information from the broadcast protocol into usable and meaningful information. The in-node aggregation is responsible for creating **message records**, which are used to facilitate in-network aggregation of message information between nodes.

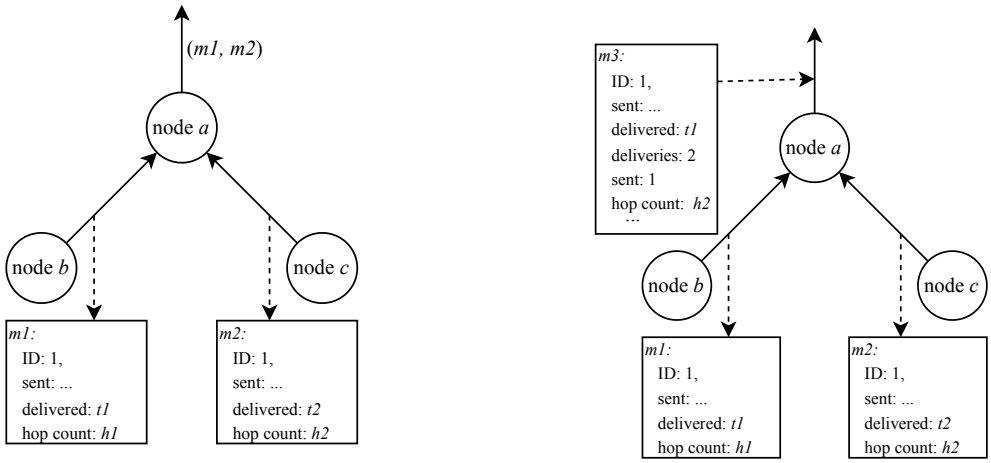
Message Records In-node aggregation processes different records of the same a message into a single record. This record contains all the necessary information about a message that Overlord needs in order to make decisions. A message record contains: (i) the message's unique identifier, (ii) its creation timestamp, (iii) the highest recorded delivery timestamp, (iv) the number of times the message was delivered, (v) the number of times it was received, (vi) the number of duplicates and (vii) its LDH (last delivery hop).

Raw records created from the broadcast protocol are transformed into message records. When MON-Collect collects the monitored information from all other nodes, it joins the message records into the same single record. This way, only the important information is stored and transmitted to the next nodes, as unnecessary information is discarded.

Figure 4.1 illustrates this phenomenon. Figure 4.1a presents an alternative where no aggregations are made. As both node b and c receive the message with ID 1, both create a message record. While node b delivered the message at $t1$, node c delivered the message at time $t2$, where $t1 > t2$. Moreover, in the case of node b, the message hop count is $h1$, while for node c, the message hop count is $h2$, where $h2 > h1$. Both nodes send this record to node a, their parent. However, without any aggregations, node a simply forwards this data to its parent. This is not desirable since, in this case, Overlord only needs the message's highest delivery timestamp, and its LDH. Figure 4.1b presents the alternative, where instead of simply forwarding the two records, it aggregates them into a new $m3$ message record, discarding both $t2$ and $h1$, keeping only the highest values for that message.

With this information about each message, the Overlord is able to calculate the necessary metrics to make any decisions about adapting the fanout, as well as some other relevant metrics that will help a developer understand how the system is performing (for instance, the reliability might be high, however message latency can still be high, and it can be important for a developer to know this). When a collection is taking place, and a node has received its own monitored data, as well as other nodes', this node will aggregate each message record, keeping only the LDH, the highest timestamp of delivery, and summing all the delivery, received and duplicate counts. As the nature of MON-Collect is to create a tree for collection, it ensures that no loops are created without any extra steps, meaning that the information of one single node is considered exactly once, making it possible for these summations and aggregations to be done without any risks.

I don't think I'm explaining myself very well here.



(a) Node a forwards both $m1$ and $m2$ to its parent, keeping both hop counts and delivered timestamps.

(b) Instead of forwarding both records to its parent, node a creates $m3$ with only the necessary information.

Figure 4.1: A Visual representation of the benefits of record aggregation in this case study

4.1.2 Fanout Adaptation Process

With the metric collection of every node of the system at the Overlord node, it must detect if the system is showing good performance, and if not, how to counter these issues by means of adapting the fanout parameter. For this purpose we use the Rule Engine, the behavior of which can be found more detailed in Section 3.3.

Since Overlord makes decisions through ECA rules, one must be created to deal with values of reliability below l (the threshold of acceptable reliability). The first iteration of this rule can be found at Pseudocode 1.

Pseudocode 1 Handle Reliability Rule - First Iteration

```
if reliability < l then
    fanout ← fanout + 1
end if
```

Although the reliability directly affects the decided fanout, it is not the only consideration to be made. Having a fanout unnecessarily high introduces a lot of duplicate messages, which is undesirable if they are not necessary. If increasing the fanout does not improve the reliability, then we shouldn't increase it infinitely, as it risks polluting the network. Because of this, we introduced a cap value to the fanout, which is defined as $\log n + c$, where n is the number of nodes in the system, and c is a configurable parameter. Like this, the fanout can increase to help with any system disturbance, however it cannot grow

infinitely. This cap can hinder possible improvements from Overlord, since it limits how much it can increase the fanout. There might be a situation where increasing the fanout more than c might produce a better reliability. However, this comes down to a trade-off, as it is far more dangerous to let Overlord increase this value infinitely. The developer must adjust this c in a way that it allows Overlord to improve system performance, but not at the cost of a scenario where redundancy becomes too high.

I added this part. Review.

Another aspect to consider, is updating the fanout as the number of nodes changes. This is not a straightforward matter. For example, if the Overlord receives monitored information from 200 nodes, and in the next collection it only receives information from 30 nodes, it is possible that the number of participants n decreased. However, it is also possible that this is the result of a failure. Moreover, even if the number of participants decreases, we shouldn't directly reduce the fanout to $\log n$, as it could be a drastic change to the system, and possibly make it function in a way that is undesired. Instead, we should opt for a more careful transition.

Moreover, decreasing the fanout when the reliability is only acceptable, might not be desirable, as this may produce an even lower reliability, possibly under the threshold l. Consequentially, the fanout will only decrease if the reliability is at 100%. As such, when the node count of the system decreases nothing happens. Only when d consecutive collections are made (where d is a configurable parameter), where $\log n$ remains unchanged and the reliability is always at 100%. Decreasing the fanout happens one by one until the fanout is equal to $\log n$, or until the reliability goes bellow the l defined threshold.

Maybe change the letters out for actual variable names

Combining all aspects explained, and considering that both c, d and l are configurable parameters, the pseudocode that defines the reliability rule is presented in Pseudocode 2.

Pseudocode 2 Handle Reliability Rule

```
if reliability < l then
    counter ← 0
    if fanout + 1 < log n + c then
        fanout ← fanout + 1;
    end if
else if log last_n < log n then
    counter ← 0
    fanout ← fanout + 1;
else if log last_n > log n and reliability = 1 then
    if counter = d then
        counter ← 0;
        fanout ← fanout − 1;
    else
        counter ← counter + 1;
    end if
end if
```

Number of Nodes	Expected Fanout
20	3
54	4
148	5
403	6

Table 4.1: Number of nodes in the system vs the expected fanout the broadcast should be set to.

4.1.3 Evaluation Scenarios

With a rule to adapt the fanout parameter in place, we require scenarios to test how it performs and determine its benefits in relation to a solution without Overlord. For this purpose, we designed a few scenarios that allow us to evaluate Overlord's performance against a solution without Overlord. These scenarios are: (i) Node influx, in which we test how Overlord adapts the fanout upon new nodes entering the system, (ii) Crashed nodes or leaving nodes, which is a scenario that evaluates how Overlord acts when the amount of participants of the system reduces, (iii) Free riders scenario, which tests if increasing the fanout parameter to a value higher than $\log n$, improves reliability in case some nodes choose to only receive messages and not forward them to neighbors, and (iv) Packet loss, which allows us to test this same aspect, only instead of some nodes not forwarding messages, a certain percentage of messages are lost and not delivered.

Overlord Failure Another aspect we considered while testing and evaluating Overlord, was the possible failure of a node during the creation of the tree. Since Overlord collects metrics through a tree, if one node fails and does not respond, then all its collected data will not be received by Overlord. Moreover, any children of this node will not have its collected data received by Overlord. The closer to Overlord the node that does not respond is, the more nodes will be affected. To evaluate this aspect, we will also perform an evaluation with different percentages of failed nodes and determine Overlord's behavior under the free riders scenario.

4.1.3.1 Node Influx

In this scenario, the system will experience an influx of new participants. The objective is to observe how Overlord adapts the fanout parameter in relation to the number of participants joining the system, and how this compares to a system with a static fanout.

The same experiment will be conducted for two versions of the application. One with Overlord, and another without it. Groups of new participants will be joining periodically. The number of new participants in each period corresponds to the number needed for Overlord to increase the fanout parameter. Table 4.1, presents the relation between the number of nodes in the system, and their expected optimal fanout[36].

Change this to 3 when ready.

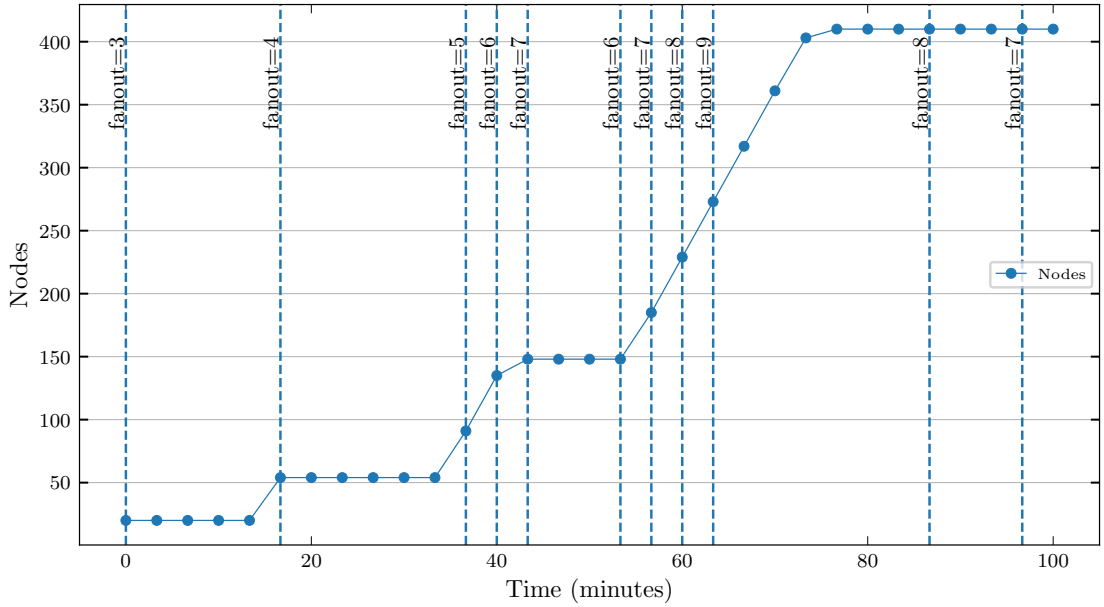


Figure 4.2: Node count evolution over time

Experiment Details The experiment spanned 1 hour and 40 minutes, having 4 district periods, in which new participants join. It begins with 20 nodes. Around minute 17, 34 new nodes join. The second period of influx begins at minute 35, where 94 new nodes join, which means a total of 148 nodes in the system. Just after the 57-minute mark, the final wave of nodes starts joining. 262 nodes start joining the system, making our final amount of nodes to be 410 (just over the 403 needed). As the amount of new nodes here is significant, this process takes around 20 minutes. As such, after all nodes join, the system runs without interference for an additional 25 minutes.

Figure 4.2 presents the evolution of the number of nodes in the system, as well as the decisions made by Overlord. In order to understand such decisions, it is also necessary to understand the monitored reliability of the system during periods where large amounts of nodes join.

Figure 4.3 presents this information. As it is observable, the reliability lowers during periods where new participants are joining the system. This is a consequence of older messages still being processed when new nodes join. For instance, consider a message that is created and disseminated. This message reaches all participants of the system. Before the next collection of metrics, new participants join, and start receiving new messages. However, during the next collection of the system, Overlord has no way of knowing how many nodes were in the system for each message (as this would imply a more complex approach to monitoring metrics). Even with a solution, there is no clear way to decide if a newer node should receive messages sent before it joined the system. Consequentially, when processing messages disseminated previously to new nodes joining the system, the Overlord assumes these nodes should have received the message. This presents us with the lower reliability observed.

I really hate this because I just thought of a simpler solution, but oh well...

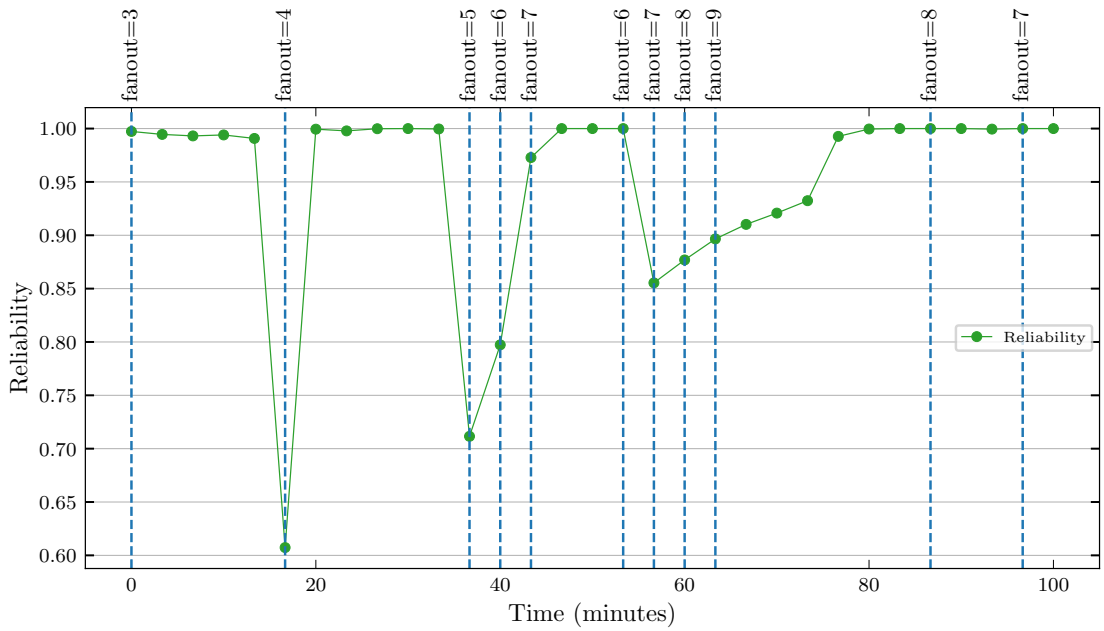


Figure 4.3: Average reliability of Overlord evolution over time.

Consequentially, we observe that Overlord tries to improve the reliability by increasing the fanout parameter. For instance, during the second influx of nodes, the Overlord increases the fanout to 6 and 7, as a consequence of lower reliability, not of the number of nodes increasing. However, just before nodes start joining once more, Overlord starts the optimization phase, checking that reliability is acceptable, and decreasing the fanout to 6. If the new wave of nodes had not joined, Overlord would have decreased the fanout once more, reaching its optimal state.

Moreover, between minute 60 and minute 80, the reliability is constantly low, as the Overlord reaches the maximum fanout it can increment to. Overlord tries to increase the fanout due to the low reliability, however it does not go above 9, since it reaches its maximum incremental value during this wave of new nodes. This restriction would only be removed when the system reached the 403 number of participants, where Overlord could increase the fanout once more to 10, as $\log n$ would have also increased. However, as reliability increases at the end of this wave, Overlord begins the optimization phase, reducing the fanout. Had the experiment ran for longer, the fanout would have reached its optimal state.

This phenomenon is undesired, and is mostly a limitation of choosing to monitor metrics on demand and in a centralized manner. However, considering the bigger picture, the rule we designed should include all possible scenarios that could occur during the application's lifespan. This means that the rule has no way of knowing if the reliability is low as a consequence of the increase of nodes, or as a consequence of another occurring phenomenon, so it decides to take a cautious approach, and increase fanout nonetheless to help reliability. The cap to the increase of the fanout parameter was created as to, in the

Since I now have come up with a solution, I feel like I'm just lying here

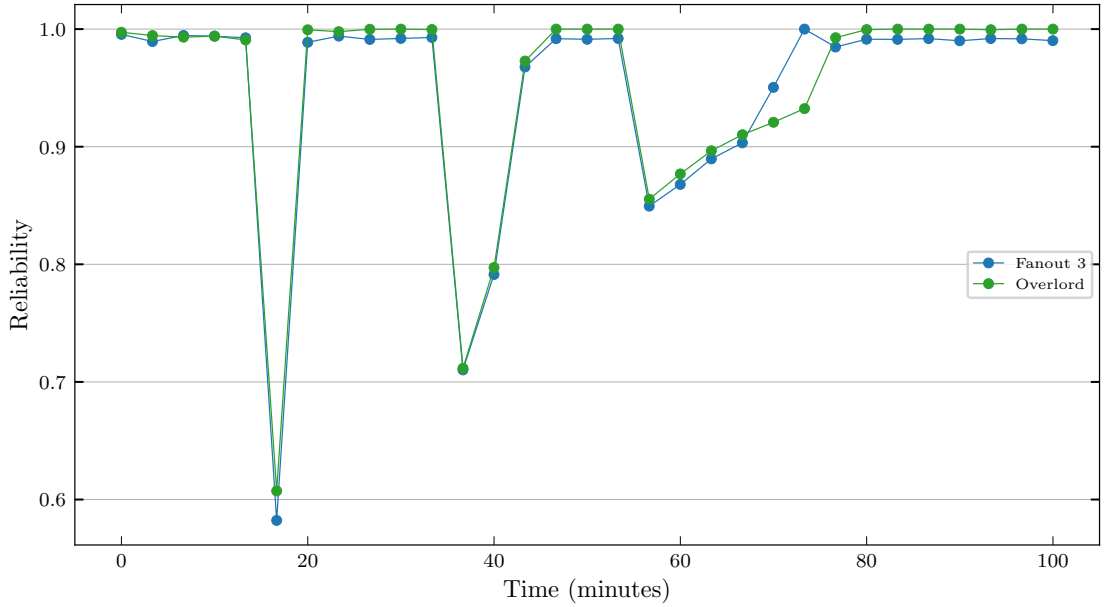


Figure 4.4: Comparison of the average reliability evolution over time.

event that Overlord makes the wrong decision, and increasing the fanout does not improve reliability, the cost of this mistake does not grow infinitely, and is only in place during this disturbance in the reliability. When the system reaches higher reliability, then it begins the phase of optimization, as the system is no longer underperforming. As a consequence of improving system performance, it does not always make the correct decision, however, it always tends to the most optimal state.

I really do not like this part. Should I just say that I didn't have time to implement the correct solution, but the behavior makes sense and improves the system?

After discussing these results, it is important to compare them to a version of this application not running Overlord, with a static configuration, which is depicted in Figure 4.4. Overlord presents the same reliability curve in Figure 4.3, while adding the curve correspondent to application with a static optimistic fanout of 3 (considering the initial state where only 20 nodes are present). Both curves are similar, where Overlord experiences marginal improvements in relation to the static application after the increase in number of participants. The Overlord application, when no new participants are joining the system, reports a reliability of around 100%, where the optimistic application reports a reliability of 99%. Although this result does not portray a significant improvement, a reliability of 99% indicates that, for every message, at least one node does not receive it, which contradicts the main objective of the application.

Moreover, upon normal conditions of the system, Overlord will always tend to adapt the fanout to be $\log n$, which will, with a high probability, mean that every node of the system will receive each message[36]. Even though under normal conditions, the optimistic application does not present significant underperformance, the experiments in Sections 4.1.3.3 and 4.1.3.5, will show how, even with the most optimal fanout value (relatively to the number of participants), when the system is under disturbances, its performance is significantly affected. Consequentially, having a fanout value even lower

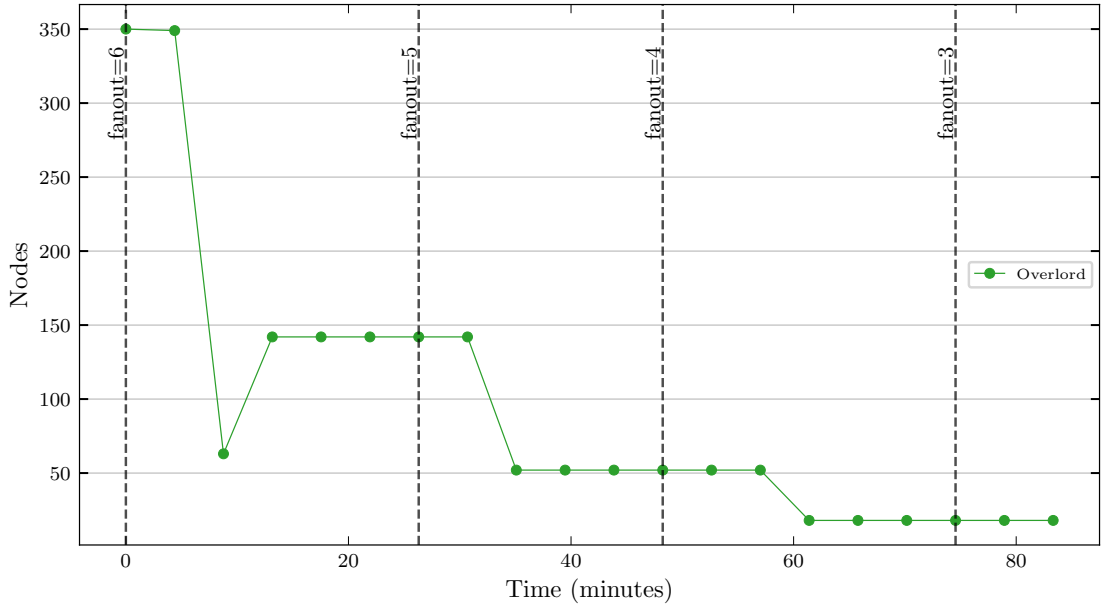


Figure 4.5: Node count evolution over time.

than the optimal one, will only increase this dip in performance.

Thus, our solution tends to correctly adapt the fanout parameter to each influx of nodes, while also proving itself more resilient, as the optimistic version of the application, not only underperforms under normal conditions, while also being extremely vulnerable to any disturbances that might occur in the system.

4.1.3.2 Crashed Nodes

In this experiment, we test how Overlord reacts to the scenario where some nodes of the system crash. Similarly to the experiment in Section 4.1.3.1, we observe how Overlord adapts the fanout parameter to the shifting number of participants in the system, however, in this case, instead of this number increasing, it decreases. It also has the component of crashing the nodes, meaning that they leave the system immediately on command. The main objective with this scenario, is to observe how Overlord optimizes the fanout parameter, decreasing it when the system is performing correctly.

Experiment Details This experiment ran for a total of 83 minutes. The amount of nodes crashed evolves through waves, and in each wave, the number of nodes that leaves the system, is the exact number necessary to reduce the fanout (as dictated by the $\log n$ calculation), as depicted in Table 4.1, meaning that with each wave, we should observe Overlord reduce the fanout. The experiment begins with 350 nodes, while decreasing to 148 at the 15-minute mark. At minute 30, the number of nodes drops once again to 54. The final wave of crashed nodes begins just before the 60-minute mark, where the number of nodes decreases to 20 nodes. The application then runs with the 20 nodes for 25 minutes.

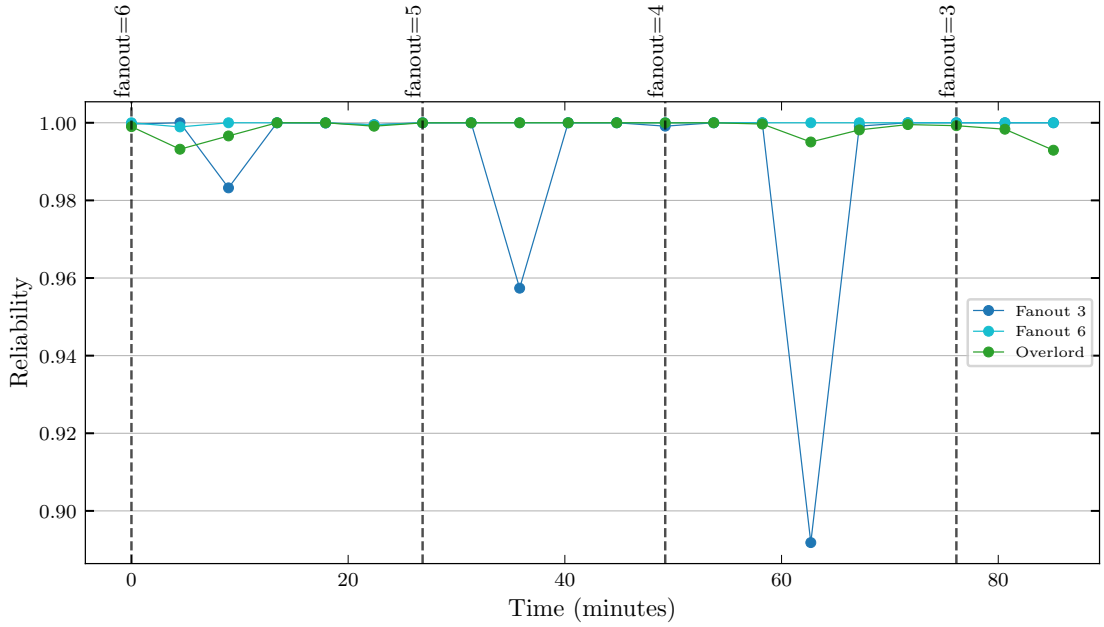


Figure 4.6: Average reliability evolution over time.

Figure 4.5 illustrates the evolution of the number of nodes of the system, as well as the decisions that the Overlord made overtime. As we can observe, with every decrease in number of nodes, the Overlord, after three collections where reliability is stable, decides to optimize the fanout, and reduce it, which is the intended behavior.

An important matter, is concerning the third collection, where the reported number of nodes is just above 50, increasing then to 148. This represents a failure in the collection phase, as when the metric collection tree was being created, some nodes crashed. Both their monitored data, and the one from their children was not considered in that collection. However, in the next metric collection, the amount of nodes reported is the correct one, and it does not prevent Overlord from making the correct decision. More on how loss of information affects Overlord can be found under Section 4.1.3.4.

Now we must compare the performance of our solution to static configuration alternatives. In this case, we will compare it against one that sets its fanout to 6 since it is the most optimal fanout for the number of nodes at the beginning of execution, which is the pessimistic application, as it considers only the worst scenario (in the experiment). We will also run an optimistic version with its fanout set to 3, which corresponds to the most optimal fanout for when the system has only 20 participants, which happens at the end of execution.

Figure 4.6 presents the average reliability reported by the three versions of the application tested. Both the pessimistic and Overlord applications were constant in terms of reported average reliability, reporting always around 100%. However, the optimistic application, even though it reported high reliability during periods of normal execution, during the crash of nodes, reliability was affected, dropping to 90% in the worst case,

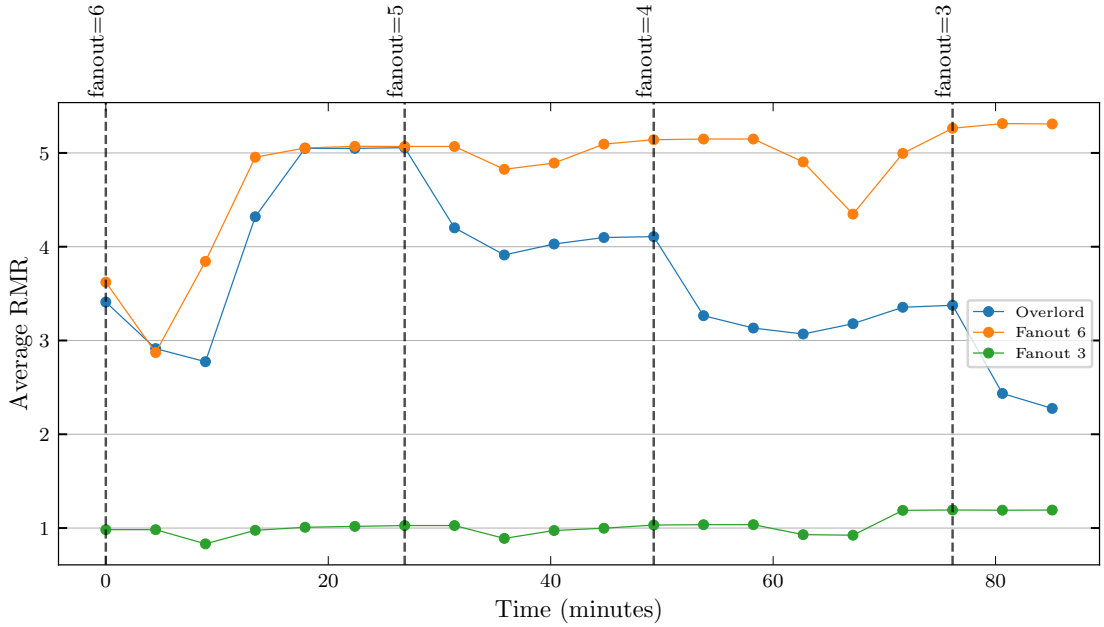


Figure 4.7: Average RMR evolution over time.

whereas the other alternatives did not suffer from this issue. Moreover, since we wish to compare optimization in terms of efficiency, we will be comparing their reported average RMR.

This comparison can be observed in Figure 4.7. At the beginning of execution, both the pessimistic and Overlord applications report the same RMR values. Just when Overlord first decides to optimize, it immediately reports a lower RMR, which considering it keeps the same average reliability, means that it achieves the same performance, with less redundancy. This decrease in RMR happens with every optimization decision of Overlord. Consequentially, with each decision, Overlord's reported RMR begins to draw itself apart from the one reported by the pessimistic application. Towards the end of the experiment, Overlord reports an average RMR of 2, where the pessimistic application still indicates an average RMR of around 5, meaning that for every node, each message is received 3 times more than in the Overlord application, which considering their performance is the comparable, means this value is inadequate.

Moreover, the optimistic application maintains a lower RMR value, being always around 1. This value is optimal as it means that each node receives a message exactly once, with almost no duplicates being received. However, as mentioned previously, this usually comes at the cost of a high reliability value. With Figure 4.6, we can observe this trade-off, as the optimistic application experiences drops in reliability during instability phases of the system. This means that, although this application is efficient and almost generates no duplicate messages, it fails to be reliable, which is the main objective of the application. Moreover, in the next experiments, we can also observe how reliability is affected by other disturbances that can occur in the system, which will reinforce how fragile a lower fanout

(a) Node a will not receive message m since all the neighbors with message m are pit nodes.

(b) As the fanout increased to 5, a now has a new neighbor f which delivers m to it.

Figure 4.8: A visual example of the expected improvements of Overlord on the free riders experiment

option is.

4.1.3.3 Free Riders

This experiment reflects a specific scenario, in which some nodes only want the benefits of participating in the system, without contributing to their responsibilities with the intent of saving resources. Many studies reported high percentages of free-riders in P2P systems[64]. In this case, this implies that these nodes do not forward messages to their neighbors after delivering them. It impacts reliability directly, which provokes Overlord to react and try to increase it. In this experiment, free riders are referred to as "pit nodes", since this better reflects their impact on the system, rather than their intention. A certain percentage of the nodes of the system are chosen to be "pit nodes". In order to increase the reliability under these conditions, Overlord will increase the fanout of the broadcast protocol. Figure 4.8a represents a small portion of a hypothetical system where the fanout parameter is set to 4, and nodes colored red are pit nodes, while green nodes are normal functioning nodes. In this figure, node a will not receive any messages that are forwarded by any of its neighbors because all of them are pit nodes. If we increase the fanout to 5, another node that might contain the same messages can be added to its membership, which means that this new node will forward the message to node a (as is shown in Figure 4.8b). It is possible that this new node is also a pit node, although the fanout can increase again until most nodes have at least one normal node as a neighbor. Another caveat of this experiment, is concerning messages created by pit nodes, which in this case are forwarded to its neighbors.

This paper is very sus. It says this but does not cite anything.

Experiment Details This experiment was done with 200 participants in the system. The defined acceptable reliability is above 98%. We will be comparing three versions of the message dissemination application: (i) A version with an optimistic static fanout, which

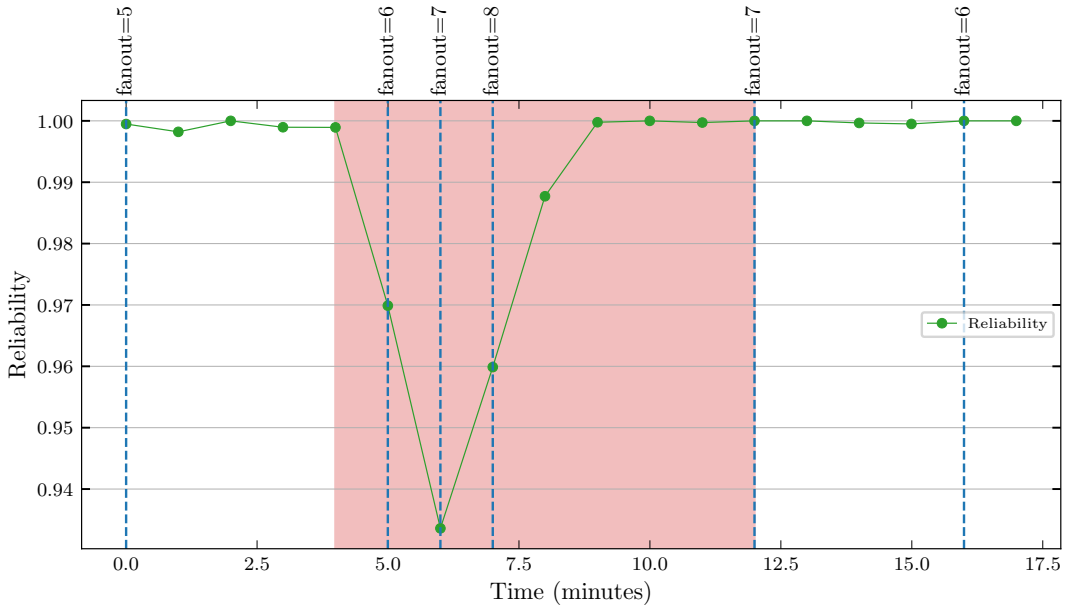


Figure 4.9: Average reliability evolution over time

does not consider any disturbances on the system, (ii) A version with a pessimistic static fanout, considering the consequences of this experiment, and (iii) A version running Overlord and the rule specified in Section 4.1.2, which will adapt the fanout in accordance to the monitored reliability. For 5 metric collections the system works as indented (to simulate normal behavior). At the 5th collection, pit nodes are turned on, and about 50% of the nodes stop forwarding messages. Pit nodes are turned on for 8 more rounds, giving time for Overlord to react and its changes to take effect and become visible.

Another aspect to consider, is the bias that randomization introduces. Since we use HyParView[37], for different experiments, the overlay network connecting the nodes will be different. Moreover, the nodes that become pit nodes are also selected randomly. This is undesirable for comparing different solutions, because these variables can introduce a bias not related to the performance of the different solutions. Since we are comparing three different versions of the same application (optimistic static fanout, pessimistic static fanout and overlord adaptive fanout), it is important that for these different versions, the overlay networks created are the same, and the decided pit nodes are also the same, in order to reduce possible biases. This does not mean that only one membership and pit node set is used in the experiments, but for all experiments, the same sets of memberships and pit nodes were used for all versions of the application.

In Figure 4.9, we present the evolution of the average reliability over time of the application that is running Overlord, in combination with the different fanout values it decided. The area that is shaded with red, represents the time during which pit nodes were activated, and the vertical markers show the decided fanout by Overlord as a consequence of the metrics collected. The initial fanout is 5. From the start of the experiment to the 4-minute mark, the system runs normally, and the reliability is constantly at the 100%

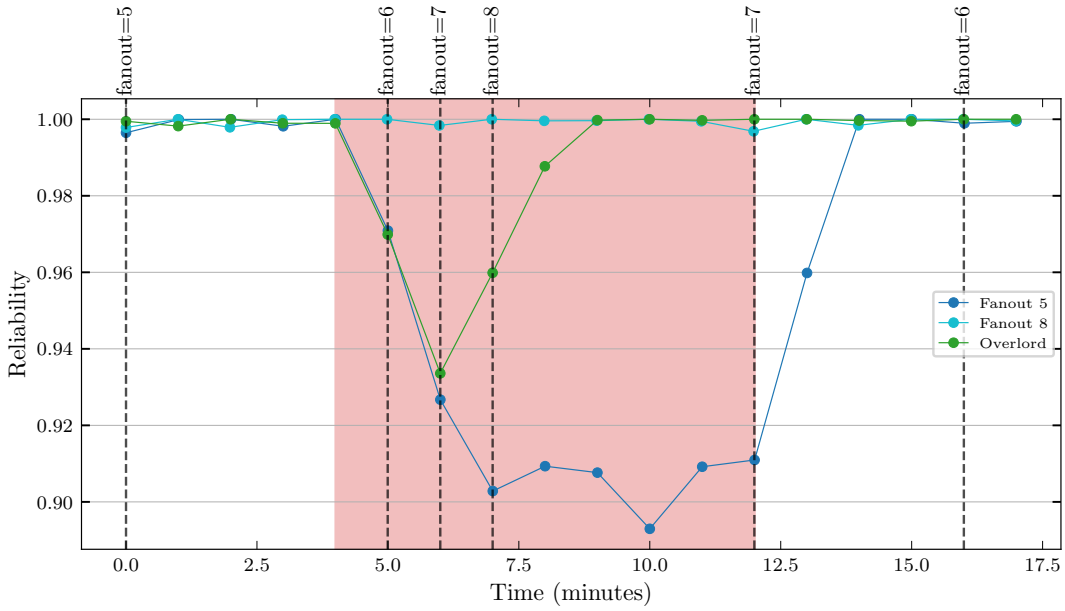


Figure 4.10: Comparison of Average Reliability Evolution over Time

mark. At the 4th minute, pit nodes are activated. The effect is already visible in the next metric collection, as reliability drops to 97%, which is bellow the acceptable threshold. The effect becomes more visible in the next collection, as some messages sent before pit nodes activation are not yet processed at this time. At minute 6, the reliability is at its lowest, being below 94%, which is undesirable. Although it is a high number, for a system tasked with broadcasting messages to all participants, it means that in this case, in average, for every message, 12 nodes did not receive it. This triggers Overlord to increase the fanout once more. As Overlord does so, the average reliability increases too. As older messages are processed, reliability increases until it reaches 100%. At minute 12, pit nodes are turned off. Coincidentally, three runs with 100% reliability have passed and so Overlord decreases the fanout. This happens again, and so at minute 16, Overlord decreases the fanout once more. The rise in the reliability as a consequence of increasing the fanout is a positive result, although, these results do need to be compared to solutions not running Overlord in order to determine its benefits.

Figure 4.10 presents the evolution of the average reliability of the three different versions of the application. As it is visible, the depicted line corresponding to the application running Overlord, is the same as in Figure 4.9.

I have a version of graph in Fig 4.10 without the fanout decisions, is it better?

All applications behave similarly during the first 4 minutes. When the pit nodes are enabled, both the application running Overlord and the application with the fanout set to 5, experience a dip in their average reliability, dropping to around 90%. The optimistic application does not experience any drops in reliability during the system disturbance. The Overlord application, recovers due to increasing its fanout to account for the disturbance.

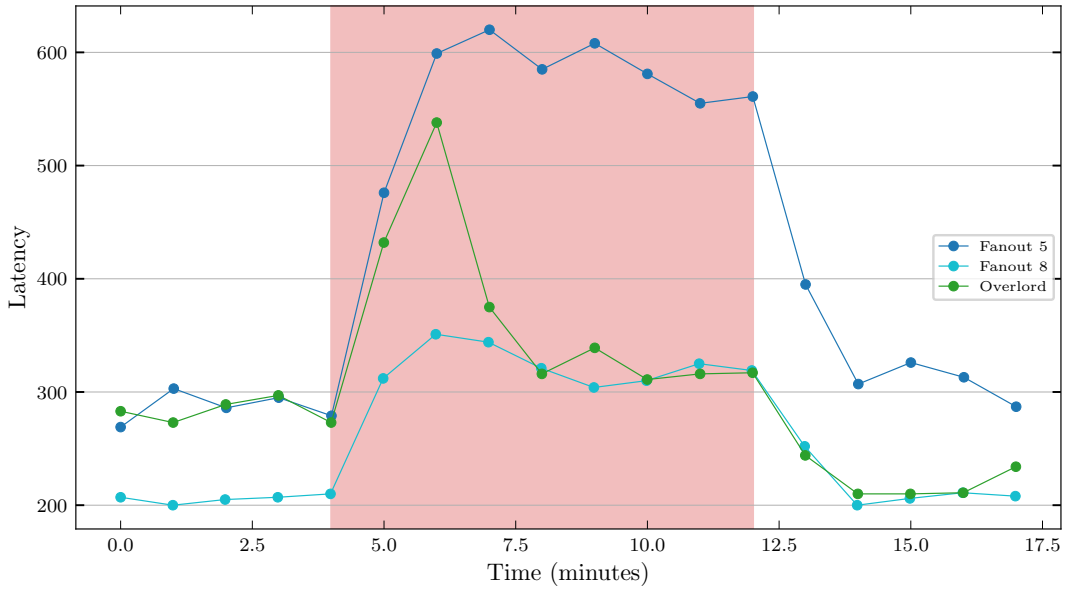


Figure 4.11: Comparison of Average Latency Evolution over Time

In four metric collections, its average reliability returns to the 100% mark, matching the pessimistic application.

Another aspect to consider, is the average latency of messages during the period in which pit nodes are enabled. As is presented in Figure 4.11, all three application behave similarly in the first 5 minutes with an average latency of around 200ms, and close to the 300ms for the Overlord and optimistic fanout applications. Upon the disturbance period, latency rises. However, for the optimistic application, it rises to over 600ms, while for the pessimistic application, only to around 350ms. In the case of Overlord, initially its latency rises to just under 600ms, while dropping with each increase of the fanout until it matches the 300ms latency reported by the pessimistic solution.

After these results, the question of why not simply use the pessimistic solution and have the fanout static at a higher value in case of disturbances arises. This version of the application, even though it is prepared for disturbances in the system, produces a lot of duplicate messages during phases of normal system functioning, which is also undesirable. Even though it suffers no dip in reliability, this comes at a cost, which is the high number of duplicate messages it produces. To measure this, we use the RMR metric, introduced in Section 2.1.3.

Figure 4.12, shows the comparison between the average RMR of each version of the application. Overlord adapts the fanout to achieve the best reliability value, with the lowest RMR[36]. The positive performance of the application with a static fanout set to 8, comes at the cost of a high average RMR. The optimistic application, even though it presents a lower RMR, has the cost of a low reliability in times of system disturbance. The Overlord application, presents an alternative, in which during periods of normal functioning, the RMR is closer to the one presented by the optimistic application, and

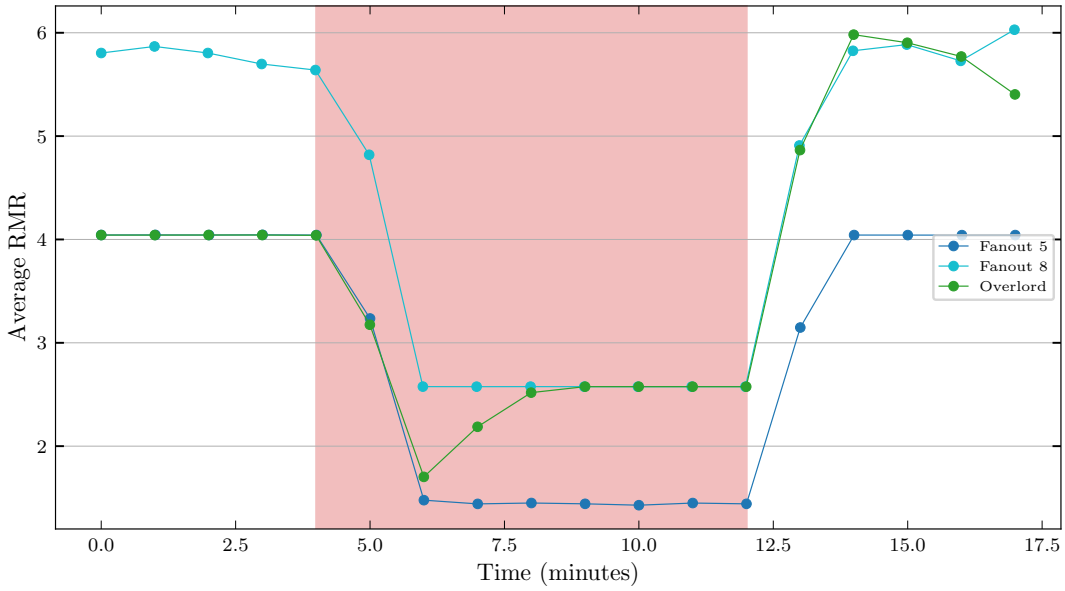


Figure 4.12: Comparison of the Average RMR over Time.

in times of system malfunction, it rises closer to the one presented by the pessimistic application, while providing an optimal reliability value in contrast to the optimistic application.

4.1.3.4 Dealing With Loss of Information

Our solution uses a disposable tree for collecting information. One aspect that surged during the elaboration of this dissertation was the uncertainty of the reaction from Overlord, when it loses information. As such, we decided to use the same scenario referred in Section 4.1.3.3, and compare different experiments with Overlord, where with each, more information would be lost. The objective is to find the percentage of the tree with which Overlord still reports a correct state of the system and still reacts to the disturbance in its environment.

Experiment Details The details of this experiment are the same as in Section 4.1.3.3, however, instead of testing different static configurations, we only test Overlord. Moreover, during the period where pit nodes are enabled (which corresponds to 8 collections), in the first 3 collections, a certain percentage of the metric collection tree is lost. This provides us with a period where a disturbance is affecting system performance, and where Overlord does not have the complete information of the system. After these 3 collections the tree is restored. We tested four different versions of Overlord. The first one, did not lose any percentage of the tree, maintaining all information of the system at all times. Then, we tested versions where only 50%, 25% and 5% of the nodes of the system remained. In the figures, yellow shaded regions represent collections where the information from nodes

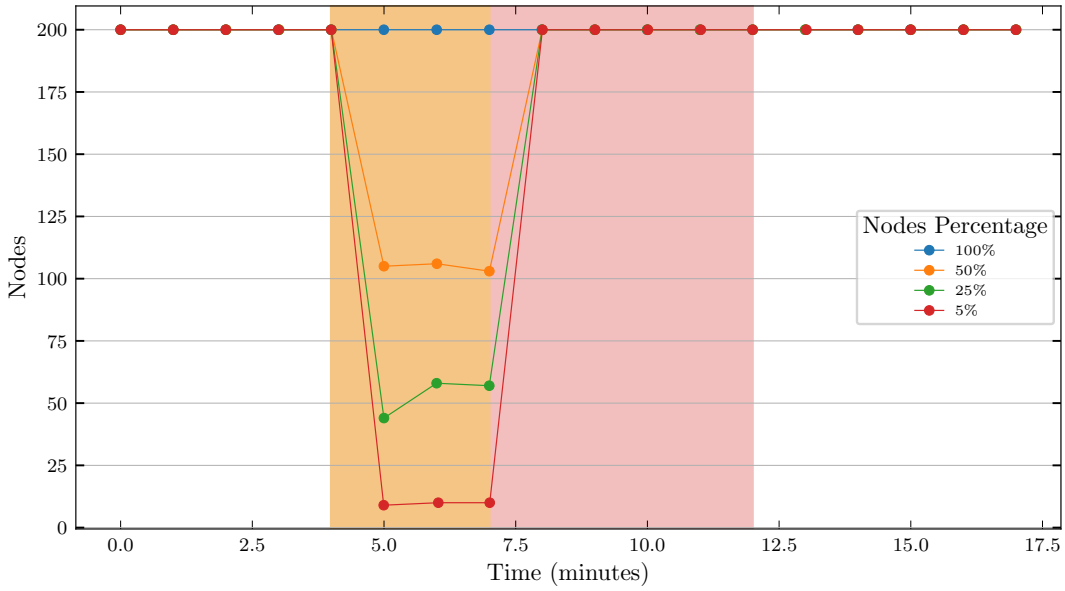


Figure 4.13: Evolution of the number of nodes over time.

was lost, and the red shaded area represents the area where the system disturbance was still in place. The yellow area always overlaps the red one.

Figure 4.13 presents the evolution of the reported number of nodes overtime, for each of the tests. Note that these do not represent the amount of nodes present in the system, but only the number of nodes that successfully reported their information to the Overlord. A node being lost, means that it did not respond to its parent. This means that its child's information will also be compromised. Moreover, the exact number of nodes varies, as a new tree is created for each collection, meaning that different collections can have different trees. Nodes lost due to their parents in one collection, can have a different set of parents and be considered in the next one.

Figure 4.14 represents the evolution of the reported reliability by each of the applications. A way to know if the reported reliability is the correct one, is to observe if any dips occur when the information is restored (after the yellow shading). Since no new disturbances are introduced, there is no reason for a lower reliability, meaning simply that Overlord reported the wrong information.

Moreover, Figure 4.15 represents the decisions made by Overlord. In addition to Figure 4.14, we can establish a reason for each decision of Overlord. First we discuss the version without any loss of information. As expected, it experiences a dip in reliability, which prompts the Overlord to increase the fanout, consequentially, improving it. The version where 50% of the system is lost, is more complicated. After the pit nodes are enabled, this version reports a lower average reliability, which prompts the fanout to increase. However, instead of increasing to fanout 8, it stops at 7. This is due to the cap we introduced to the fanout increments. As Overlord believes there are only around 100 nodes (instead of the 200), it can only increase its fanout to this value. However, this

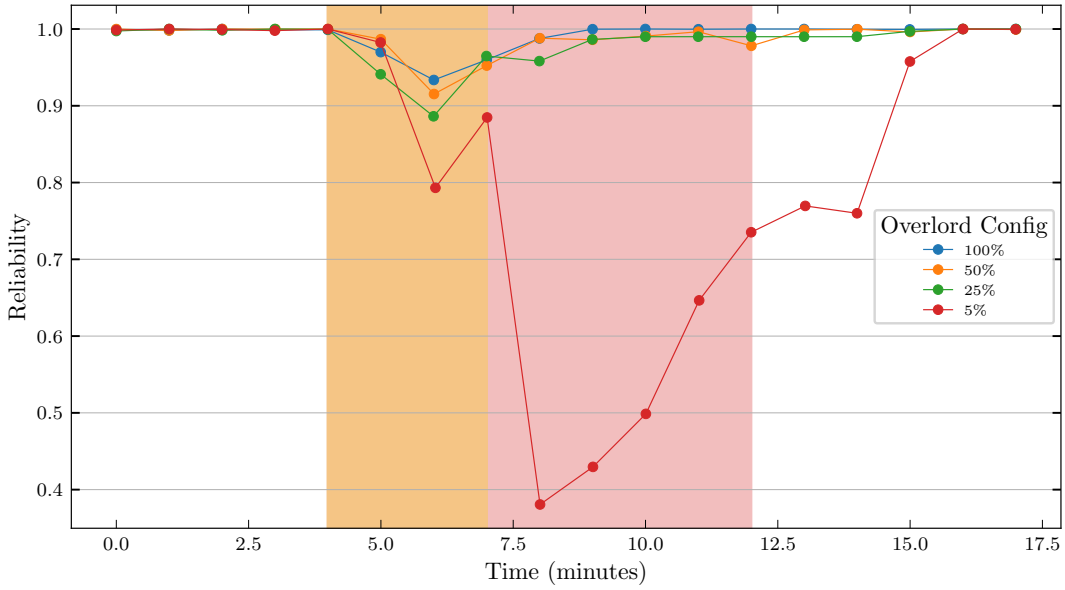


Figure 4.14: Evolution of average reliability over time.

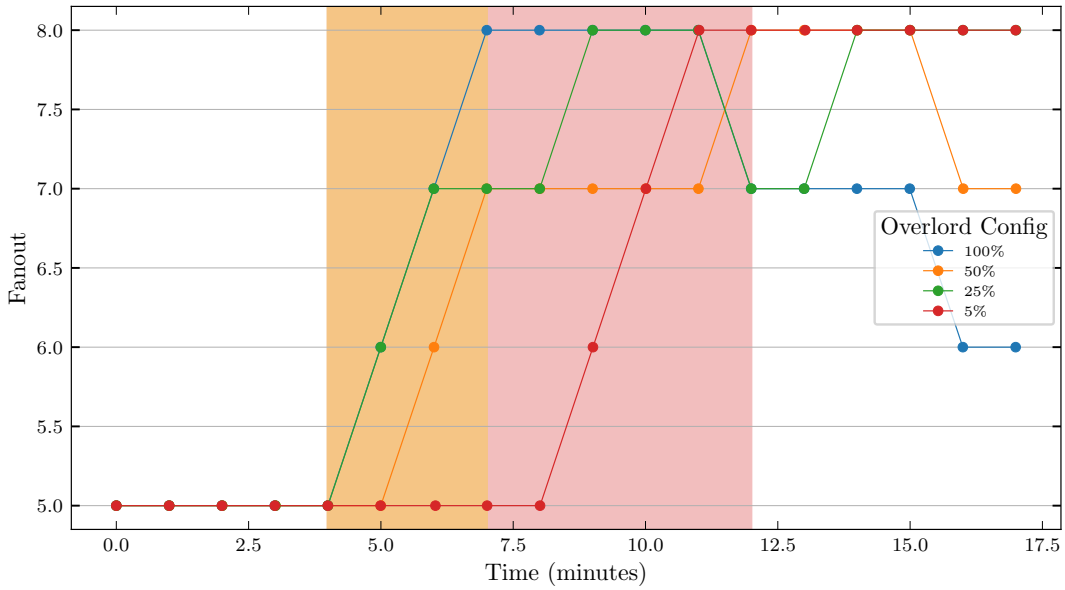


Figure 4.15: Evolution of average reliability over time.

has a positive effect, as after information is restored, reliability is above the accepted threshold. The version with 25% of the system also increases its fanout to 7, however after the information is restored, the reliability reported is sightly lower than the previous one. This indicates us that Overlord did not have a correct value of the real reliability, however, it quickly recovers, increasing its fanout to 8 and restoring the reliability. Finally, the version with only 5% of the system, has a completely incorrect assessment of the reliability, as it reports 80%, which rises to almost 90%. After information is restored, the real reliability is presented, which corresponds to 40%.

In essence, losing information, always affects the decisions Overlord can make, especially due to the cap on reliability that we introduced. As such, decisions should not be affected, as long as the amount of nodes remaining does not affect the optimal fanout. In terms of reported information, the application with 50% of the nodes correctly depicted the true reliability of the system, whereas the remaining versions showed gaps between the information monitored and the true state of the system, with the 5% version of the system, having a significantly wrong idea of the system's performance.

4.1.3.5 Packet Loss

This experiment consists of deliberately dropping a certain percentage of the messages broadcasted by the nodes in the system, meaning that for each message a node receives, there is a percentage p that this message will be lost and not delivered. This will lower the reliability because some messages will not be registered by all nodes, which should prompt Overlord to increase the fanout in order to improve the reliability. Increasing the fanout should work since it means increasing the number of neighbors to which a node sends a message, which increases the number of messages sent.

Figure 4.16 shows the main principle behind this experiment, by representing a small portion of a hypothetical system. In Figure 4.16a, node a creates message m. Since fanout is set to 4, the message is sent to all four neighbors (b, c, d and e), but since 4 out of 5 messages are lost, none of them actually deliver the message. If the fanout increases to 5, as is represented in Figure 4.16b, f is added to a's membership, meaning that f will receive message m. Node f will also send the message to 5 of its neighbors, but it will only be delivered by one of them. If the fanout increases again, 2 neighbors receive the message, and this can happen until enough nodes receive the message in order to disseminate it to the whole system.

Since Babel is built on TCP connections, lost messages is not truly a realistic scenario. If a message is lost, retransmission is triggered until the message is delivered. In this application, a message is only registered as delivered if it is delivered during a certain period of time, so even if a node receives a retransmitted message, it may not be delivered in time[35]. As such, this experiment is not truly a realistic scenario, and was created as a methodical and more deterministic way of lowering the reliability of the system on demand, and observe the effects of adapting the fanout parameter. Instead of devising a

Here choose if we can justify this experiment being real or if it is just a "lower

(a) Since 4 out of 5 messages are lost, none of the neighbors of a receive message m.

(b) As the fanout increased to 5, a now has a new neighbor f receives m.

Figure 4.16: A visual example of the expected improvements of Overlord on the packet loss experiment.

What more experiments could we have done?

more realistic different scenario, which would make observations between comparisons of different solutions more difficult, this experiment is a simple disturbance that reduces the number of biases that can happen during testing, and helps observe the differences between having a static fanout, and having a fanout adapted to disturbances.

Experiment Details The experiment was conducted with 200 nodes participating in the system, with the threshold of acceptable reliability at 98%. As in the other experiments, we will be comparing three versions of the application (i) A version with an optimistic static fanout, which does not consider any disturbances on the system, (ii) A version with a pessimistic static fanout, considering the consequences of this experiment, and (iii) A version running Overlord and the rule specified in Section 4.1.2, which will adapt the fanout in accordance to the monitored reliability. During the first 5 metric collections, the system works as intended, without any disturbances in order to simulate normal operation. At the 5th metric collection, packet dropping is turned on, and every message received from other nodes has a 25% chance of being discarded. This behavior lasts for 8 more rounds, in order to allow Overlord to react to any issues found, and at the end of these 8 rounds, normal behavior is restored.

Similarly to the free riders experiment, HyParView can introduce a certain bias, making it harder to compare different versions of the same application due to the membership changing. In the same way as mentioned in Section 4.1.3.3, the same set of HyParView generated memberships was used for every version of the application tested in order to reach a less biased conclusion.

Figure 4.17 presents the fluctuations of the average reliability over time monitored by Overlord, as well as the different decisions it made on each metric collection. The red shaded area represents the period in which packets were being lost. During the first 4 minutes, the system runs as expected with no disturbance, during 5 metric collections.

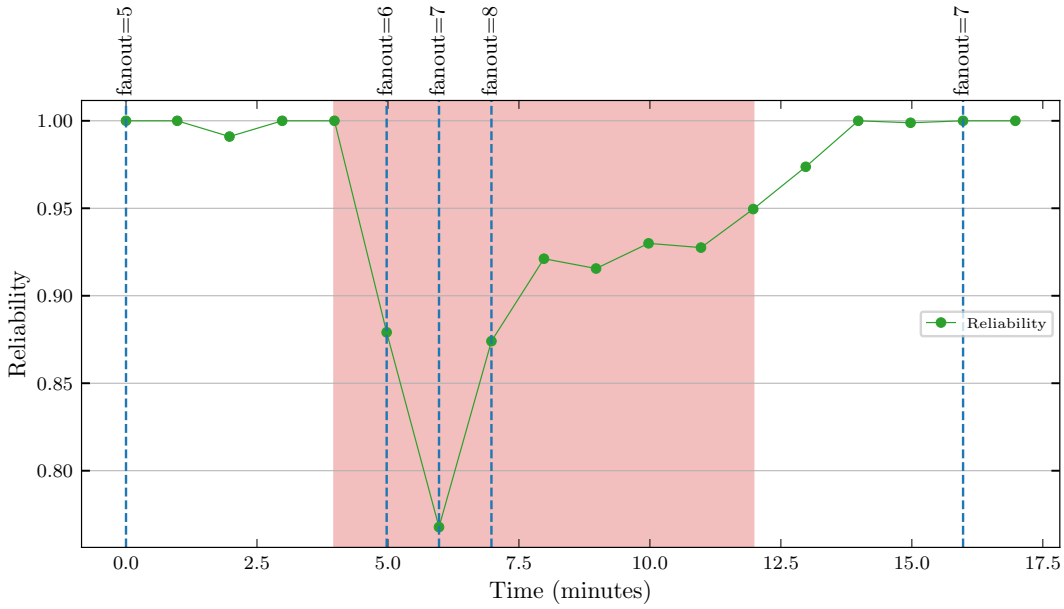


Figure 4.17: Average reliability evolution over Time

At the 5th collection, the packet loss is enabled in all nodes, and packets are lost with a probability of 25%. As observable in Figure 4.17, the first dip occurs at the 5-minute mark with an average reliability of 87%. Due to some messages that were sent before packet loss activation still being processed, the reliability is not yet at it lowest. However, it is low enough for Overlord to increase the fanout. At minute 6, the average reliability is at its lowest, with a value of 76%, meaning that Overlord increases the fanout to 7. This results in an improvement to the reliability, leading it to rise to 87%, which is still below the acceptable threshold. Consequently, Overlord increases the fanout to 8, which prompts the reliability to increase to around 92%. This value is still under the acceptable threshold of 98%, however, Overlord cannot increase it again due to having reached the maximum value for the fanout parameter defined for this experiment, which is $\log 200 + 3 = 8$. This cap prevents the fanout growing infinitely. If this value were adjusted, Overlord would increase it again, as it was depicted in the free riders experiment, Section 4.1.3.3.

Maybe this isn't good. Because I'm essentially saying that Overlord does not increase the fanout on its own to get the best reliability. I have tests with a lower packet drop percentage (15%) which don't present the same problems. We have to decide if it's better to show a "feature" of the rule in order for it not to grow infinitely, or if It's too much of a "My solution doesn't work as well as it should because I limited it", and use a more perfect version of the test.

As mentioned previously, these results alone are not enough to draw conclusions, so two other versions of the application were also tested in order to compare these different options, and determine what differences using Overlord provides. These different versions, are the ones referred in Section 4.1.3.

I have a version of graph in Fig 4.18 without the fanout decisions, is it better?

Figure 4.18 shows the evolution of the average reliability over time of the three different

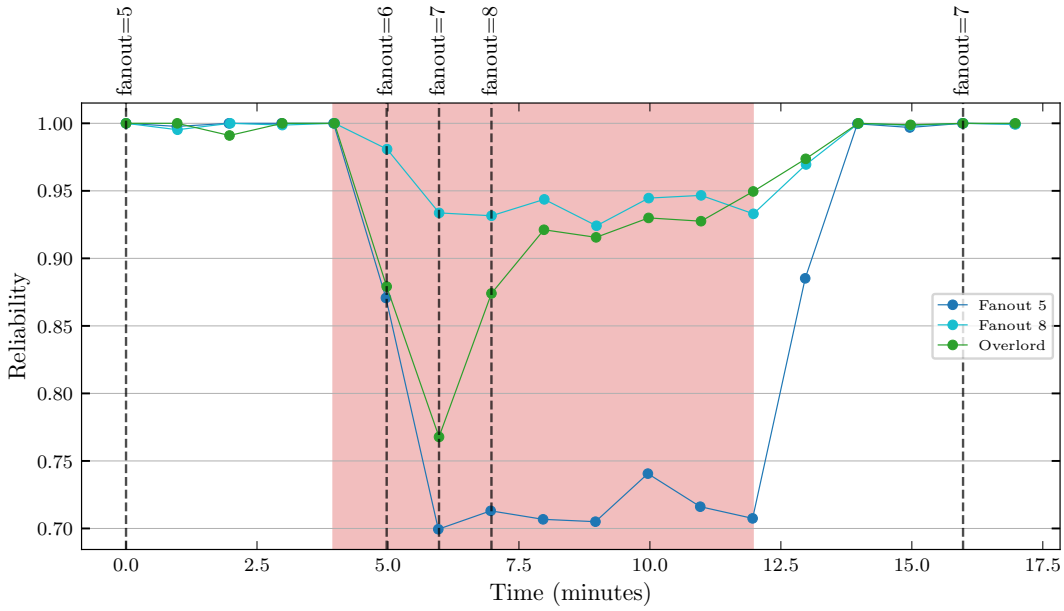


Figure 4.18: Comparison of Average Reliability Evolution over Time

versions of the application that were tested in this experiment. The overlord curve is the same as shown in Figure 4.17. During the normal functioning period of the application, all three applications register a similar average reliability of 100%. The first collection after enabling the packet loss shows that both the Overlord and the optimistic application experience a similar dip in their average reliability, falling to around 87%. At the 7th minute, reliability keeps lowering, however, the optimistic application experiences a lower reliability of 70%, in contrast to the Overlord application, which reports a reliability of 76%. This is due to Overlord having changed its fanout to 6. This distancing from the optimistic application continues as Overlord increases its fanout (as shown in Figure 4.17), until it matches the pessimistic application with an average reliability of around 92%. The optimistic application remains in the 70% reliability for the entirety of the packet loss period. Consequentially, for a single message, out of the 200 nodes participating in the system, about 60 nodes do not receive that message.

Moreover, it is also important to note that the latency during the disturbance period increases significantly for the optimistic fanout application, going from around 200ms in periods of normal functioning, to over 700ms during packet loss, as shown in Figure 4.19. The pessimistic application also experiences a higher latency, however much lower than the optimistic application, with a latency of 400ms. Overlord, initially follows the optimistic application in terms of latency values, but as it increases its fanout, this value drops from 600ms in the second metric collection during the packet loss period, to match the pessimistic application around 400ms.

Similarly to the free riders experiment, simply using a static pessimistic fanout is not a viable option as it produces a higher RMR than needed during the normal functioning period of the test. As presented by Figure 4.20, during the first 5 minutes of the experiment,

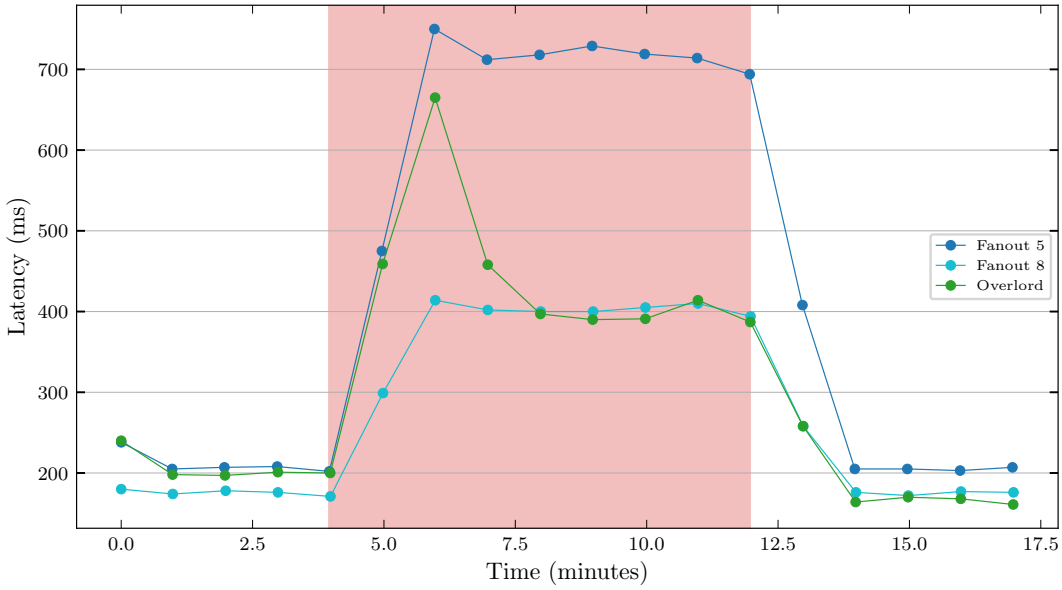


Figure 4.19: Comparison of Average Latency Evolution over Time

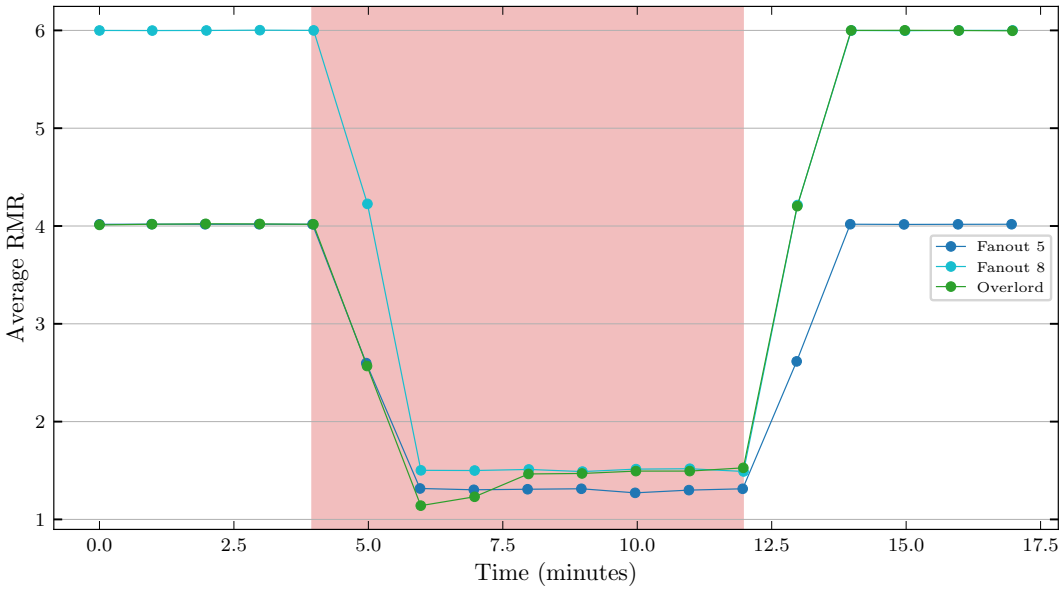


Figure 4.20: Comparison of Average Reliability Evolution over Time

the pessimistic application registers an average RMR of around 6, while presenting the same average reliability as the other two applications. Moreover, the pessimistic application has a static fanout relatively to the 25% of packets being lost. If this value were higher, this version of the application would have a worse performance, whereas Overlord would adapt the fanout to achieve a better reliability value.

4.2 Summary

In this chapter, we evaluated the developed architecture, implementing a message dissemination application as a case study on top of the created framework. First we explain the application, giving an overview from the "point of view" of a developer using Overlord to monitor and adapt this system. Then, we presented the impact of the average reliability of the system, which helps understand system performance, since it directly indicates the percentage of nodes that received messages. We also detailed how Overlord uses the rules tailored to this system in order to adapt the fanout parameter according to the average reliability monitored in each metric collection. Finally, we evaluated how Overlord performed in adapting this application by testing it in different possible scenarios that can occur during execution of these systems. Evaluation was performed by testing the adaptive solution against optimistic and pessimistic applications, with a static fanout. Our solution showed that it is capable of adapting to periods of instability, not succumbing to such issues as the optimistic application, while also being capable of optimizing itself during periods of positive performance, not introducing more duplicate messages than needed as does the pessimistic application.

5

Conclusion

In this chapter we present the main achievements of this dissertation. This includes final deliberations on the solution and its performance on the implemented case study. We also elaborate on possible future work in order to further improve on the developed solution.

5.1 Conclusions and Contributions

With this dissertation, we delved into a gap in the scientific field of self-adaptive systems, exploring the possibility of adapting protocol parameters running in decentralized systems, with one central node. Evaluation of the case study proved this kind of architecture is possible in decentralized systems, as we demonstrated that with our implementation of Overlord. In every scenario presented, Overlord presented a better alternative, running optimistically during normal periods of execution, however switching to more costly approach when disturbances caused the system to perform undesirably.

With this thesis, we made several contributions to the field of self-adaptation, providing a new architecture of designing self-adaptive systems, that allows a central node to measure the performance of the entire system, and allows developer defined rules, which are evaluated, and their outcome reconfigurations enforced through the entire system.

Apart from this, another contribution was its implementation using the Babel framework[22] in the context of the research project TaRDIS[67], which since we approached its development in a modular fashion, allowed for not only a fully functioning framework for self-adaptation of protocols, but also an implementation of the MON-Collect protocol, that allows transporting large information from nodes of a system to a single node, with the added bonus of providing INA for aggregating the data along the way. Moreover, we also contributed with a self monitoring gossip based broadcast protocol implementation, which is ready with metric collection, and also provided the implementation of the relevant aggregations, both in-node and in-network, for any developer using the metrics framework to use. Furthermore, we also contributed with a functioning Rule Engine and Reconfiguration Receiver, capable of, upon request, evaluating registered rules with access to provided monitored data and its history, and enforce these reconfigurations on

the protocols of every node of the system. Finally, we provide a functioning message application which, using the developed framework, is capable of self-adaptation of the broadcast protocol it runs, using a rule that controls its fanout parameter based on the average reliability of the system.

Finally, we also contribute with the writing of a paper that is to be submitted to the next 45th International Symposium on Reliable Distributed Systems (SRDS 2026).

I don't think I can say much more here, but this looks empty.

5.2 Future Work

While the proposed architecture of Overlord demonstrates effectiveness, it also highlights opportunities for further development.

The first and most obvious one is the single point of failure on Overlord's architecture. As mentioned in Chapter 2, centralized systems contain a single point of failure, meaning that, if the Overlord node fails, the entire system will still function, however it will not react to changes in the system as the central node is absent. Overlord should always be closer to a super-node (as explained in Section 2.1.1.1), however even a node with these characteristics can be subjected to failures, such as losing connection or crashing. Because of this, one possible line of improving on this issue is to add a leader election algorithm (of a select array of super-nodes). Like this, when the overlord node is not accessible, a new one could be elected and the system would keep functioning as intended.

Moreover, another research direction could be to explore the possibility of a system with an architecture similar to Overlord adapting entire protocol stacks, instead of simply adapting parameters of protocols.

Another aspect worth mentioning is the matter of larger systems, with thousands of nodes. Since, due to testing limitations, we only performed tests on large systems (between around 200-400 nodes). This means that there is no way of knowing how Overlord would behave in a much larger system. As mentioned previously, Overlord collects metrics by creating a tree with all the participants of a system. This is a process that takes time, and so, the period between metric collections is limited by the time that an Overlord run takes in its entirety. Moreover, the longer this process takes, the more changes could happen to the system, leaving the created tree vulnerable to failures. In order to speed up this process, one idea is to instead of having a fully centralized architecture, to create groups of nodes, and have a leader per group of nodes. This leader would behave like the Overlord in terms of how it collects monitored data, however the component of evaluating rules and adapting the system would be stripped out. The Overlord node would trigger a new collection by sending an instruction to each leader, which would consequentially begin a MON-Collect run on each of its group of nodes. Upon completion of this process, each leader would then send the outcome of this operation to the Overlord node so that it could evaluate the defined rules.

Must update by the end.

Bibliography

[1] A. Alasaad, S. Gopalakrishnan, and V. C. Leung. "Extending P2PMesh: Topology-aware Schemes for Efficient Peer-to-peer Data Sharing in Wireless Mesh Networks". In: *Wireless Communications and Mobile Computing* 13.5 (2013-04), pp. 483–499. ISSN: 1530-8669, 1530-8677. DOI: 10.1002/wcm.1115. (Visited on 2026-03-28) (cit. on p. 7).

[2] R. Anthony. "A Policy-Definition Language and Prototype Implementation Library for Policy-based Autonomic Systems". In: *2006 IEEE International Conference on Autonomic Computing*. Dublin, Ireland: IEEE, 2006, pp. 265–276. ISBN: 978-1-4244-0175-8. DOI: 10.1109/ICAC.2006.1662407. (Visited on 2025-01-22) (cit. on p. 25).

[3] P. Arcaini, E. Riccobene, and P. Scandurra. "Modeling and Analyzing MAPE-K Feedback Loops for Self-Adaptation". In: *2015 IEEE/ACM 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. Florence, Italy: IEEE, 2015-05, pp. 13–23. ISBN: 978-0-7695-5567-6. DOI: 10.1109/SEAMS.2015.10. (Visited on 2025-01-22) (cit. on p. 19).

[4] R. Bakhshi et al. "Formal Analysis Techniques for Gossiping Protocols". In: *ACM SIGOPS Operating Systems Review* 41.5 (2007-10), pp. 28–36. ISSN: 0163-5980. DOI: 10.1145/1317379.1317385. (Visited on 2025-01-24) (cit. on p. 9).

[5] A. Bandara et al. "A Goal-Based Approach to Policy Refinement". In: *Proceedings. Fifth IEEE International Workshop on Policies for Distributed Systems and Networks, 2004. POLICY 2004.* Yorktown Heights, NY, USA: IEEE, 2004, pp. 229–239. ISBN: 978-0-7695-2141-1. DOI: 10.1109/POLICY.2004.1309175. (Visited on 2025-01-22) (cit. on p. 24).

[6] K. P. Birman et al. "Bimodal Multicast". In: *ACM Transactions on Computer Systems* 17.2 (1999-05), pp. 41–88. ISSN: 0734-2071, 1557-7333. DOI: 10.1145/312203.312207. (Visited on 2025-02-10) (cit. on pp. 9, 13).

[7] F. Bordignon and G. Tolosa. *Gnutella: Distributed System for Information Storage and Searching Model Description* (cit. on p. 15).

[8] M. Castro et al. "Scribe: A Large-Scale and Decentralized Application-Level Multicast Infrastructure". In: *IEEE Journal on Selected Areas in Communications* 20.8 (2002-10), pp. 1489–1499. ISSN: 0733-8716. DOI: 10.1109/JSAC.2002.803069. (Visited on 2026-03-27) (cit. on p. 17).

[9] J. Chomicki and J. Lobo. "Monitors for History-Based Policies". In: *Policies for Distributed Systems and Networks*. Ed. by G. Goos et al. Vol. 1995. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 57–72. ISBN: 978-3-540-41610-4 978-3-540-44569-2. DOI: 10.1007/3-540-44569-2_4. (Visited on 2025-01-23) (cit. on p. 25).

[10] I. Clarke et al. "Freenet: A Distributed Anonymous Information Storage and Retrieval System". In: *Designing Privacy Enhancing Technologies*. Ed. by G. Goos et al. Vol. 2009. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 46–66. ISBN: 978-3-540-41724-8 978-3-540-44702-3. DOI: 10.1007/3-540-44702-4_4. (Visited on 2025-01-25) (cit. on p. 15).

[11] P. A. Costa and J. Leitao. "Practical Continuous Aggregation in Wireless Edge Environments". In: *2018 IEEE 37th Symposium on Reliable Distributed Systems (SRDS)*. Salvador, Brazil: IEEE, 2018-10, pp. 41–50. ISBN: 978-1-5386-8301-9. DOI: 10.1109/SRDS.2018.00015. (Visited on 2025-01-22) (cit. on p. 21).

[12] P. A. Costa, J. Leitao, and Y. Psaras. "Studying the Workload of a Fully Decentralized Web3 System: IPFS". In: *Distributed Applications and Interoperable Systems*. Ed. by M. Patino-Martinez and J. Paulo. Vol. 13909. Cham: Springer Nature Switzerland, 2023, pp. 20–36. ISBN: 978-3-031-35259-1 978-3-031-35260-7. DOI: 10.1007/978-3-031-35260-7_2. (Visited on 2025-02-04) (cit. on p. 1).

[13] N. Damianou et al. "The Ponder Policy Specification Language". In: *Policies for Distributed Systems and Networks*. Ed. by G. Goos et al. Vol. 1995. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 18–38. ISBN: 978-3-540-41610-4 978-3-540-44569-2. DOI: 10.1007/3-540-44569-2_2. (Visited on 2025-01-22) (cit. on p. 25).

[14] A. Demers et al. "Epidemic Algorithms for Replicated Database Maintenance". In: *Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing - PODC '87*. Vancouver, British Columbia, Canada: ACM Press, 1987, pp. 1–12. DOI: 10.1145/41840.41841. (Visited on 2026-03-27) (cit. on pp. 9, 13).

[15] P. Druschel and A. Rowstron. "PAST: A Large-Scale, Persistent Peer-to-Peer Storage Utility". In: *Proceedings Eighth Workshop on Hot Topics in Operating Systems*. Elmau, Germany: IEEE Comput. Soc, 2001, pp. 75–80. ISBN: 978-0-7695-1040-8. DOI: 10.1109/HOTOS.2001.990064. (Visited on 2025-01-25) (cit. on p. 17).

[16] F. O. Ehiagwina et al. "A State-of-the-Art Survey of Peer-to-Peer Networks: Research Directions, Applications and Challenges". In: *Journal of Engineering Research and Sciences* 1.1 (2022-02), pp. 19–38. ISSN: 28314085. DOI: 10.55708/js0101003. (Visited on 2025-01-24) (cit. on p. 7).

[17] Eng Keong Lua et al. "A Survey and Comparison of Peer-to-Peer Overlay Network Schemes". In: *IEEE Communications Surveys & Tutorials* 7.2 (2005), pp. 72–93. ISSN: 1553-877X, 2373-745X. DOI: 10.1109/COMST.2005.1610546. (Visited on 2025-02-11) (cit. on p. 8).

[18] P. T. Eugster et al. "From Epidemics to Distributed Computing". In: *IEEE Computer* 37 (2004), pp. 60–67. URL: https://api.semanticscholar.org/CorpusID:1489599 (cit. on p. 9).

[19] E. Fasolo et al. "In-Network Aggregation Techniques for Wireless Sensor Networks: A Survey". In: *IEEE Wireless Communications* 14.2 (2007-04), pp. 70–87. ISSN: 1536-1284. DOI: 10.1109/MWC.2007.358967. (Visited on 2025-01-22) (cit. on p. 22).

[20] A. Feng et al. "In-Network Aggregation for Data Center Networks: A Survey". In: *Computer Communications* 198 (2023-01), pp. 63–76. ISSN: 01403664. DOI: 10.1016/j.comcom.2022.11.004. (Visited on 2025-01-22) (cit. on p. 21).

[21] M. J. Fischer. "The Consensus Problem in Unreliable Distributed Systems (a Brief Survey)". In: *Foundations of Computation Theory*. Ed. by G. Goos et al. Vol. 158. Berlin, Heidelberg: Springer Berlin Heidelberg, 1983, pp. 127–140. ISBN: 978-3-540-12689-8 978-3-540-38682-7. DOI: 10.1007/3-540-12689-9_99. (Visited on 2026-03-27) (cit. on p. 7).

[22] P. Fouto et al. "Babel: A Framework for Developing Performant and Dependable Distributed Protocols". In: *2022 41st International Symposium on Reliable Distributed Systems (SRDS)*. 2022 41st International Symposium on Reliable Distributed Systems (SRDS). Vienna, Austria: IEEE, 2022-09, pp. 146–155. ISBN: 978-1-6654-9753-4. DOI: 10.1109/SRDS55811.2022.00022. URL: https://ieeexplore.ieee.org/document/9996836/ (visited on 2025-01-22) (cit. on pp. 2, 3, 32, 37, 83).

[23] D. Garlan et al. "Rainbow: Architecture-Based Self-Adaptation with Reusable Infrastructure". In: *Computer* 37.10 (2004-10), pp. 46–54. ISSN: 0018-9162. DOI: 10.1109/MC.2004.175. URL: http://ieeexplore.ieee.org/document/1350726/ (visited on 2025-02-11) (cit. on pp. 2, 31).

[24] K. Goudarzi and J. Kramer. "Maintaining Node Consistency in the Face of Dynamic Change". In: *Proceedings of International Conference on Configurable Distributed Systems*. Annapolis, MD, USA: IEEE Comput. Soc. Press, 1996, pp. 62–69. ISBN: 978-0-8186-7395-5. DOI: 10.1109/CDS.1996.509347. (Visited on 2025-01-23) (cit. on p. 26).

[25] M. Hiltunen et al. "Survivability through Customization and Adaptability: The Cactus Approach". In: *Proceedings DARPA Information Survivability Conference and Exposition. DISCEX'00*. DARPA Information Survivability Conference and Exposition. DISCEX'00. Vol. 1. Hilton Head, SC, USA: IEEE Comput. Soc, 1999, pp. 294–307. ISBN: 978-0-7695-0490-2. DOI: 10.1109/DISCEX.2000.825033. URL: http://ieeexplore.ieee.org/document/825033/ (visited on 2025-01-31) (cit. on pp. 2, 28).

[26] P. Horn. *Autonomic Computing: IBM's Perspective on the State of Information Technology*. 2001 (cit. on pp. 2, 18).

[27] M. C. Huebscher and J. A. McCann. "A Survey of Autonomic Computing—Degrees, Models, and Applications". In: *ACM Computing Surveys* 40.3 (2008-08), pp. 1–28. ISSN: 0360-0300, 1557-7341. DOI: 10.1145/1380584.1380585. (Visited on 2025-01-22) (cit. on pp. 18–21, 24, 26).

[28] *IPFS: Interplanetary File System*. 2015 (cit. on p. 1).

[29] Joanne Taaffe. *Six Companies Dominate Network Traffic as Speed and Access Grows*. 2021-11 (cit. on p. 1).

[30] H. Kamoda et al. *Policy Conflict Analysis Using Free Variable Tableaux for Access Control in Web Services Environments*. 2005-01 (cit. on p. 24).

[31] K. Kaur and Y. Kumar. "Swarm Intelligence and Its Applications towards Various Computing: A Systematic Review". In: *2020 International Conference on Intelligent Engineering and Management (ICIEM)*. London, United Kingdom: IEEE, 2020-06, pp. 57–62. ISBN: 978-1-7281-4097-1. DOI: 10.1109/ICIEM48762.2020.9160177. (Visited on 2025-02-07) (cit. on p. 2).

[32] J. Kephart. "Research Challenges of Autonomic Computing". In: *Proceedings. 27th International Conference on Software Engineering, 2005. ICSE 2005.* St. Louis, MO, USA: IEEe, 2005, pp. 15–22. DOI: 10.1109/ICSE.2005.1553533. (Visited on 2025-01-23) (cit. on p. 19).

[33] A.-M. Kermarrec, L. Massoulie, and A. Ganesh. "Probabilistic Reliable Dissemination in Large-Scale Systems". In: *IEEE Transactions on Parallel and Distributed Systems* 14.3 (2003-03), pp. 248–258. ISSN: 1045-9219. DOI: 10.1109/TPDS.2003.1189583. (Visited on 2026-03-26) (cit. on pp. 9, 11, 13, 58).

[34] J. Kramer and J. Magee. "Analysing Dynamic Change in Software Architectures: A Case Study". In: *Proceedings. Fourth International Conference on Configurable Distributed Systems (Cat. No.98EX159)*. Annapolis, MA, USA: IEEE Comput. Soc, 1998, pp. 91–100. ISBN: 978-0-8186-8451-7. DOI: 10.1109/CDS.1998.675762. (Visited on 2025-01-23) (cit. on p. 26).

[35] K. Lee and S. Chanson. "Packet Loss Probability for Real-Time Wireless Communications". In: *IEEE Transactions on Vehicular Technology* 51.6 (2002-11), pp. 1569–1575. ISSN: 0018-9545. DOI: 10.1109/TVT.2002.804843. (Visited on 2026-03-28) (cit. on p. 77).

[36] J. Leitao, J. Pereira, and L. Rodrigues. "Epidemic Broadcast Trees". In: *2007 26th IEEE International Symposium on Reliable Distributed Systems (SRDS 2007)*. Beijing, China: IEEE, 2007-10, pp. 301–310. ISBN: 978-0-7695-2995-0. DOI: 10.1109/SRDS.2007.27. (Visited on 2026-03-19) (cit. on pp. 1, 3, 10, 12, 13, 40, 63, 66, 73).

[37] J. Leitao, J. Pereira, and L. Rodrigues. "HyParView: A Membership Protocol for Reliable Gossip-Based Broadcast". In: *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07)*. Edinburgh, UK: IEEE, 2007-06, pp. 419–429. ISBN: 978-0-7695-2855-7. DOI: 10.1109/DSN.2007.56. (Visited on 2025-02-06) (cit. on pp. 8, 11, 71).

[38] J. Leitao, L. Rosa, and L. Rodrigues. "Large-Scale Peer-to-Peer Autonomic Monitoring". In: *2008 IEEE Globecom Workshops*. New Orleans, Louisiana, USA: IEEE, 2008-11, pp. 1–5. ISBN: 978-1-4244-3061-1. DOI: 10.1109/GLOCOMW.2008.ECP.18. (Visited on 2024-05-02) (cit. on p. 14).

[39] J. Leitão, J. Pereira, and L. Rodrigues. "Gossip-Based Broadcast". In: *Handbook of Peer-to-Peer Networking*. Ed. by X. Shen et al. Boston, MA: Springer US, 2010, pp. 831–860. ISBN: 978-0-387-09750-3 978-0-387-09751-0. DOI: 10.1007/978-0-387-09751-0_29. (Visited on 2026-03-09) (cit. on pp. 1, 3, 9, 10, 13, 58, 59).

[40] J. C. A. Leitão. "Topology Management for Unstructured Overlay Networks". PhD thesis. 2010 (cit. on pp. 8, 9, 12).

[41] J. Liang et al. "MON: On-demand Overlays for Distributed System Management". In: (2005-12) (cit. on pp. 12, 40).

[42] J. Lobo, R. Bhatia, and S. A. Naqvi. "A Policy Description Language". In: *AAAI/IAAI*. 1999. URL: https://api.semanticscholar.org/CorpusID:962745 (cit. on p. 25).

[43] E. Lupu and M. Sloman. "Conflicts in Policy-Based Distributed Systems Management". In: *IEEE Transactions on Software Engineering* 25.6 (Nov.-Dec./1999), pp. 852–869. ISSN: 00985589. DOI: 10.1109/32.824414. (Visited on 2025-01-22) (cit. on p. 24).

[44] M. L. Massie, B. N. Chun, and D. E. Culler. "The Ganglia Distributed Monitoring System: Design, Implementation, and Experience". In: *Parallel Computing* 30.7 (2004-07), pp. 817–840. ISSN: 01678191. DOI: 10.1016/j.parco.2004.04.001. (Visited on 2026-03-27) (cit. on p. 14).

[45] P. Maymounkov and D. Mazieres. "Kademlia: A Peer-to-Peer Information System Based on the XOR Metric". In: *Revised Papers from the First International Workshop on Peer-to-Peer Systems*. IPTPS '01. Berlin, Heidelberg: Springer-Verlag, 2002, pp. 53–65. ISBN: 3540441794 (cit. on pp. 1, 8).

[46] P. Maymounkov and D. Mazières. "Kademlia: A Peer-to-Peer Information System Based on the XOR Metric". In: *Peer-to-Peer Systems*. Ed. by G. Goos et al. Vol. 2429. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 53–65. ISBN: 978-3-540-44179-3 978-3-540-45748-0. DOI: 10.1007/3-540-45748-8_5. (Visited on 2025-01-24) (cit. on p. 7).

[47] H. Miranda, A. Pinto, and L. Rodrigues. "Appia, a Flexible Protocol Kernel Supporting Multiple Coordinated Channels". In: *Proceedings 21st International Conference on Distributed Computing Systems*. 21st International Conference on Distributed Computing Systems. Mesa, AZ, USA: IEEE Comput. Soc, 2001, pp. 707–710. ISBN: 978-0-7695-1077-4. DOI: 10.1109/ICDSC.2001.919005. URL: http://ieeexplore.ieee.org/document/919005/ (visited on 2025-01-22) (cit. on p. 29).

[48] M. Nofer et al. "Blockchain". In: *Business & Information Systems Engineering* 59.3 (2017-06), pp. 183–187. ISSN: 2363-7005, 1867-0202. DOI: 10.1007/s12599-017-0467-3. (Visited on 2025-02-05) (cit. on p. 1).

[49] P. Oreizy et al. "An Architecture-Based Approach to Self-Adaptive Software". In: *IEEE Intelligent Systems* 14.3 (1999-05), pp. 54–62. ISSN: 1094-7167. DOI: 10.1109/5254.769885. (Visited on 2025-01-22) (cit. on p. 21).

[50] M. Parashar and S. Hariri. "Autonomic Computing: An Overview". In: *Unconventional Programming Paradigms*. Ed. by D. Hutchison et al. Vol. 3566. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 257–269. ISBN: 978-3-540-27884-9 978-3-540-31482-0. DOI: 10.1007/11527800_20. (Visited on 2025-01-22) (cit. on pp. 19, 20).

[51] D. Paulico. "Metrics and Real-Time Monitoring in Distributed Systems". PhD thesis. 2025 (cit. on pp. 33, 34, 37, 59).

[52] L. D. Paulson. "Computer System, Heal Thyself." In: *Computer* 35.8 (2002), pp. 20–22 (cit. on p. 18).

[53] B. Pourebrahimi, K. Bertels, and S. Vassiliadis. "A Survey of Peer-to-Peer Networks". In: (2005) (cit. on pp. 1, 6, 7).

[54] *Prometheus - Monitoring System & Time Series Database.* 2012 (cit. on p. 54).

[55] F. Quin, D. Weyns, and O. Gheibi. "Decentralized Self-Adaptive Systems: A Mapping Study". In: *2021 International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*. 2021-05, pp. 18–29. DOI: 10.1109/SEAMS51251.2021.00014. arXiv: 2103.09074 [cs]. (Visited on 2026-03-27) (cit. on p. 2).

[56] S. Ratnasamy et al. "A Scalable Content-Addressable Network". In: *ACM SIGCOMM Computer Communication Review* 31.4 (2001-10), pp. 161–172. ISSN: 0146-4833. DOI: 10.1145/964723.383072. (Visited on 2025-01-25) (cit. on p. 16).

[57] R. Rodrigues and P. Druschel. "Peer-to-Peer Systems". In: *Communications of the ACM* 53.10 (2010-10), pp. 72–82. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/1831407.1831427. (Visited on 2025-01-24) (cit. on p. 7).

[58] L. Rosa, L. Rodrigues, and A. Lopes. *Appia to R-Appia: Refactoring a Protocol Composition Framework for Dynamic Reconfiguration*. di-fcul-tr-07-4. Department of Informatics, University of Lisbon, 2007-03. URL: http://hdl.handle.net/10455/2907 (cit. on pp. 2, 29).

[59] L. W. F. Rosa. "Self-Management of Systems Built from Adaptable Components". PhD thesis. Instituto Superior Técnico, 2012 (cit. on pp. 2, 19, 23, 27).

[60] A. Rowstron and P. Druschel. "Pastry: Scalable, Decentralized Object Location, and Routing for Large-Scale Peer-to-Peer Systems". In: *Middleware 2001*. Ed. by G. Goos et al. Vol. 2218. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 329–350. ISBN: 978-3-540-42800-8 978-3-540-45518-9. DOI: 10.1007/3-540-45518-3_18. (Visited on 2025-01-24) (cit. on pp. 8, 17).

[61] A. Rowstron et al. "Scribe: The Design of a Large-Scale Event Notification Infrastructure". In: *Networked Group Communication*. Ed. by G. Goos et al. Vol. 2233. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 30–43. ISBN: 978-3-540-42824-4 978-3-540-45546-2. DOI: 10.1007/3-540-45546-9_3. (Visited on 2026-03-27) (cit. on pp. 1, 17).

[62] R. Schollmeier. "A Definition of Peer-to-Peer Networking for the Classification of Peer-to-Peer Architectures and Applications". In: *Proceedings First International Conference on Peer-to-Peer Computing*. Linkoping, Sweden: IEEE Comput. Soc, 2002, pp. 101–102. ISBN: 978-0-7695-1503-8. DOI: 10.1109/P2P.2001.990434. (Visited on 2025-01-24) (cit. on pp. 5, 6).

[63] D. E. Seborg. *Process Dynamics and Control*. Fourth edition. Hoboken, NJ: Wiley, 2017. ISBN: 978-1-119-28591-5 (cit. on p. 19).

[64] M. Singh, C. Kumar, and P. Nath. "Challenges and Protocols for P2P Applications in Multi-hop Wireless Networks". In: *2018 Second International Conference on Computing Methodologies and Communication (ICCMC)*. Erode: IEEE, 2018-02, pp. 310–316. ISBN: 978-1-5386-3452-3. DOI: 10.1109/ICCMC.2018.8487911. (Visited on 2026-03-27) (cit. on p. 70).

[65] I. Stoica et al. "Chord: a scalable peer-to-peer lookup protocol for Internet applications". In: *IEEE/ACM Transactions on Networking* 11.1 (2003), pp. 17–32. DOI: 10.1109/TNET.2002.808407 (cit. on p. 8).

[66] D. Sykes et al. "From Goals to Components: A Combined Approach to Self-Management". In: *Proceedings of the 2008 International Workshop on Software Engineering for Adaptive and Self-Managing Systems*. Leipzig Germany: ACM, 2008-05, pp. 1–8. ISBN: 978-1-60558-037-1. DOI: 10.1145/1370018.1370020. (Visited on 2025-01-22) (cit. on p. 24).

[67] *TaRDIS: Trustworthy and Resilient Decentralised Intelligence for Edge Systems*. 2023 (cit. on pp. 3, 33, 83).

[68] S. Tarkoma. *Overlay Networks*. 0th ed. Auerbach Publications, 2010-02. ISBN: 978-1-4398-1373-7. DOI: 10.1201/9781439813737. (Visited on 2025-01-26) (cit. on p. 8).

[69] R. Van Renesse, K. P. Birman, and W. Vogels. "Astrolabe: A Robust and Scalable Technology for Distributed System Monitoring, Management, and Data Mining". In: *ACM Transactions on Computer Systems* 21.2 (2003-05), pp. 164–206. ISSN: 0734-2071, 1557-7333. DOI: 10.1145/762483.762485. (Visited on 2026-03-28) (cit. on p. 14).

[70] Y. Vandewoude et al. "Tranquility: A Low Disruptive Alternative to Quiescence for Ensuring Safe Dynamic Updates". In: *IEEE Transactions on Software Engineering* 33.12 (2007-12), pp. 856–868. ISSN: 0098-5589. DOI: 10.1109/TSE.2007.70733. (Visited on 2025-01-23) (cit. on p. 27).

[71] Q. H. Vu, M. Lupu, and B. C. Ooi. *Peer-to-Peer Computing: Principles and Applications*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010. ISBN: 978-3-642-03513-5 978-3-642-03514-2. DOI: 10.1007/978-3-642-03514-2. (Visited on 2025-01-24) (cit. on p. 5).

[72] W. Walsh et al. "Utility Functions in Autonomic Systems". In: *International Conference on Autonomic Computing, 2004. Proceedings.* New York, NY, USA: IEEE, 2004, pp. 70–77. ISBN: 978-0-7695-2114-5. DOI: 10.1109/ICAC.2004.1301349. (Visited on 2025-01-22) (cit. on pp. 24, 25).

[73] D. Weyns, S. Malek, and J. Andersson. "FORMS: Unifying Reference Model for Formal Specification of Distributed Self-Adaptive Systems". In: *ACM Transactions on Autonomous and Adaptive Systems* 7.1 (2012-04), pp. 1–61. ISSN: 1556-4665, 1556-4703. DOI: 10.1145/2168260.2168268. (Visited on 2026-03-27) (cit. on p. 2).

[74] Y. Xiao et al. "Distributed Consensus Protocols and Algorithms". In: *Blockchain for Distributed Systems Security*. Ed. by S. Shetty, C. Kamhoua, and L. Njilla. 1st ed. Wiley, 2019-04, pp. 25–50. ISBN: 978-1-119-51960-7 978-1-119-51962-1. DOI: 10.1002/9781119519621.ch2. (Visited on 2026-03-27) (cit. on p. 7).

[75] B. B. Yang and H. Garcia-Molina. *Designing a Super-Peer Network*. 2003 (cit. on p. 6).